THIS THESIS IS APPROVED BY:

Principal Supervisor: Ph.D. Vo Duc Hoang

Suggestions/Comments:

……………………………………………………………………………………
……………………………………………………………………………………
……………………………………………………………………………………
……………………………………………………………………………………
……………………………………………………………………………………
……………………………………………………………………………………
……………………………………………………………………………………
……………………………………………………………………………………
……………………………………………………………………………………
……………………………………………………………………………………
……………………………………………………………………………………
……………………………………………………………………………………
……………………………………………………………………………………
……………………………………………………………………………………
……………………………………………………………………………………
……………………………………………………………………………………
……………………………………………………………………………………
……………………………………………………………………………………
……………………………………………………………………………………
……………………………………………………………………………………
……………………………………………………………………………………
……………………………………………………………………………………
……………………………………………………………………………………
……………………………………………………………………………………
……………………………………………………………………………………
……………………………………………………………………………………
……………………………………………………………………………………
……………………………………………………………………………………
……………………………………………………………………………………
……………………………………………………………………………………

REFEREE'S COMMENTS

....
....
....
....
....
....
....
....
....
....
....
....
....
....
....
....
....
....
....
....
....
....
....
....
....
....
....
....
....
....
....
....

SUMMARY

Topic title: Build a job suggestion system and integrate a chatbot based on RAG

Student name: Dang Quang Nhat Linh

Student ID: 102200177

Class: 20TCLC_DT4

Nowadays, in the era of the Fourth Industrial Revolution, with the rapid development of information technology, the demand for human resources in enterprises is increasing day by day. Meanwhile, job seekers also desire positions that match their skills and expertise. However, traditional recruitment methods still face several challenges. Recruiters often struggle to identify applicants who meet mandatory requirements, and job seekers may spend a lot of time searching and applying for suitable jobs.

The main purpose of this project is to create a system that uses a chatbot assistant to optimize the user experience—helping users find suitable jobs and assisting with various concerns on the platform. This system aims to reduce the time and effort needed to secure a desired position. It covers all steps, including job postings, job applications, resume submissions, and even website integration. Simply put, the system is designed to overcome the limitations of traditional job searching. It is suitable for both part-time and full-time opportunities.

While this is not a new topic, I hope my solution can be helpful making it easier for users to find suitable jobs and enabling enterprises to manage applicants more efficiently. Ultimately, this project contributes to the development of digital services in Vietnam and around the world.

DA NANG UNIVERSITY
UNIVERSITY OF SCIENCE AND TECHNOLOGY
FACULTY INFORMATION TECHNOLOGY

SOCIALIST REPUBLIC OF VIETNAM
Independence - Freedom - Happiness

GRADUATION PROJECT COMMENT

I. General information:

1. Student name: DANG QUANG NHAT LINH
2. Class: 20TCLC_DT4 Student ID: 102200177
3. Topic title: BUILD A JOB SUGGESTION SYSTEM AND INTEGRATE A CHAT BOT BASED ON RAG
4. Instructor: Ph.D. VO DUC HOANG Academic title/ degree: Ph.D.

II. Reviews of graduation project

1. About the urgency, novelty and usability of the topic: (2 points)

…………………………………………………………………………………………

…………………………………………………………………………………………

2. About the results of solving the tasks required by the project: (4 points)

…………………………………………………………………………………………

…………………………………………………………………………………………

3. About the form, structure and layout of the graduation project: (2 points)

…………………………………………………………………………………………

…………………………………………………………………………………………

4. The topic includes scientific value/article/problem solving of the enterprise or school: (1 point)

…………………………………………………………………………………………

…………………………………………………………………………………………

5. Existing shortcomings need to be supplemented or modified:

…………………………………………………………………………………………

…………………………………………………………………………………………

III. Spirit and attitude of the student (1 point):

………………………………………………………………………………………..

………………………………………………………………………………………..

IV. Evaluation:

1. Evaluation point: …. /10
2. Suggest: Defense permitted/ Edit to defend/ Defense not permitted

Da Nang, date……month……. 2025

Instructor

DA NANG UNIVERSITY

THE SOCIALIST REPUBLIC OF VIETNAM

UNIVERSITY OF SICIENCE AND TECHNOLOGY
FALCUTY: INFORMATION & TECHNOLOGY

Independence - Freedom - Happiness

GRADUATION PROJECT REQUIREMENTS

Student Name: DANG QUANG NHAT LINH Student ID: 102200177
Class: 20TCLC_DT4 Major: Information Technology

1. *Topic title:* BUILD A JOB SUGGESTION SYSTEM AND INTEGRATE A CHAT BOT BASED ON RAG.
2. *Project topic: ☐ has signed intellectual property agreement for final result*
3. *Initial figure and data:*

For the data to server as recommendation project, it was crawled from official websites of Indeed.

Content of the explanations and calculations:

Content of the explanations contains 5 parts

- **Introduction**: This chapter gives information about the context and purpose of the project as well as giving the scope of the problems which will be focused on the thesis.
- **Chapter 1**: Theories and Technologies
- **Chapter 2**: Analysis and Design
- **Chapter 3**: Implementation and Evaluation
- **Chapter 4**: Conclusion

4. Drawing, charts (specify types and size of drawing)
   - Use case diagram
   - Activity diagram
   - Sequence diagram
   - Architecture structure diagram
5. *Instructor*:

Ph. D Vo Duc Hoang, Information Technology Faculty, Danang University of Science and Technology, The University of Danang, Vietnam.

6. *Date of assignment: Dec 16th, 2024.*

7. *Date of completion:* *Jun 5th, 2025.*

Danang, Jun, 2025

Head of Division…………………. **Instructor**

PREFACE

To achieve good results for this project, I have received enthusiastic help from many people. With deep affection and sincerity, I want to express my gratitude to all the people who have helped me in my study and research.

I would like to thank the teachers at Da Nang University of Science and Technology in general and the faculty of information technology in particular, who have imparted valuable knowledge through classroom hours during the past years, creating a premise for me to do well on this graduation project. And especially, I would like to express my deep gratitude to **Ph. D Vo Duc Hoang** for enthusiastically guiding, sharing valuable experiences and facilitating in many ways for me to successfully complete this project.

With the limitation of knowledge, the report may face unwanted errors, I really hope for the advice of teachers and everyone so that I can learn from it. Sincerely,

COMFIRMATION

I guarantee:

1. The contents of this senior project are performed by myself following the guidance of instructor **PhD. Vo Duc Hoang.**
2. All references used in this senior project thesis are quoted with the author's name, project name, time and location to publish clearly and faithfully.
3. All invalid copies educated statue violation or cheating will be borne the full responsibility by myself.

Student Performed

TABLE OF CONTENTS

FIGURES

TABLES

LIST OF SYMBOLS, ACRONYM

SYMBOL:

ACRONY

No	Acronym	Explanation
1	OOP	Object Oriented Programming
2	FP	Functional Programming
3	AI	Artificial Intelligence
4	SQL	Structured Query Language
5	RAG	Retrieval-Augmented Generation
6	LLM	Large Language Model
7	TF-IDF	Term Frequency - Inverse Document Frequency

INTRODUCTION

1. Purpose for doing thesis

Today, the number of job-seeking people is increasing. Along with that, the demand for human resources of enterprise in this field is increasing day by day. Accordingly, the workload is also diverse and rich. However, besides that, there still exist some problems of traditional recruitment, as Enterprises cannot determine the applicants that match with their requirements, and it may take a lot of time for applicants to find and apply suitable jobs. Moreover, with the strong development of AI, large corporations are racing to develop their own models. Taking advantage of that, I have integrated chat bot assistant to assist users, optimize users to suggest users' suitable jobs and guild users to use web applications.

2. Scope and objective

Based on that idea, I have chosen this topic with the goal of solving those problems. The main objective is to create software integrated chat bot system that allows candidates to quickly find suitable jobs according to the requirements of the jobs as well as their skill. All steps from job posting, applying, submitting resumes, even the website system for candidate roles, managing jobs managers, managing applicants, promoting jobs and their profiles for the enterprise role. And the manager all for the administrator.

3. Methods

Observation method: Observing the general situation and the problems that enterprise is facing.

Theories analyzing method: Research about related papers, analyze system and domain business.

Experimental method: Use current applications is related to the problem of providing the necessary functionality for my application in this project.

Modeling Method: After implementing and getting information from above methods, I built a model by drawing diagrams and figures to show the big picture to develop the final usable application.

4. Main functions in application

- Users can search for and apply for any job they want.
- Candidates and enterprises can quickly register and log in to the application.
- Includes a system management function for enterprises and administrators.
- Users can interact with a chatbot assistant.
- Users will receive notifications from the system.
- Users will also receive certain information via email.
- Enterprises can boost their job postings to top visibility and manage their listings.
- Both enterprises and administrators can view statistics through a dashboard.
- Enterprises can register for a subscription to purchase points.

Chapter 1 : THEORIES AND TECHNOLOGIES

1.1 Introduction

This system integrates multiple advanced technologies, particularly in the field of artificial intelligence and machine learning, to deliver intelligent, responsive, and user-focused functionality. In this section, I will present the key concepts and technologies used throughout the project.

The application incorporates two distinct AI components, each designed to serve a specific purpose:

1.1.1 AI-Powered Chatbot using embedding and LLM (RAG)

The first AI component is an intelligent chatbot system built using the LangChain framework. It utilizes text embedding techniques to convert user queries and documents into vector representations, allowing for semantic understanding. These embeddings are then used in combination with a large language model (LLM) in a Retrieval-Augmented Generation (RAG) architecture. This enables the chatbot to retrieve relevant context from a document database and generate accurate, human-like responses based on both the retrieved data and the power of the LLM.

1.1.2 AI for related job recommendations using TF-IDF and hybrid scoring

The second AI module focuses on job recommendation. It applies the TF-IDF (Term Frequency-Inverse Document Frequency) algorithm to analyse job descriptions and user profiles by extracting important keywords. Based on these keyword vectors, the system calculates the similarity between jobs and users. A hybrid scoring method is then used, combining content-based filtering (e.g., from TF-IDF) with collaborative features (e.g., user interactions or behaviours) to generate a ranked list of related job suggestions that are both personalized and relevant.

These two AI components work independently but complement each other, enhancing the overall functionality of the platform: one improves user interaction via conversational AI, and the other provides intelligent job discovery through advanced recommendation techniques.

1.2 Retrieval-Augmented Generation system

Retrieval-Augmented Generation (RAG) is a powerful architecture that combines two core components of natural language processing: retrieval and generation. Unlike traditional language models that rely solely on pre-trained knowledge, RAG enhances the model's capabilities by allowing it to dynamically retrieve external information from a document store at the time of answering a query.

1.2.1 Architecture Overview

A typical RAG system consists of two main stages:

- **Retriever**: This module searches for relevant documents or pieces of information from a knowledge base. It often uses text embedding models to convert both the query and documents into vectors and applies similarity metrics (e.g., cosine similarity) to find the most relevant documents.
- **Generator**: Once relevant documents are retrieved, the generator—typically a **large language model** (LLM)—uses both the query and the retrieved context to produce a coherent and accurate answer.

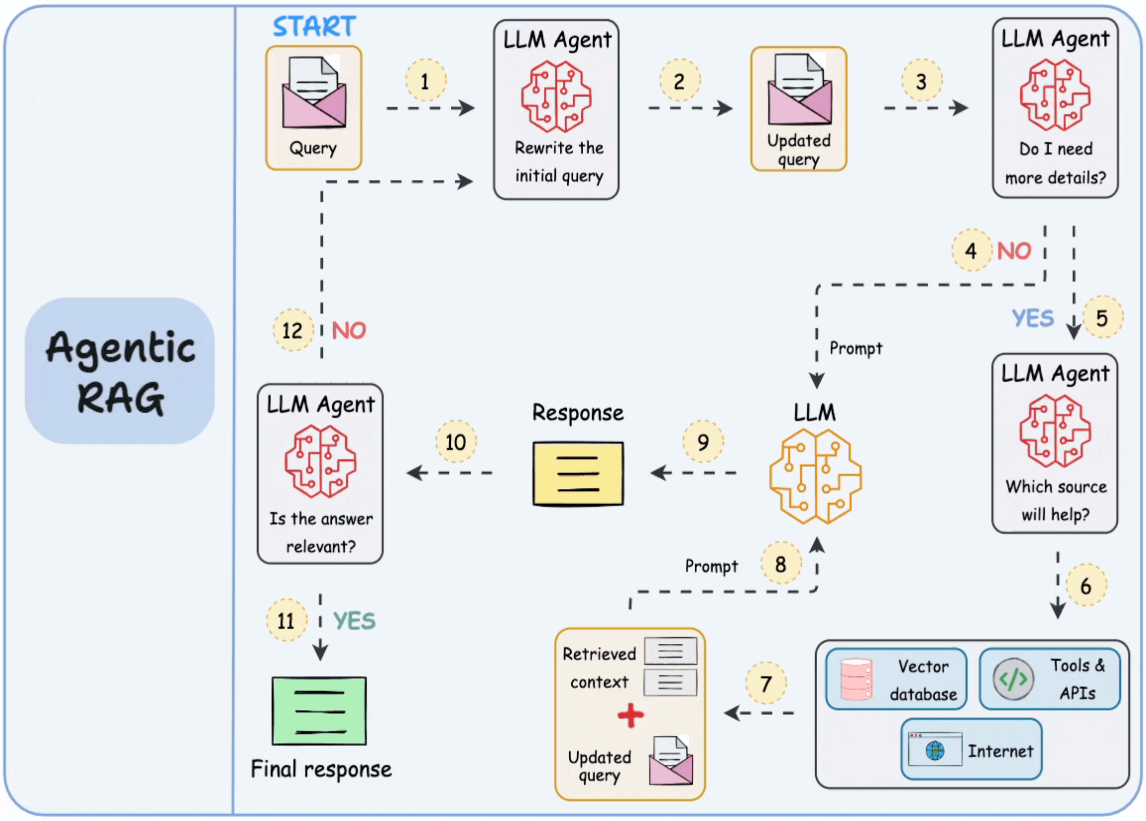

Figure 1: Agentic RAG architecture

1.2.2 Implementation in the Chatbot

The chatbot in this system is built using the LangChain framework, which supports seamless integration of RAG components. The implementation process includes:

- Creating a document vector store using a text embedding model (e.g., Sentence Transformers).
- Using similarity search to retrieve relevant documents based on user queries.
- Passing the retrieved documents and user query into an LLM (e.g., GPT or similar) to generate the final response.

This setup enables the chatbot to respond to user inquiries with high contextual relevance, particularly in domains such as job searching, application instructions, or enterprise-specific information.

1.2.3 Pros and Cons

Table 1: Pros and Cons of using RAG

Pros	Cons
Enhances language model responses with up-to-date or domain-specific information	More complex to implement compared to standalone LLMs
Reduces hallucination by grounding answers in retrieved documents	Depends on the quality and relevance of the document store
Enables dynamic updates of knowledge without retraining the language model	Retrieval step may return irrelevant or noisy documents if not properly tuned
Scalable to large corpora using vector databases and embedding search	Requires good vector indexing (e.g., FAISS) and text embedding for effective performance

1.3 Related jobs system

The related jobs system in this project is a specialized recommendation engine designed to suggest relevant job postings to users based on content similarity and user interaction data. It plays a crucial role in helping users discover opportunities that match their skills and interests efficiently.

Unlike traditional recommendation systems that focus on general products or ratings, this system emphasizes job matching, where textual content (job descriptions, skills, titles, etc.) is central. The system is built using a combination of TF-IDF (Term Frequency-Inverse Document Frequency) and a hybrid scoring mechanism that balances content relevance with other factors such as popularity or user engagement.

1.3.1 TF-IDF-Based Content Similarity

TF-IDF is a statistical measure used to evaluate the importance of a word in a document relative to a corpus. In the context of job recommendation:

- Each job posting is treated as a document.
- Important keywords (skills, technologies, responsibilities) are extracted from the job description.
- A TF-IDF vector is generated for each job, forming a high-dimensional representation of its content.

The similarity between two jobs (or between a job and a user profile) is calculated using cosine similarity between their TF-IDF vectors. Jobs with similar textual content are considered related.

Term frequency works by looking at the frequency of a *particular term* you are concerned with relative to the document. There are multiple measures, or ways, of defining frequency.

$$t\int i,j = \frac{n_{i,j}}{\sum_{k} ni,j}$$

Figure 2: Formula for calculating IF

Inverse document frequency looks at how common (or uncommon) a word is amongst the corpus. IDF is calculated as follows where ***t*** is the term (word) we are looking to measure the commonness of and ***N*** is the number of documents (d) in the corpus (D).. The denominator is simply the number of documents in which the term, ***t***, appears in.

$$idf(i,\ D) = \log log\ (\frac{N}{count\ (d \in D : t \in d)})$$

Figure 3: Formula for calculating IDF

1.3.2 Hybrid Scoring Method

While TF-IDF captures textual similarity, it may overlook user behaviour patterns or job trends. Therefore, a hybrid scoring approach is applied to improve recommendation accuracy.

The **Hybrid Score** is computed as a combination of:

- Content Similarity Score (from TF-IDF)
- Interaction Score (based on click, view, or apply data)
- Popularity Score (based on how often a job is interacted with by all users)

1.3.3 Pros and Cons

Table 2: Pros and Cons of using TF-IDF

Pros	Cons
Simple and easy to implement	Does not capture semantic meaning or context
Computationally efficient and lightweight	Treats each word independently (bag-of-words model)
Effective for basic similarity tasks (e.g., document or job matching)	Sensitive to vocabulary mismatch (e.g., synonyms not recognized)
Requires no labeled data or training	Can perform poorly on short or ambiguous text

1.4 Technical constraint

1.4.1 Next.js

❖ **Introduction**

Next.js is a popular open-source React framework developed by Vercel. It enables developers to build server-side rendered (SSR) and statically generated web applications using React. Next.js provides features like file-based routing, automatic code splitting, API routes, and built-in support for performance optimization, making it suitable for modern web development.

❖ **Architecture and Environments**

Next.js runs on the Node.js runtime environment and supports both server-side and client-side rendering. It leverages React components for UI building and uses a hybrid rendering approach — combining Static Site Generation (SSG), Server-Side Rendering (SSR), and Client-Side Rendering (CSR) where necessary. Understanding how Next.js handles API routes, middleware, and rendering behavior is essential for optimizing performance and building scalable applications.

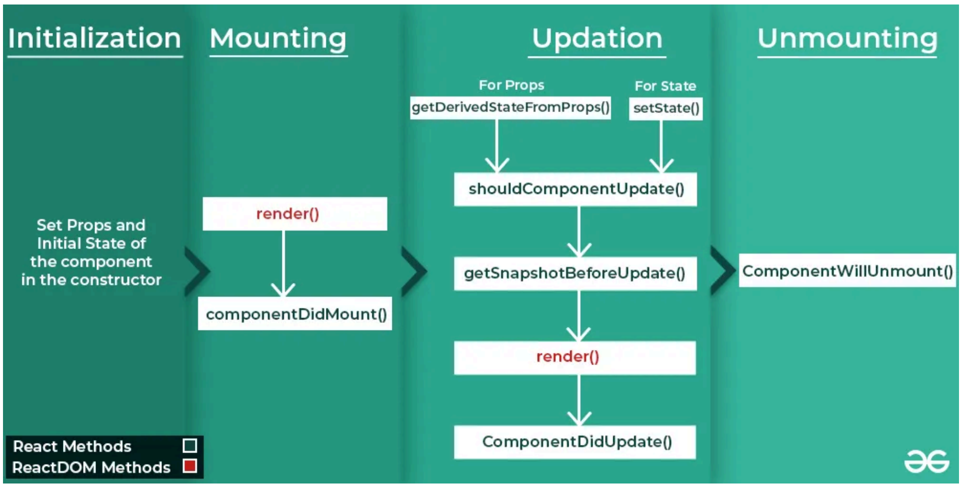

Figure 4: Life cycle overview of Next.js

- ❖ **Use case**

Build a server-client architecture for an application with UI responsibilities.

1.4.2 NestJs

- ❖ **Introduction**

Nest (NestJs) is a framework for building efficient, scalable NodeJS server-side applications. It uses progressive JavaScript, is built with and fully supports Typescript and combine elements of OOP(Object oriented Programming), FP (Functional Programming)

Especially, Essentially, NestJS is a layer on top of Node that has supercharged methods and implementations that can help us write server-side applications quick and easy. Nest is very convenient to content all your needs.

- ❖ **Why we need NestJs when we have Node?**

  - Nest provide a layer of abstraction on top of node, which means that it can now leverage the features of Node as well as expose APIs for better performance and efficiency
  - Nest also provide compatible modules interacting with ORM's which can take advantage of working with database

- ❖ **Use case**

Build a server-side application with modular and scalable architecture.

1.4.3 FastAPI

❖ **Introduction**

FastAPI is a modern, high-performance web framework built with Python for developing web applications and APIs. Based on Python standards like type hinting, FastAPI helps developers quickly build maintainable and efficient RESTful APIs. The goal of FastAPI is to make API development fast, simple, and reliable, while offering an excellent developer experience.

❖ **Architecture and environment**

FastAPI is built on powerful libraries such as Starlette (for web handling) and Pydantic (for validation and serialization). When launching a FastAPI application, it starts a development server with asynchronous processing, automatically generates API documentation using Swagger UI or ReDoc, and validates input data based on Python type hints.

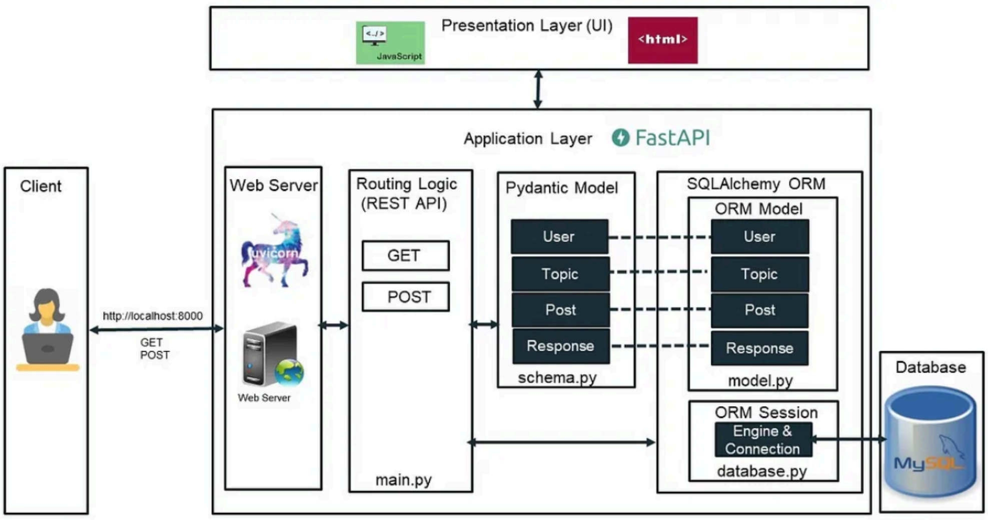

Figure 5: FastAPI architecture

❖ **Use case**

Build a high-performance backend API for AI-powered services.

1.4.4 Docker

❖ **Introduction**

Docker is an open-source platform designed to automate the deployment, scaling, and management of applications using containerization. It allows developers to package an application and all of its dependencies into a standardized unit called a container, which ensures that the application runs consistently across different environments.

❖ **Architecture and Environment**

Docker uses a client-server architecture. The Docker client communicates with the Docker daemon, which is responsible for building, running, and managing containers. Applications are defined in a Dockerfile, which includes instructions for building the container image, such as installing dependencies and setting up the environment.

Once the image is built, it can be deployed consistently across development, staging, and production environments using Docker Engine.

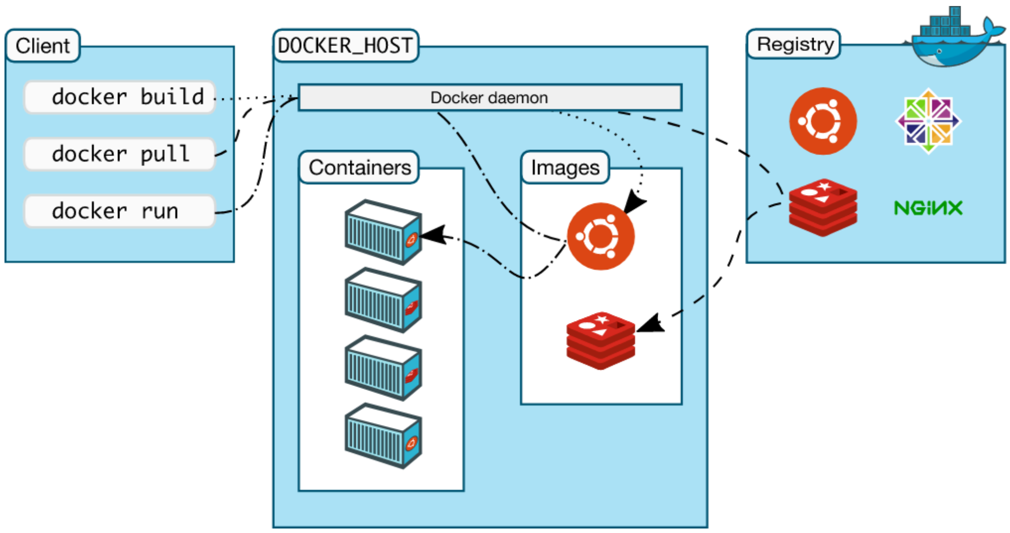

Figure 6: Docker architecture

❖ **Use case**

Containerize and deploy applications consistently across different environments.

1.4.5 Langchain

❖ **Introduction**

LangChain is open-source framework designed to simplify the development of applications powered by llms such as OpenAI's GPT, Anthropic's Claude, or Hugging Face models. LangChain provides tools to help developers build intelligent, multi-step, context-aware applications by connecting language models with external data sources and computation environments.

❖ **Architecture and Environment**

LangChain introduces a modular architecture that allows developers to combine different components into complex workflows. These include:

- Prompt Templates – Predefined structures for consistent LLM interaction.

- Chains – Sequences of steps (e.g., prompts → model calls → actions).
- Agents – LLMs that can decide what actions to take (such as querying a database or calling an API) based on user input.
- Memory – Enables applications to remember past interactions and use them in future prompts.

LangChain can run in various environments, including local machines, cloud services, or containerized platforms like Docker.

❖ **Use case**

Develop agentic systems that combine retrieval-augmented generation (RAG) with reasoning capabilities.

1.4.6 Third-party services (Paypal, Google, Facebook)

❖ **Introduction**

Modern web applications often require integration with third-party platforms to extend functionality, improve user experience, and handle services such as authentication, payments, and analytics. Among the most commonly integrated platforms are PayPal**,** Google**,** and Facebook, each providing robust APIs to support these needs.

These third-party services help developers offload complex functionalities like payment processing, OAuth-based login, and user data synchronization, allowing them to focus more on the core features of their application.

❖ **Architecture and Environment**

Third-party integrations typically follow client-server architecture, where the frontend (e.g., Next.js) interacts with the service provider's APIs, and the backend (e.g., Fast API) handles secure data processing, storage, and validation.

- OAuth 2.0 – Used by Google and Facebook for secure user authentication and authorization.
- REST APIs / Webhooks – Used by PayPal to handle payment events and updates in real time.

These integrations often require secure API keys, token management, and callback URL configuration to ensure safe and reliable communication.

❖ **Use case**

Enable secure user authentication through third-party identity providers.

1.4.7 Firebase cloud messaging

❖ **Introduction**

Firebase Cloud Messaging (FCM) is a cross-platform messaging solution provided by Firebase that enables developers to send push notifications and in-app messages to users on Android, iOS, and web platforms. FCM supports both targeted messages (e.g., user-specific updates) and broadcast messages (e.g., promotions or alerts), making it a powerful tool for enhancing user engagement and real-time communication

❖ **Architecture and Environment**

FCM operates on a client-server model where messages are sent from a backend server (or Firebase Console) to client applications via device tokens. These tokens uniquely identify user devices and are used to route notifications accurately.

The typical flow includes:

- Client app registers with FCM and receives a unique device token.
- Backend stores this token and uses it to send push notifications via FCM HTTP or legacy APIs.
- FCM routes the message to the correct device using its infrastructure.

❖ **Use case**

Send real-time push notifications to users across platforms.

1.4.8 Redis

❖ **Introduction**

Redis (REmote DIctionary Server) is an open-source, in-memory data structure store, used as a database, cache, and message broker. It supports a wide range of data structures such as strings, hashes, lists, sets, sorted sets, bitmaps, hyperloglogs, and geospatial indexes. Known for its high performance, low latency, and simplicity, Redis is widely used in scenarios that require fast access to data, real-time analytics, pub/sub messaging, and caching.

Redis is written in ANSI C and typically operates as a single-threaded process. It is optimized for speed, often serving millions of read and write operations per second in production environments. Redis also supports persistence through snapshots and append-only files (AOF), allowing data to survive server restarts.

❖ **Architect and environment**

Redis follows a client-server model and uses a single-threaded, event-driven architecture. The core architectural components include:

- Redis Server: The main daemon process that handles client requests.
- Clients: Applications that connect to the Redis server to perform operations.
- Command Dispatcher: Redis processes commands sequentially using an event loop, ensuring simplicity and high throughput.
- Data Structures: Stored in memory using efficient algorithms and custom encoding to optimize for memory usage and speed.

❖ **Use case**

Ideal for caching frequently accessed data, managing cronjob scheduling, and implementing message queues to enable efficient and scalable application workflows.

1.4.9 Postgres and vector database extensions

❖ **Introduction**

PostgreSQL is a powerful, open-source object-relational database system known for its robustness, extensibility, and compliance with SQL standards. With the rise of AI and large language model (LLM) applications, PostgreSQL has evolved to support vector similarity search through various extensions, making it suitable for storing and querying high-dimensional vector embeddings.

Docker image: ankane/pgvector

❖ **Architecture and Environment**

PostgreSQL supports vector search through specialized extensions that integrate vector storage and similarity search into the traditional relational database structure. Common extensions include:

- pgvector – A popular PostgreSQL extension that enables storage and similarity search of vectors (e.g., embeddings from LLMs).
- PostGIS – Adds support for geographic objects, useful in geospatial queries.
- TimescaleDB – An extension for time-series data (not vector-specific, but often used alongside).

The pgvector extension allows storing vectors as native data types and performing operations such as cosine similarity, inner product, and Euclidean distance efficiently using index structures like IVFFlat.

❖ **Use case**

Store and manage relational data with strong consistency and complex querying.

Chapter 2 : SYSTEM OVERVIEW

1.1 Descriptive overview of objects

The system has three main user Groups

- **Administrator:** A person has the highest privilege in system. Admins operate functions of user management, enterprise management, item management, admin still take responsibly for security tracking and mange tools of the system and statistics.
- **Enterprise: (**recruiter) When Enterprise register new user and accepted by system, they could change the information of company, post new jobs, manage the jobs, boost jobs, and manage the applicants applied for job.
- **User**: People who are looking for a job in the system. Once a user has completed registration and authentication, they can update personal information, upload their CV, interact with the chatbot assistant, and save favorite enterprises or job postings.

1.2 Descriptive functionality

1.2.1 For Admin

- ❖ Login with admin: User must be logged in with an Admin account to use all the functions and benefits that this role possesses.
- ❖ Statistics
  - o Admin can statistically total jobs, users, enterprises, and revenue.
- ❖ User Management
  - o Admin has the access to view, edit, delete, update, and search any user account.
  - o Admin confirms the information comes from a company and can accept new one registration coming from that company. Then an email will be sent from the system to provide the password for the registered company account.
- ❖ Enterprise management
  - o Admin can view, approve, reject form enterprise registration.
  - o Admin can view, edit, block enterprise
- ❖ Job management
  - o Admin can view, edit, delete, search any existing job in system.
- ❖ Items Management
  - o Admin can view, edit, delete, and update any existing categories in

system.

- Admin can view, edit, delete, and update any existing tags in system.

1.2.2 For enterprise

- ❖ Register
  - Enterprises must complete the registration steps and must be confirmed by the administrator.
- ❖ Login
  - Enterprise must sign in with enterprise accounts to use the system with all the features of the enterprise.
- ❖ Profile management
  - Enterprise can view, edit their profile
- ❖ Job management
  - Enterprise can view list of jobs, search, edit, or delete any the job posted on the company site.
- ❖ Applicant management
  - Enterprise can view applicant's listings, search, edit, delete an applicant.

1.2.3 For candidate

- ❖ Register
  - Job seekers must complete registration operations to have a user account in the system.
- ❖ Login
  - Job seekers must login into the system with a valid account to use the features for job seekers.
- ❖ Manage profile
  - Build up their profile, and social network.
  - Register enterprise form if they want to be enterprise.
- ❖ Chat bot assistant
  - The chat suggests jobs for candidates based on user behaviour.
  - Interact chat bot with 4 tools: search jobs, search enterprise, website information.
- ❖ Search for a job
  - Candidates can search for job information by name, location, salary, job requirements.
- ❖ Posting and mange resumes

- o Candidates can upload resumes to the system, This CV will be store in the AWS S3.

❖ Jobs favorite management

- o Job seekers can see the recent job view, favourite job, applied job
- o They can also see the status of that job, and will be notified if it is approved or rejected by the Enterprise or when the job has ended the recruitment period

1.3 Use-case diagram

1.3.1 General

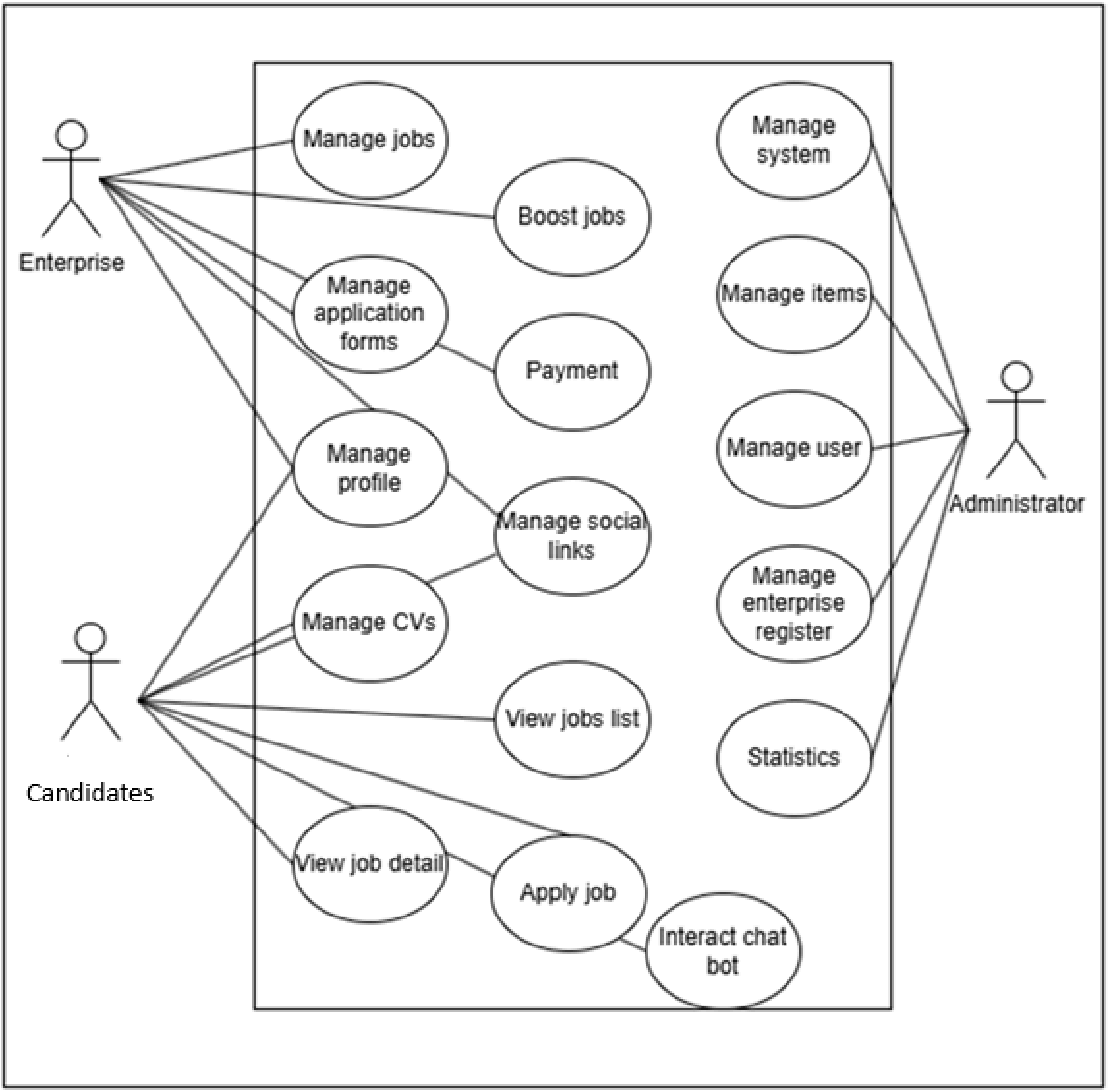

Figure 7: General Use-case diagram

1.3.2 Admin

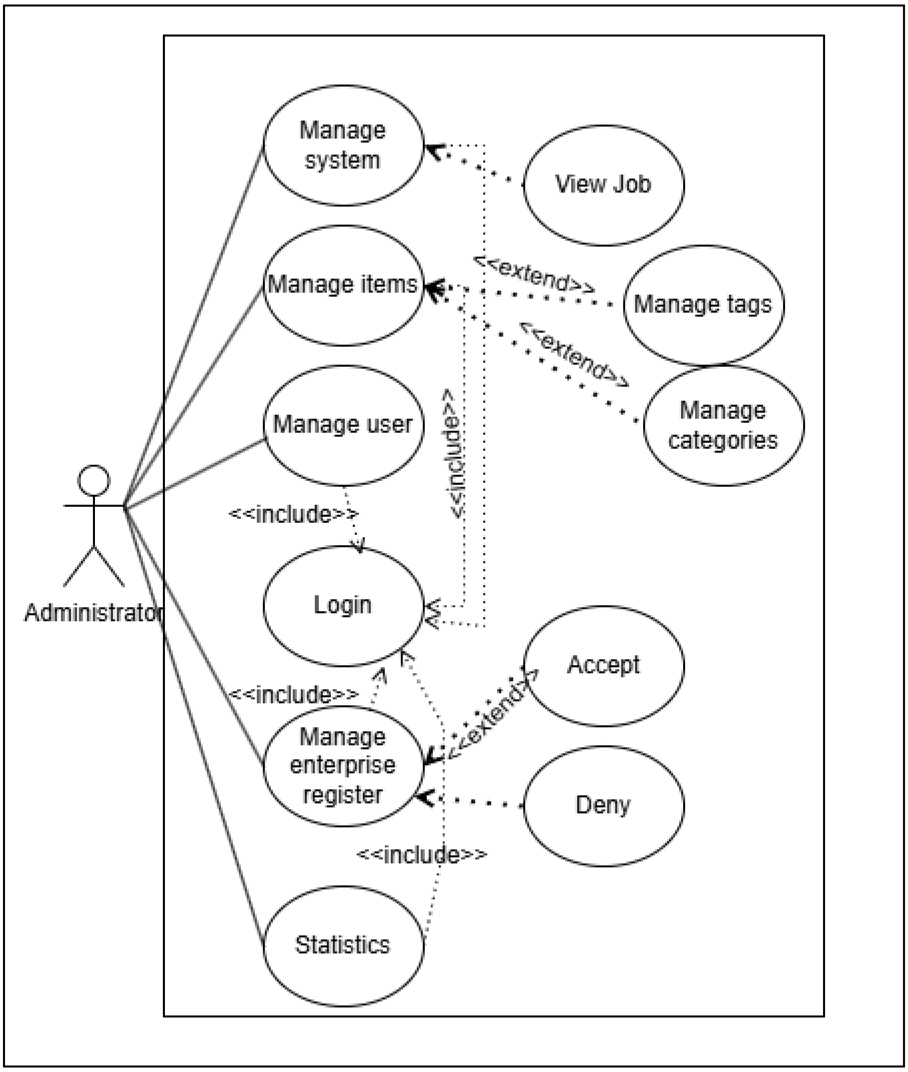

Figure 8: Use-case for Admin features

1.3.3 Candidates

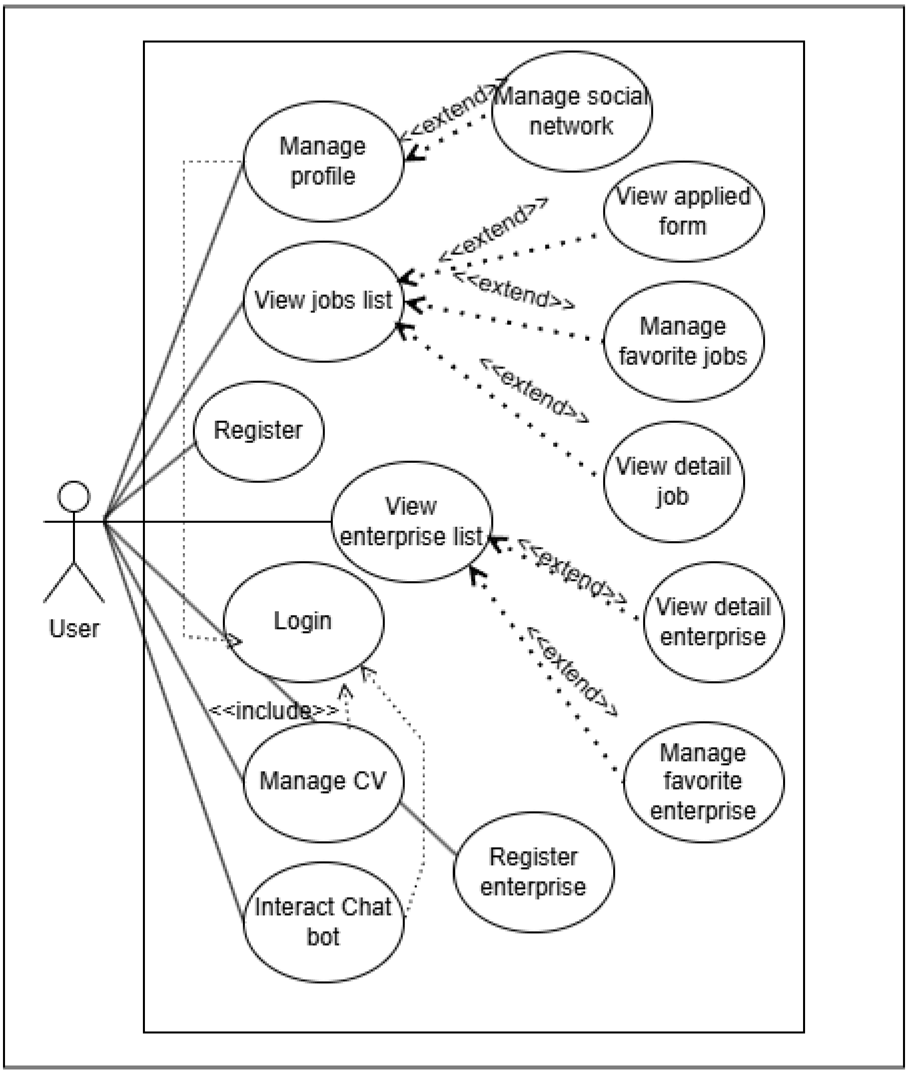

Figure 9: Use-case for User features

1.3.4 Enterprise

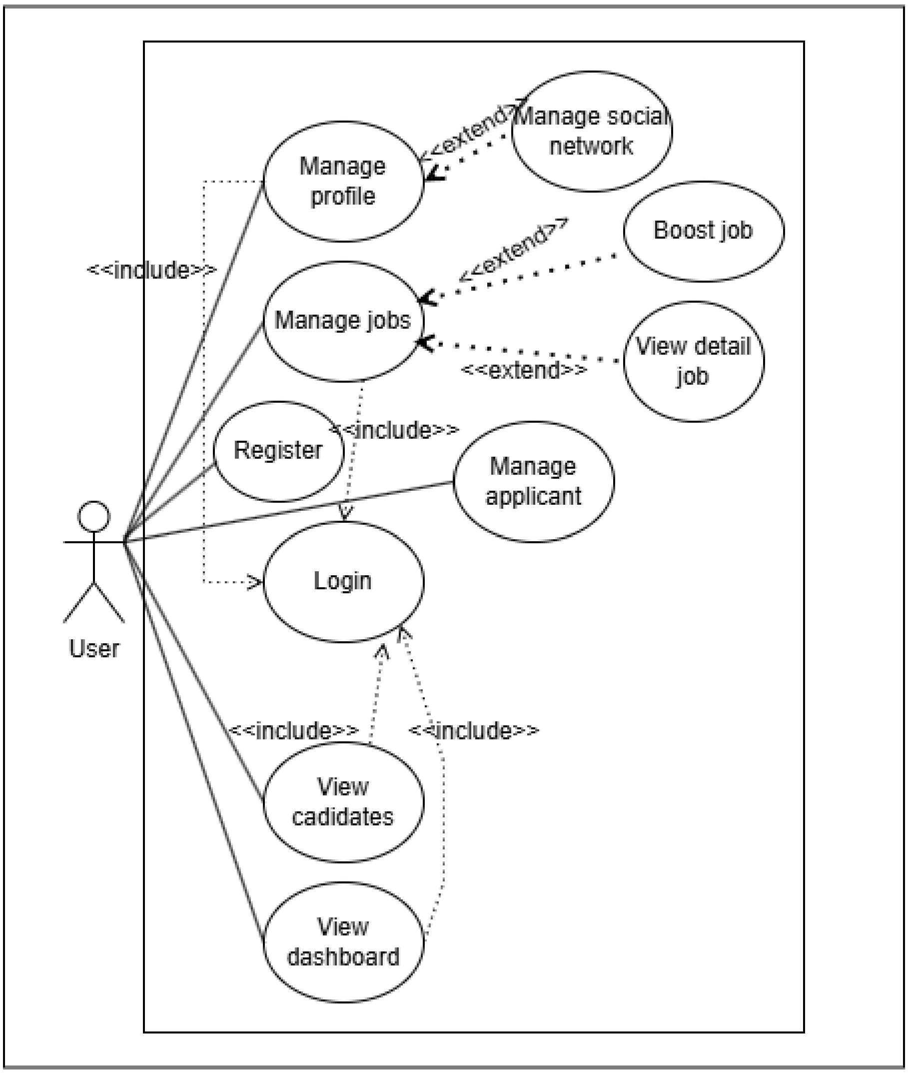

Figure 10: Use-case for Enterprise feature

1.3.5 Authentication

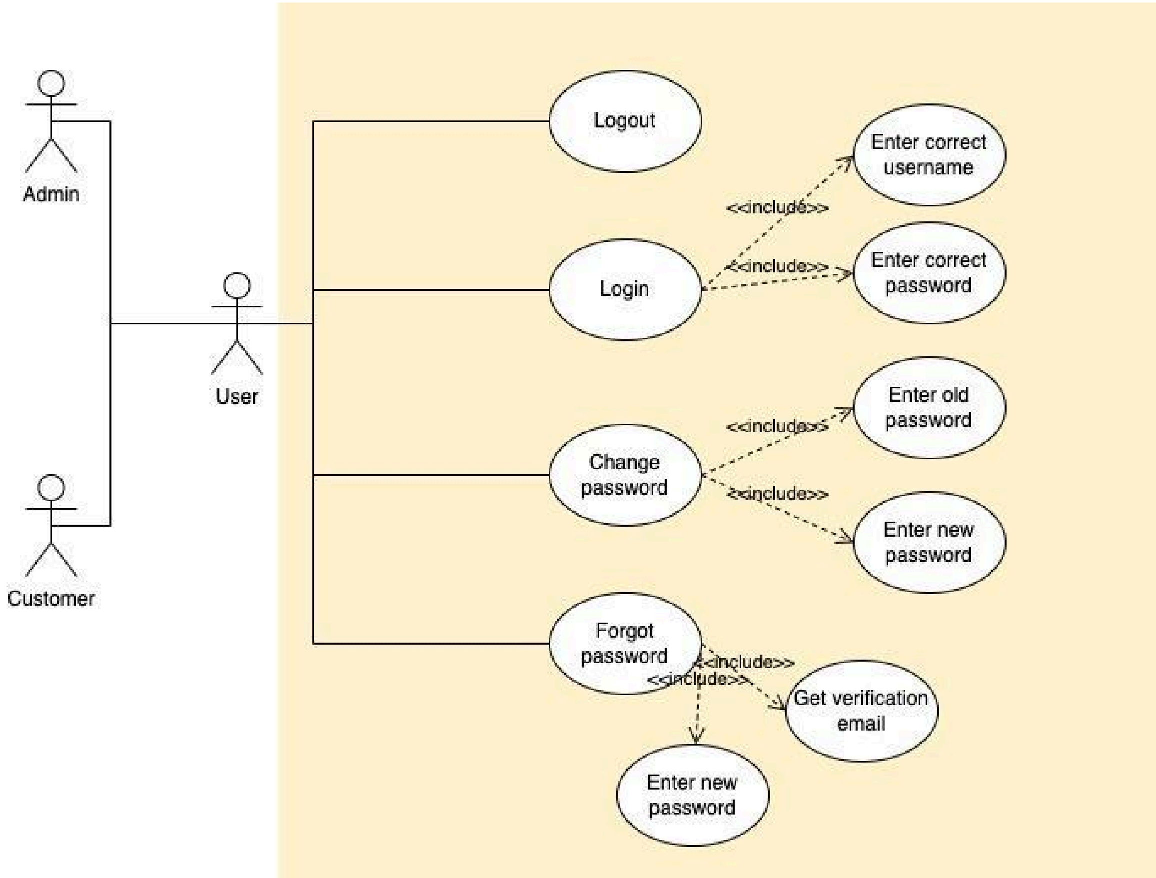

Figure 11: Use-case for Authenticate feature

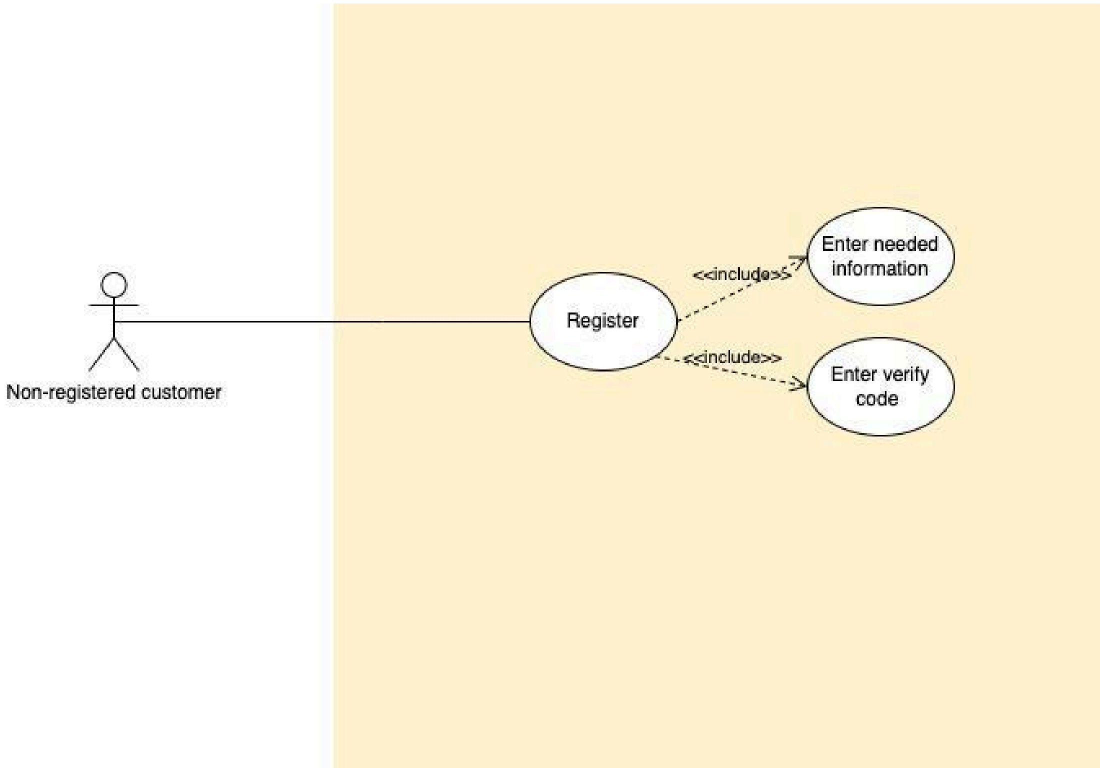

Figure 12: Use-case for registration feature

0.4 Activity diagram

0.4.1 Login

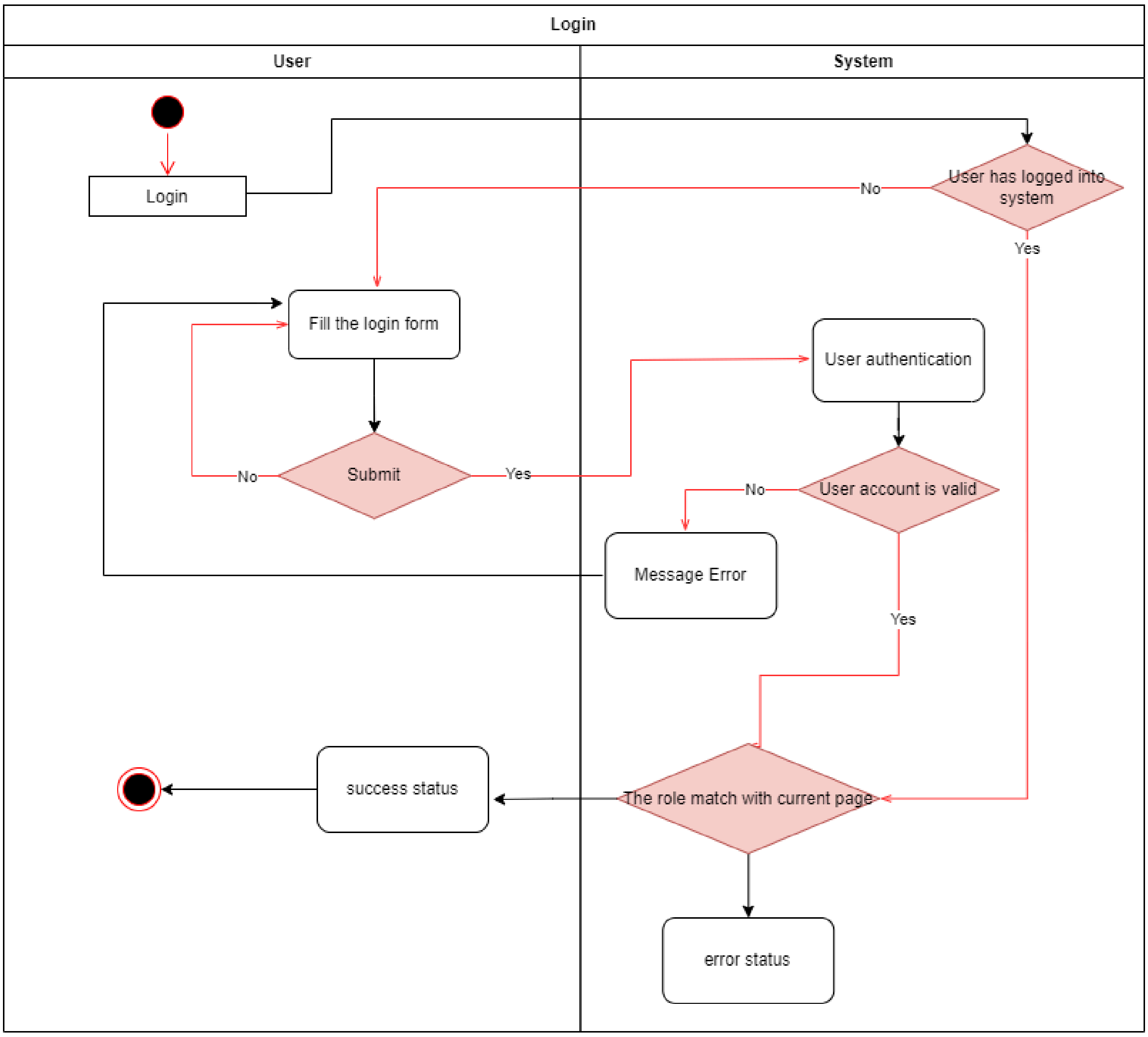

Figure 13: Activity diagram for login

0.4.2 Registration

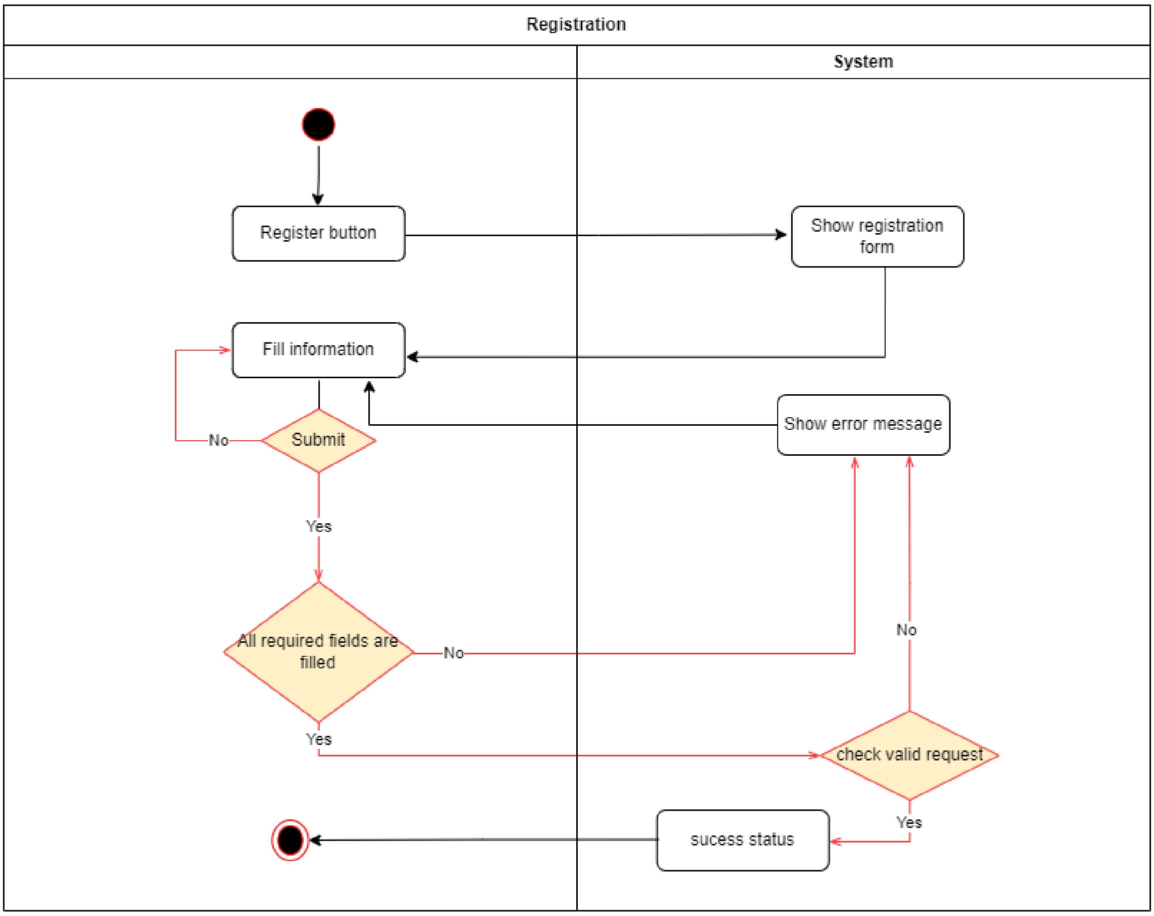

Figure 14: Activity diagram for registration

0.4.3 Create Job

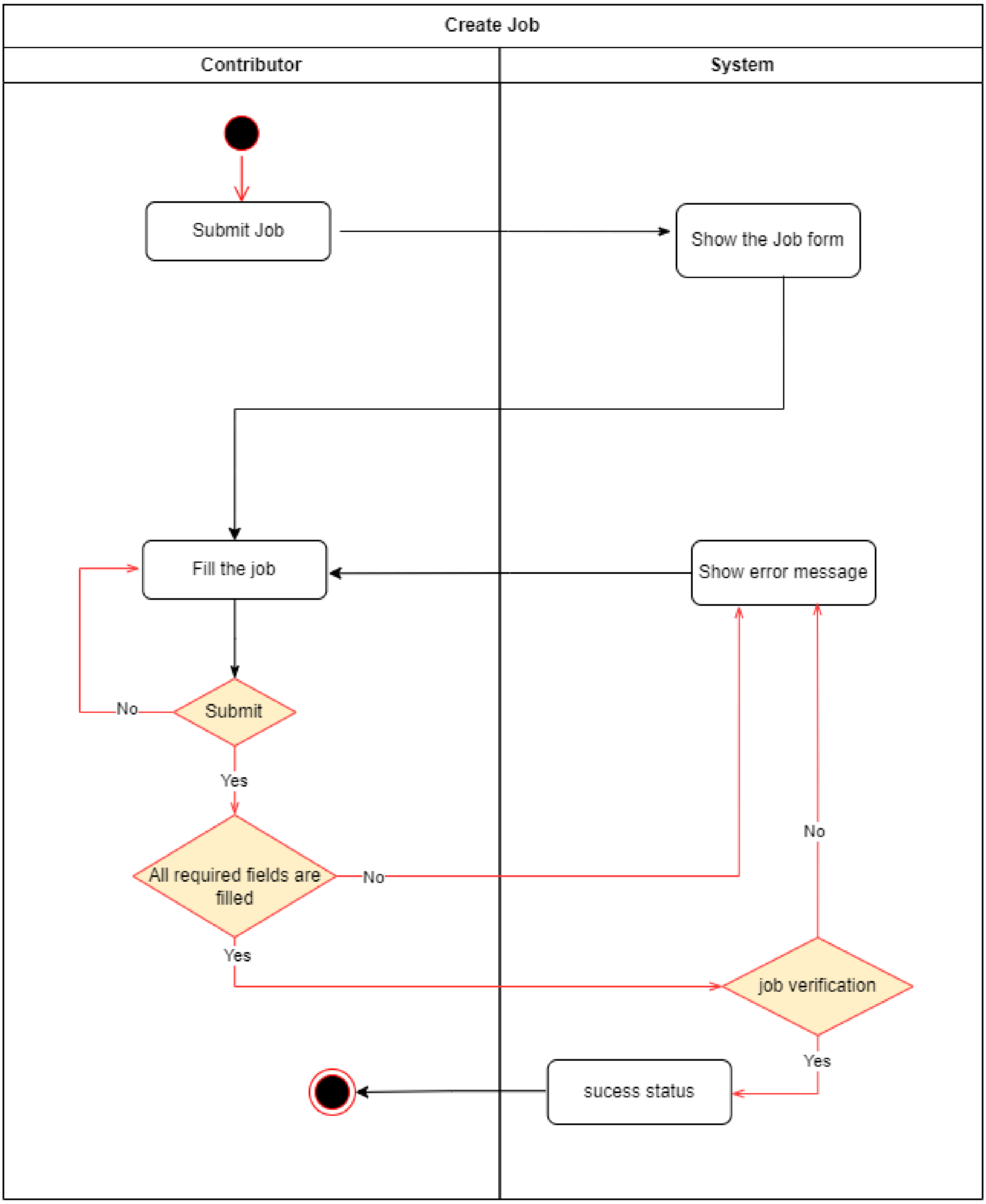

Figure 15: Activity diagram of Job creating

0.4.4 Update Job

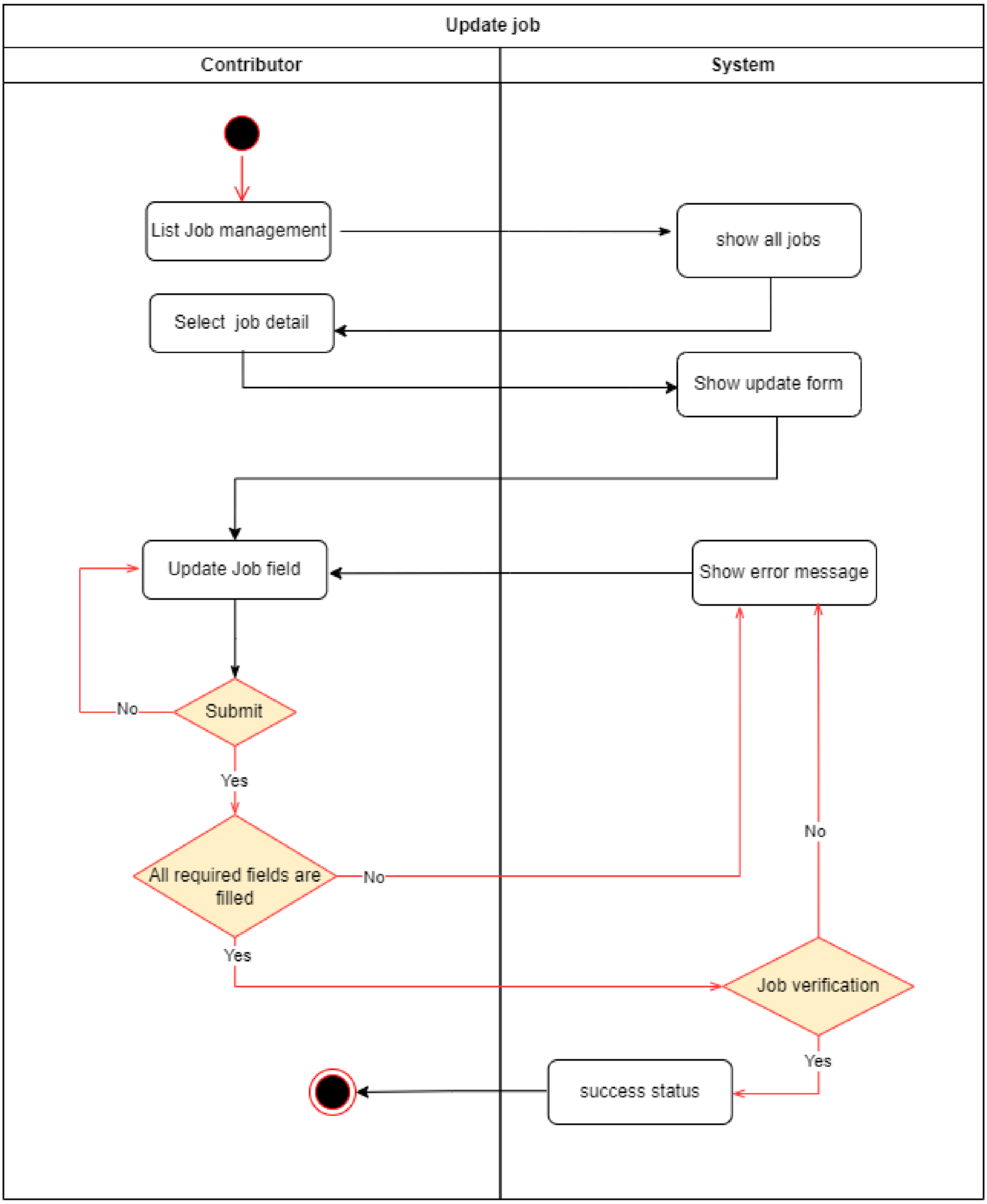

Figure 16: Activity diagram of Job updating

0.4.5 Get user

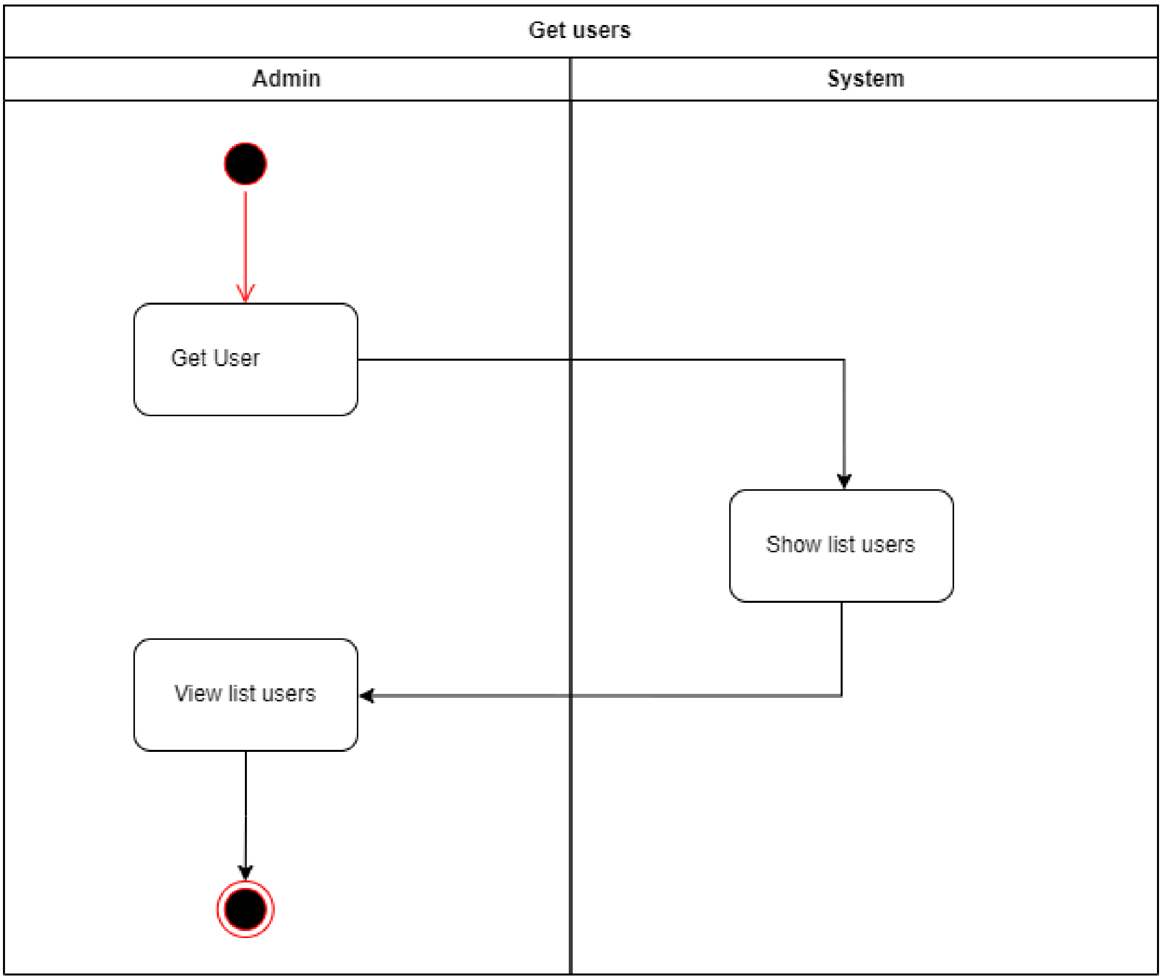

Figure 17: Activity diagram of get users

0.4.6 Get Tags

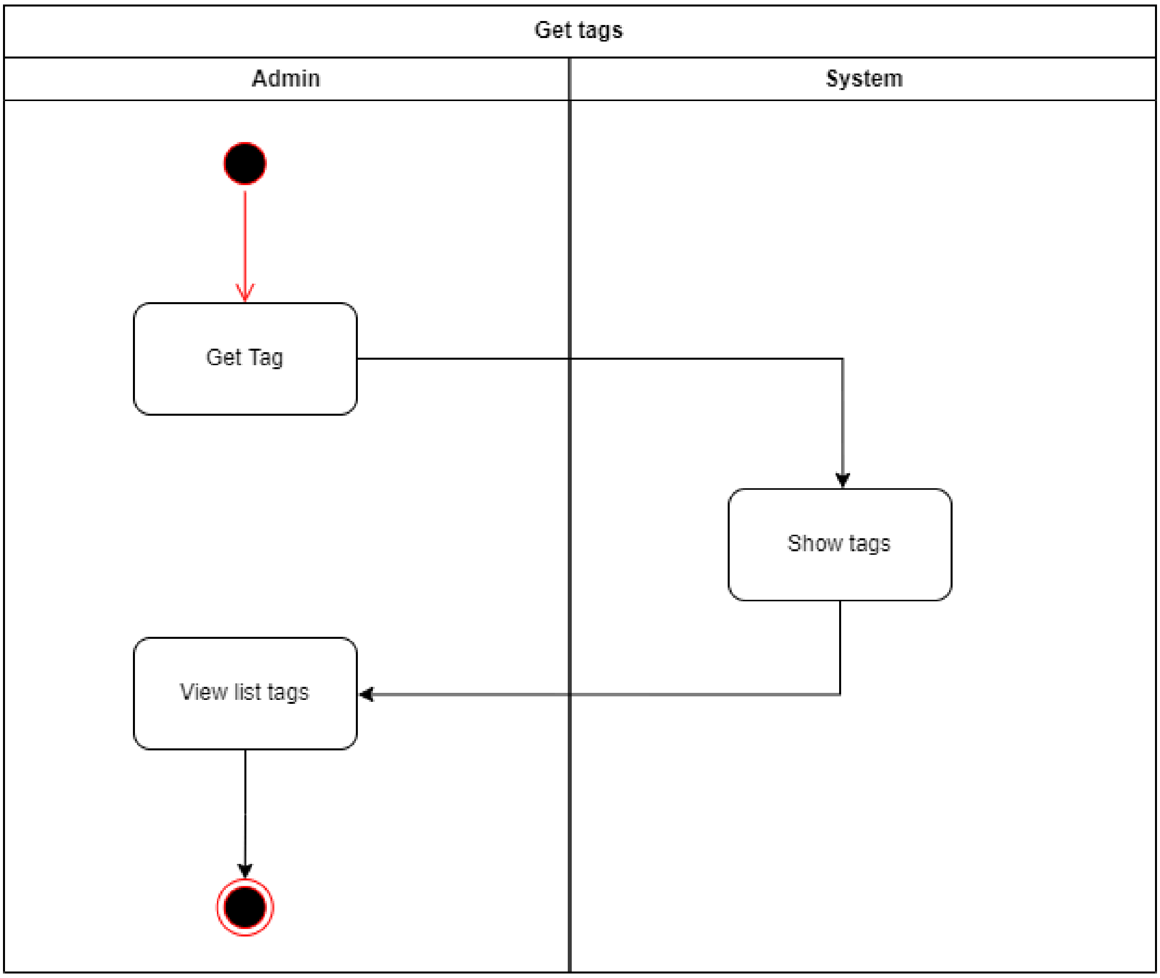

Figure 18: Activity diagram of get tags

0.5 Sequence diagrams

0.5.1 Login

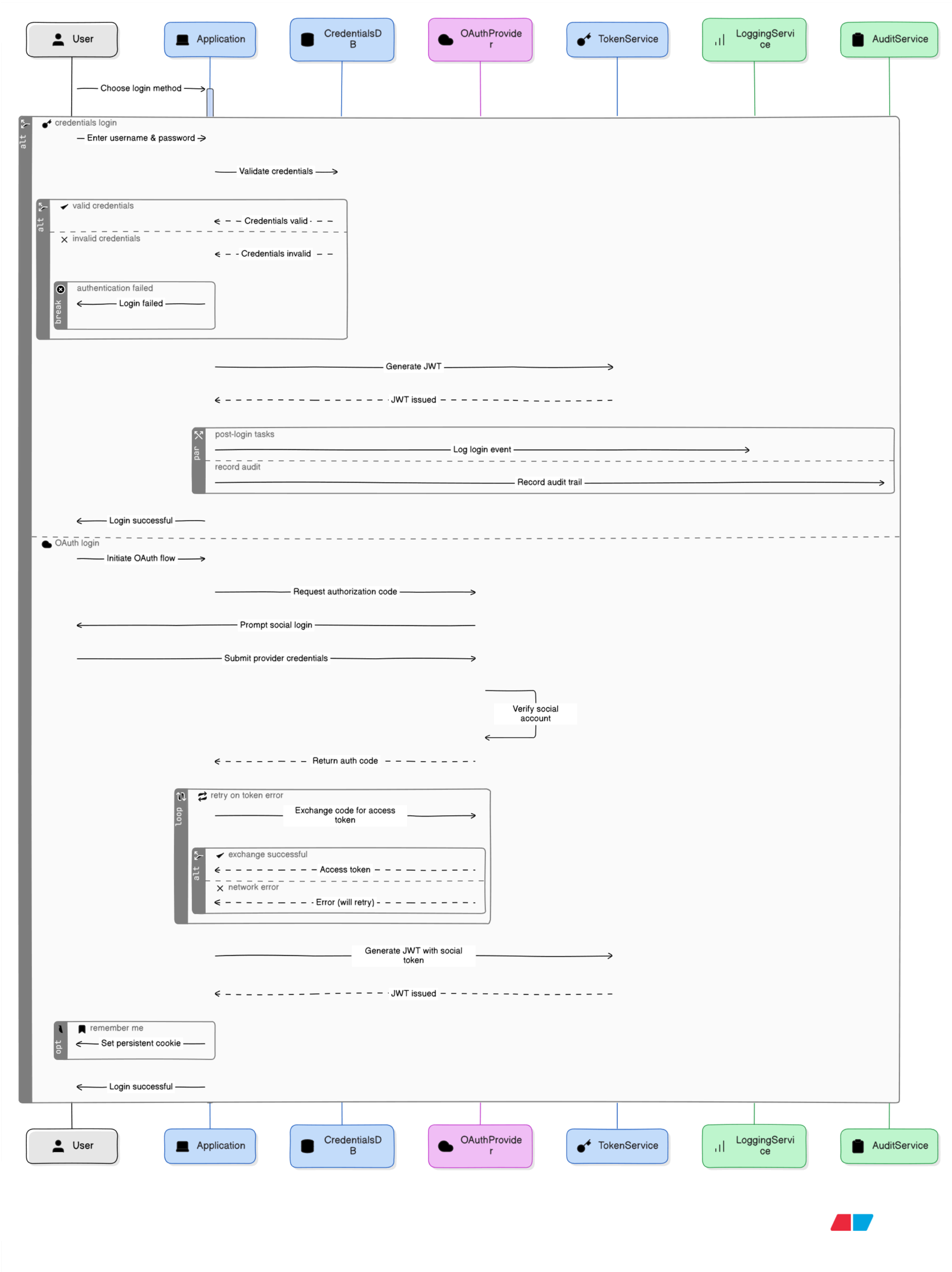

Figure 19: Sequence diagram of the Login Feature

0.5.2 Registration

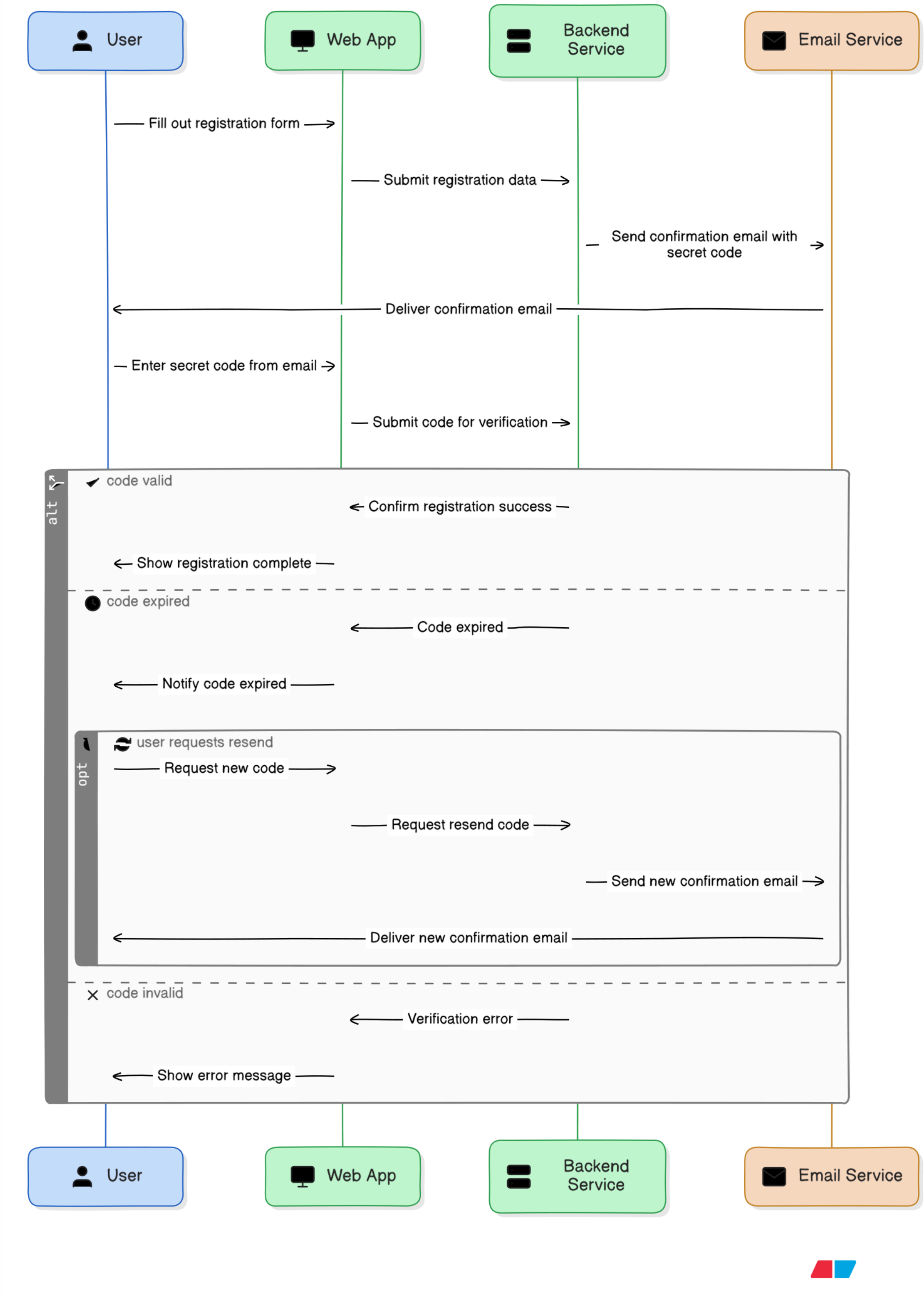

Figure 20: Sequence diagram of the registration Feature

0.5.3 Job posting

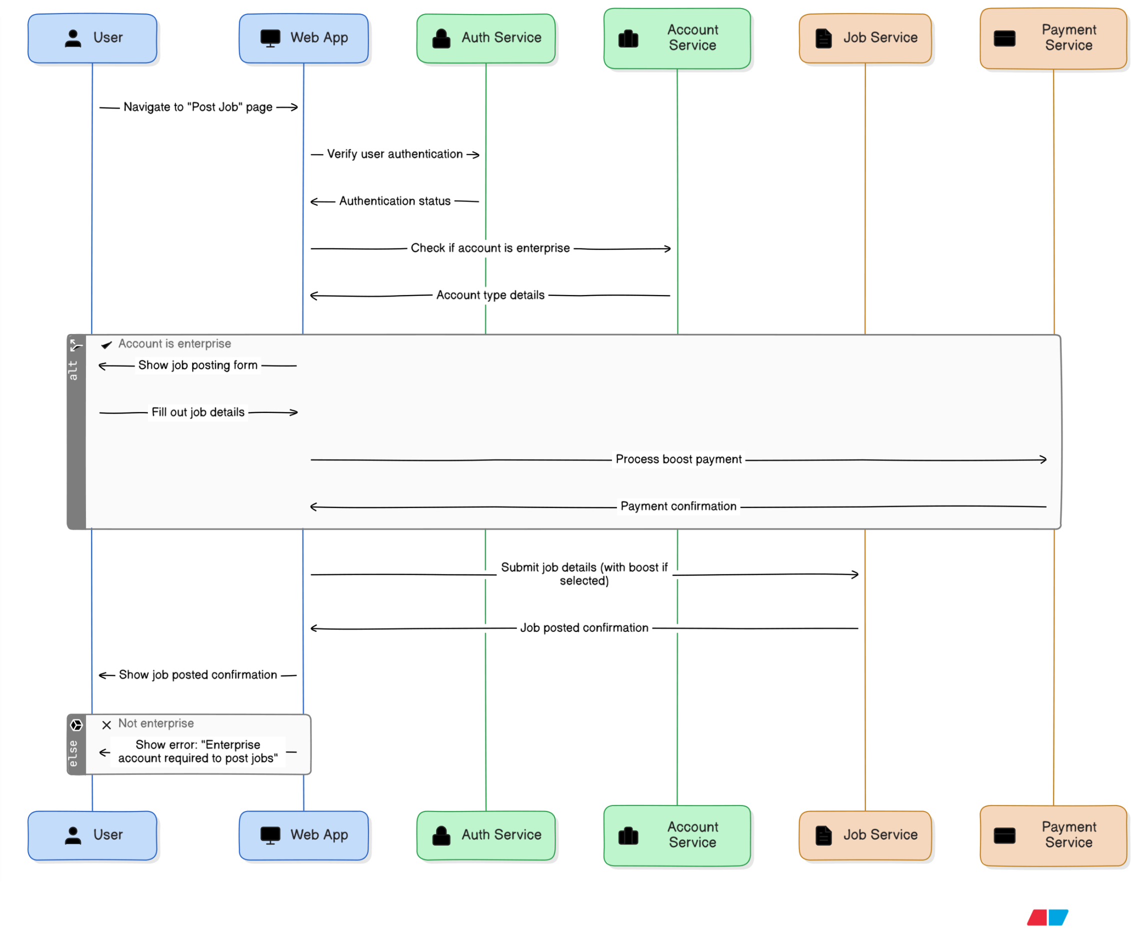

Figure 21: Sequence diagram of the Job Posting Feature

0.5.4 Apply job

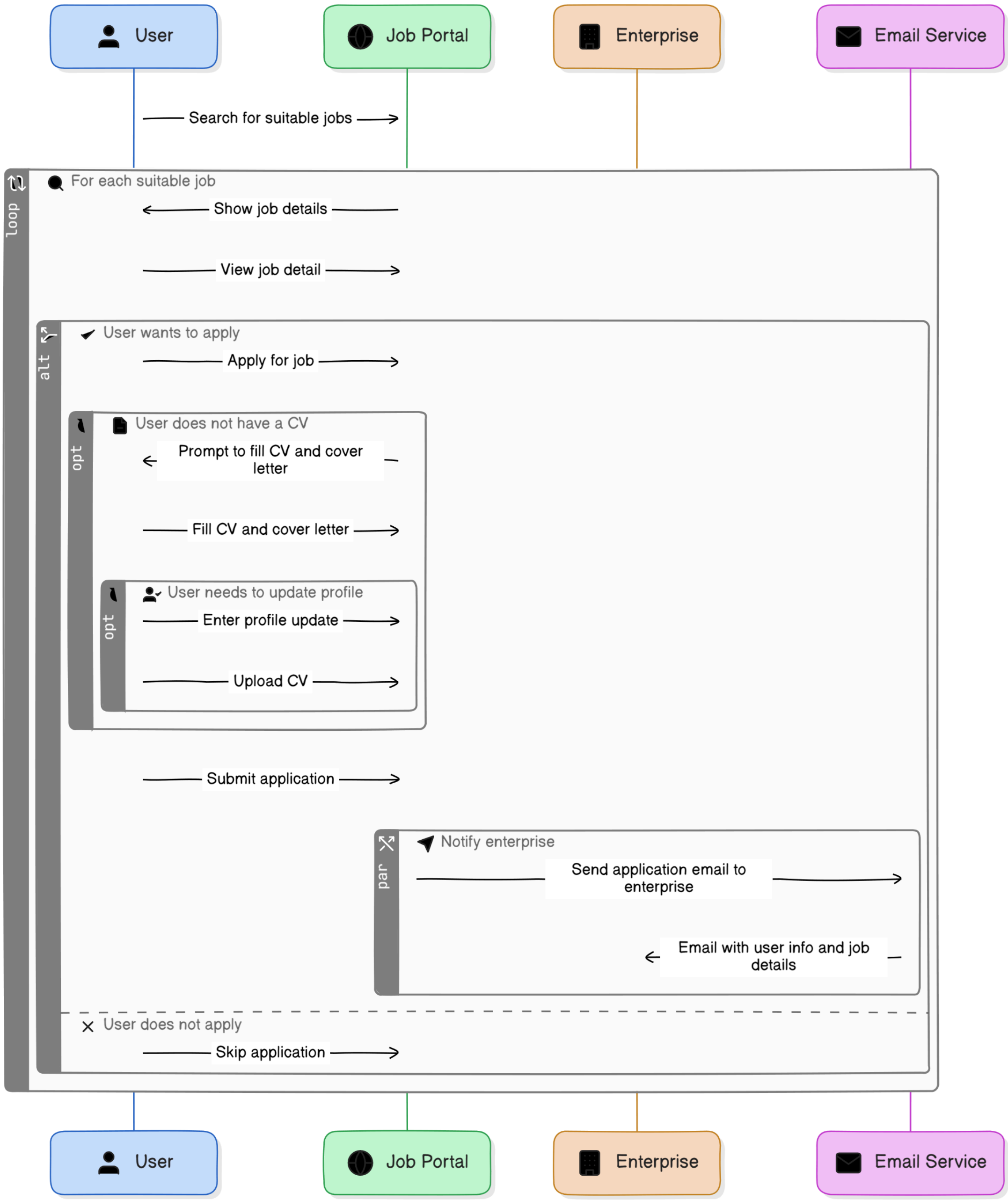

Figure 22: Sequence diagram for the Job Application Feature

0.5.5 Manage applicants

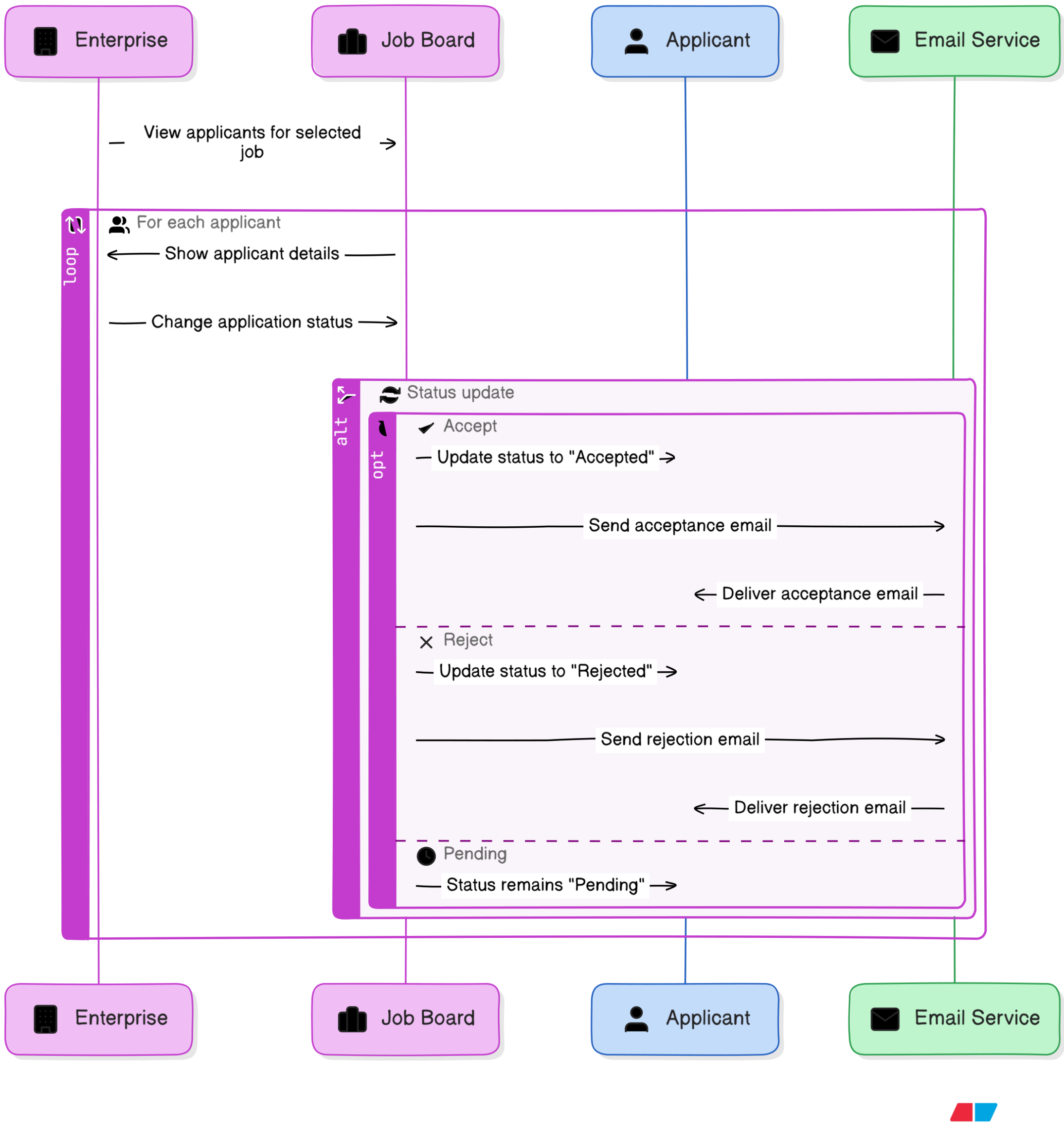

Figure 23: Sequence diagram for Managing Applicants Feature

0.5.6 Enterprise registration

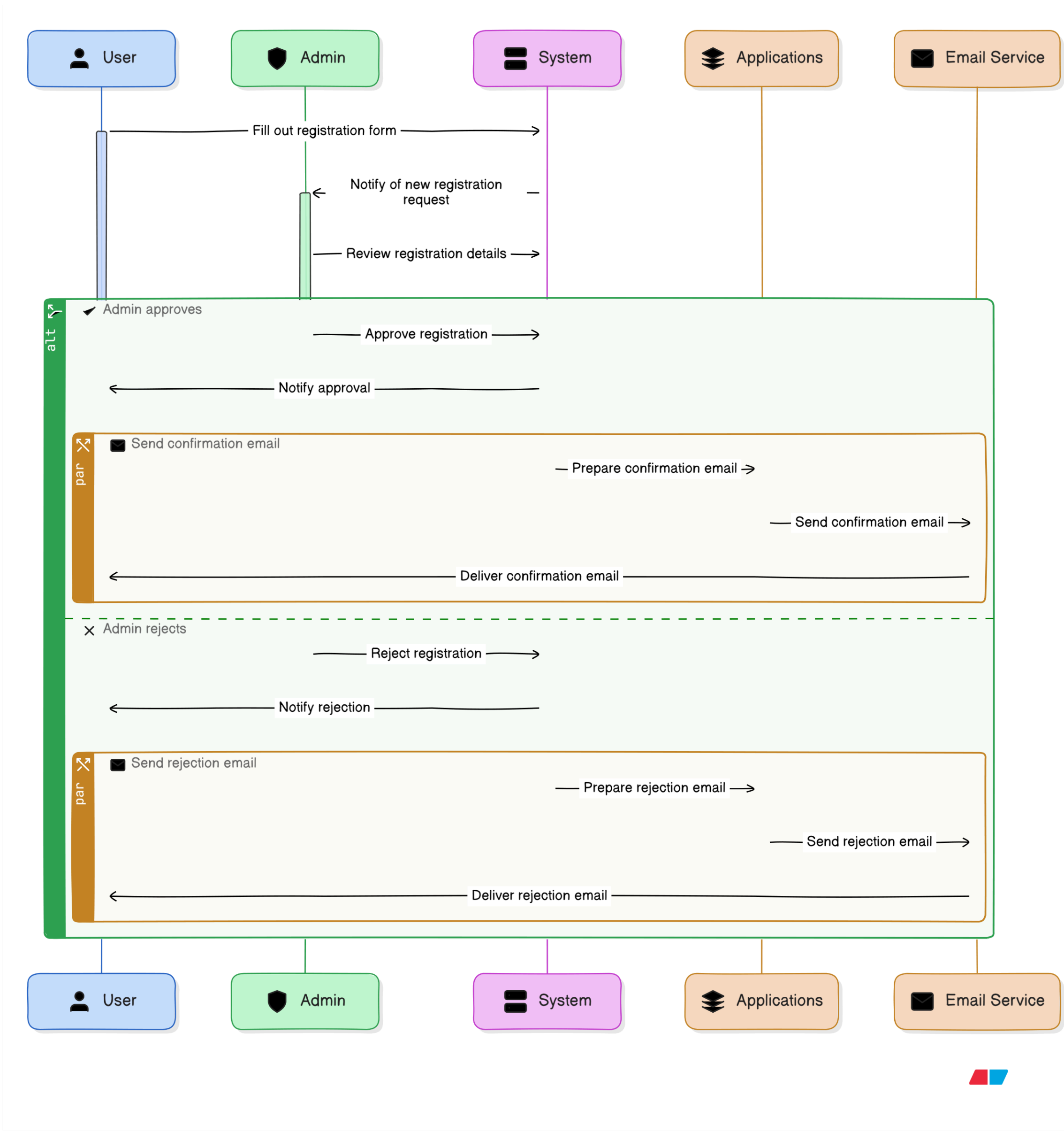

Figure 24: Sequence diagram for the Enterprise Registration Feature

0.6 Entity Diagram

0.6.1 Table Overview

Table 3: Details of Table Overview

No.	Table name	Short Description
1	Accounts	Stores account information for all users, including login credentials and roles
2	Profiles	Contains personal profile details of users such as name, birthday, gender, etc…
3	Addresses	General address table used across the system
4	Enterprises	Holds information about enterprise users
5	Jobs	Contains job postings created by enterprises
6	Tags	Master list of tags used across the system
7	Boosted Job	Contains data about job posts that have been promoted or boosted
8	Categories	Stores all general system-wide categories (if not only for jobs)
9	CVs	Stores uploaded CV documents and metadata linked to user profiles
10	Notifications	Stores notifications for users, including content and status (read/unread)
11	Transactions	Holds payment or system transaction records, including job boosts, upgrades, etc…
12	Website	Holds information about website companies

0.6.2 Entity Relationship Diagram

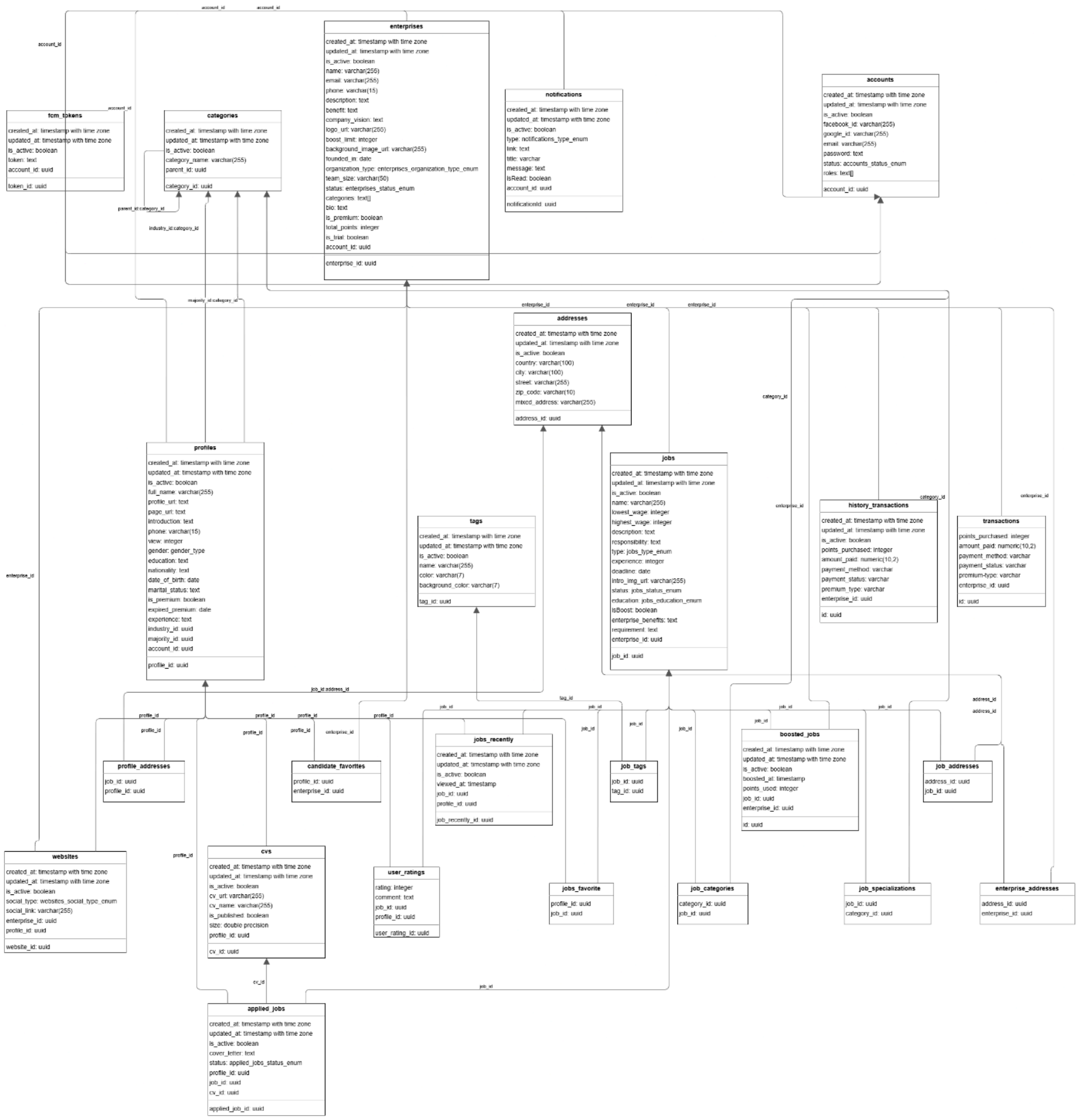

Figure 25: Entity diagram

0.6.3 Entity details

❖ **Accounts**

Table 4: Accounts entity detail

Attributes	Datatype	Null	Key	Default
accountId	UUID	NOT NULL	PK	
facebook_id	VARCHAR(50)			
google_id	VARCHAR(255)			
email	VARCHAR(50)			
password	VARCHAR(50)			
status	ENUM			PENDING
roles	TEXT			
profile	UUID		FK	
enterprise	UUID		FK	
notification	UUID		FK	
fcmToken	UUID		FK	

❖ **Profile**

Table 5: Profile entity detail

Attributes	Datatype	Null	Key	Default
profileId	UUID	NOT NULL	PK	
fullName	VARCHAR(255)	NOT NULL		
profileUrl	TEXT	NULL		
pageUrl	TEXT	NOT NULL		
introduction	TEXT	NULL		
phone	VARCHAR(15)	NULL		
view	INT	NOT NULL		
gender	ENUM	NULL		
education	TEXT	NULL		
nationality	TEXT	NULL		
dateOfBirth	DATE	NULL		

Attributes	Datatype	Null	Key	Default
maritalStatus	TEXT	NULL		
isPremium	BOOLEAN	NOT NULL		
expiredPremium	DATE	NOT NULL		
experience	TEXT	NULL		
industry_id	UUID		FK	
majority_id	UUID		FK	
account_id	UUID		FK	
websites	UUID		FK	
jobs_favorite	UUID		FK	
cvs	UUID		FK	
appliedJob	UUID		FK	
addresses	UUID		FK	
candidate_favorite	UUID		FK	

❖ **Addresses**

Table 6: Address entity detail

Attributes	Datatype	Null	Key	Default
addressId	UUID	NOT NULL	PK	
country	VARCHAR(100)	NOT NULL		
city	VARCHAR(100)	NOT NULL		
street	VARCHAR(255)	NOT NULL		
zip_code	VARCHAR(10)	NOT NULL		
mixed_address	VARCHAR(255)	NOT NULL		
job_addresses	UUID		FK	
enterprise_addresses	UUID		FK	
profile_addresses	UUID		FK	

❖ **Enterprise**

Table 7: Enterprise entity detail

Attributes	Datatype	Null	Key	Default
enterprises_id	UUID	NOT NULL	PK	
name	VARCHAR(255)			
email	VARCHAR(15)			
phone	DATE	NOT NULL		
description	TEXT			
benefit	TEXT			
company_vision	TEXT	NULL		
logo_url	VARCHAR(255)			
boost_limit	INT			0
background_image_url	VARCHAR(255)	NULL		
founded_in	DATE	NULL		
organization_type	ENUM	NULL		
team_size	VARCHAR(50)	NULL		
status	ENUM	NOT NULL		PENDING
categories	TEXT	NOT NULL		
bio	TEXT	NULL		
is_premium	BOOLEAN	NOT NULL		FALSE
total_points	INT	NOT NULL		0
is_trial	BOOLEAN	NULL		
account_id	UUID		FK	
website	UUID		FK	
jobs	UUID		FK	
addresses	UUID		FK	
profiles	UUID		FK	
boostedJobs	UUID		FK	
transactions	UUID		FK	

❖ **Jobs**

Table 8: Jobs entity detail

Attributes	Datatype	Null	Key	Default
jobId	UUID	NOT NULL	PK	
name	VARCHAR(255)	NOT NULL		
lowest_wage	INT	NULL		
highest_wage	INT	NULL		
description	TEXT	NULL		
responsibility	TEXT	NULL		
type	ENUM	NOT NULL		CONTRACT
experience	INT			0
deadline	Datetime	NULL		
into_img_url	VARCHAR(255)			
status	ENUM			OPEN
education	ENUM	NULL		
isBoost	BOOLEAN			
enterprise_benefits	TEXT	NULL		
requirement	TEXT	NULL		
enterprise_id	UUID		FK	
job_tags	UUID		FK	
profiles	UUID		FK	
appliedJob	UUID		FK	
categories	UUID		FK	
addresses	UUID		FK	
job_specializations	UUID		FK	
boostedJob	UUID		FK	

- **Tags**

Table 9: Tag entity details

Attributes	Datatype	Null	Key	Default
tag_id	UUID		PK	
name	VARCHAR(255)			
color	VARCHAR(7)	NULL		
background_color	VARCHAR(7)	NULL		
jobs	UUID		FK	

- **Boosted Job**

Table 10: Boosted Job entity detail

Attributes	Datatype	Null	Key	Default
id	UUID		PK	
boosted_at	DATE			
points_used	INT	NOT NULL		
job	UUID		FK	
enterprise	UUID	NOT NULL	FK	

- **Categories**

Table 11: Categories entity detail

Attributes	Datatype	Null	Key	Default
category_id	UUID		PK	
category_name	VARCHAR(255)			
parent_id	UUID		FK	
children	UUID		FK	
jobs	UUID		FK	

❖ **CVs**

Table 12: CVs entity detail

Attributes	Datatype	Null	Key	Default
cv_id	UUID		PK	
cv_url	VARCHAR(255)			
cv_name	VARCHAR(255)			
is_published	BOOLEAN			
size	FLOAT			0
profile_id	UUID		FK	
appliedJob	UUID		FK	

❖ **Notifications**

Table 13: Notifications entity detail

Attributes	Datatype	Null	Key	Default
notificationId	UUID		PK	
type	ENUM			
link	TEXT	NULL		
title	VARCHAR(255)	NOT NULL		
message	TEXT	NOT NULL		
isRead	BOOLEAN	NOT NULL		
account_id	UUID		FK	

❖ **Transaction**

Table 14: Transaction entity detail

Attributes	Datatype	Null	Key	Default
id	UUID		PK	
points_purchased	INT			
amount_paid	DECIMAL			
payment_method	VARCHAR(255)			
payment_status	ENUM			PENDING
premium_type	ENUM	NULL		
enterprise_id	UUID		FK	

❖ **Website**

Table 15: Website entity detail

Attributes	Datatype	Null	Key	Default
website_id	UUID		PK	
social_type	ENUM	NULL		
social_link	VARCHAR(255)	NULL		
enterprise_id	UUID		FK	
profile_id	UUID		FK	

Chapter 3 IMPLEMENTATION AND EVALUATION

0.7 Implementation

0.7.1 UI for Authentication

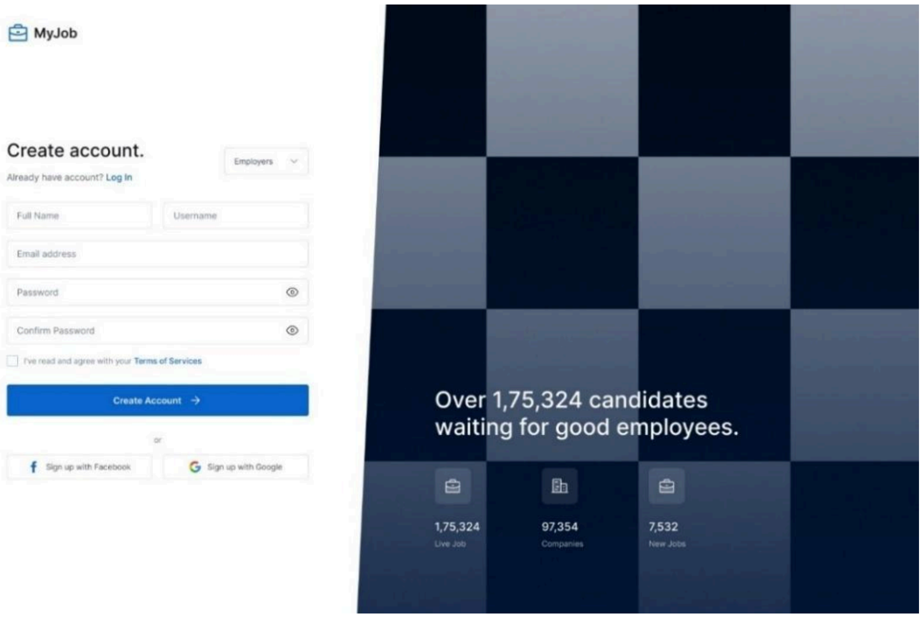

Figure 26: UI for registration

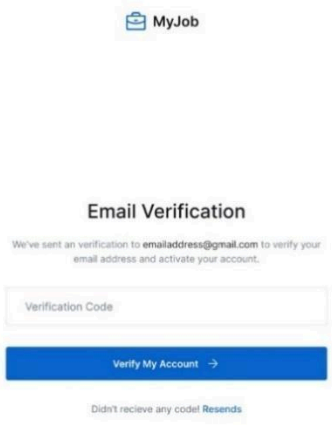

Figure 27: UI for email verification

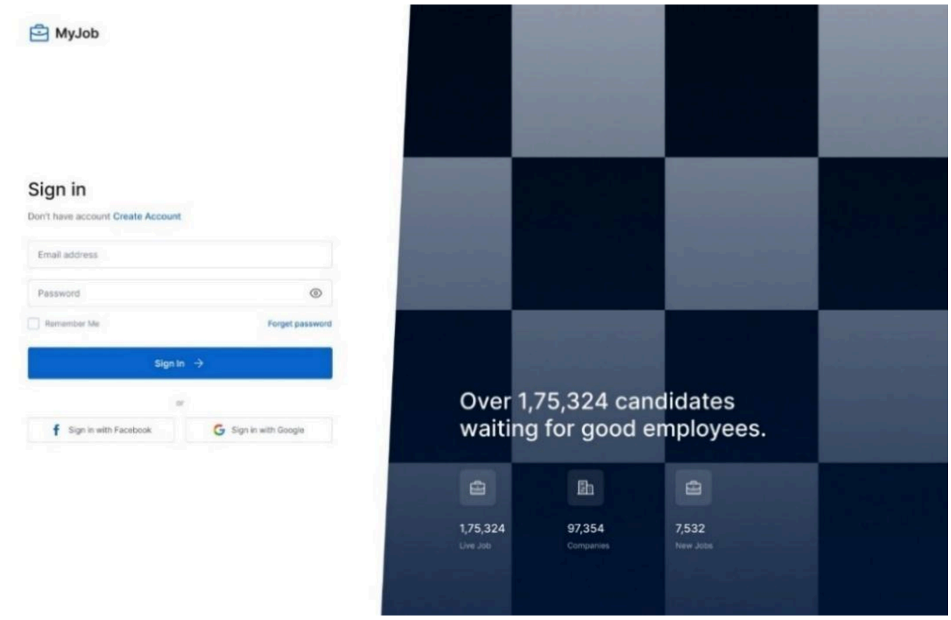

Figure 28: UI for login feature

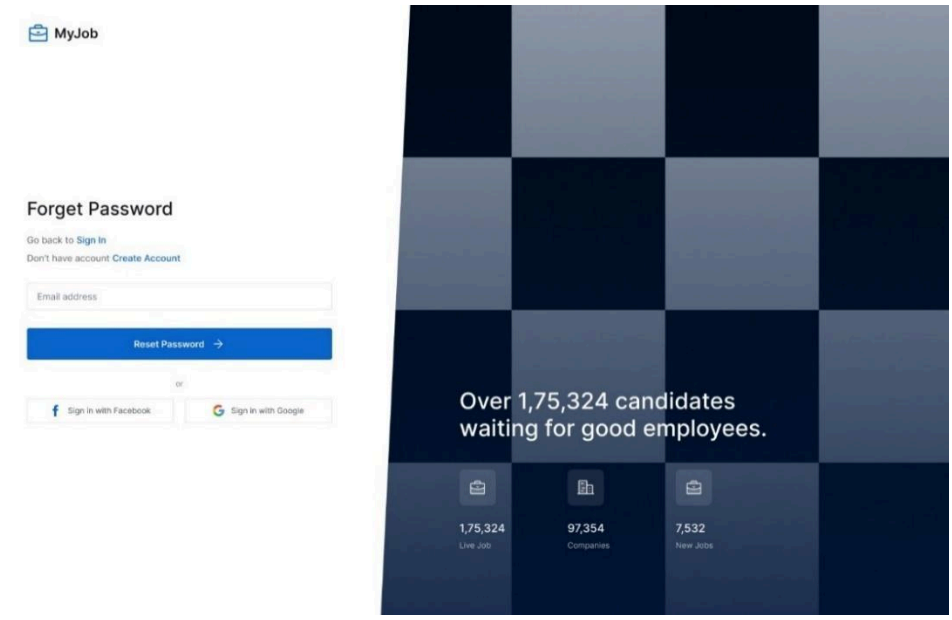

Figure 29: UI for forgot password feature

Figure 30: UI for reset password feature

0.7.2 UI for user role

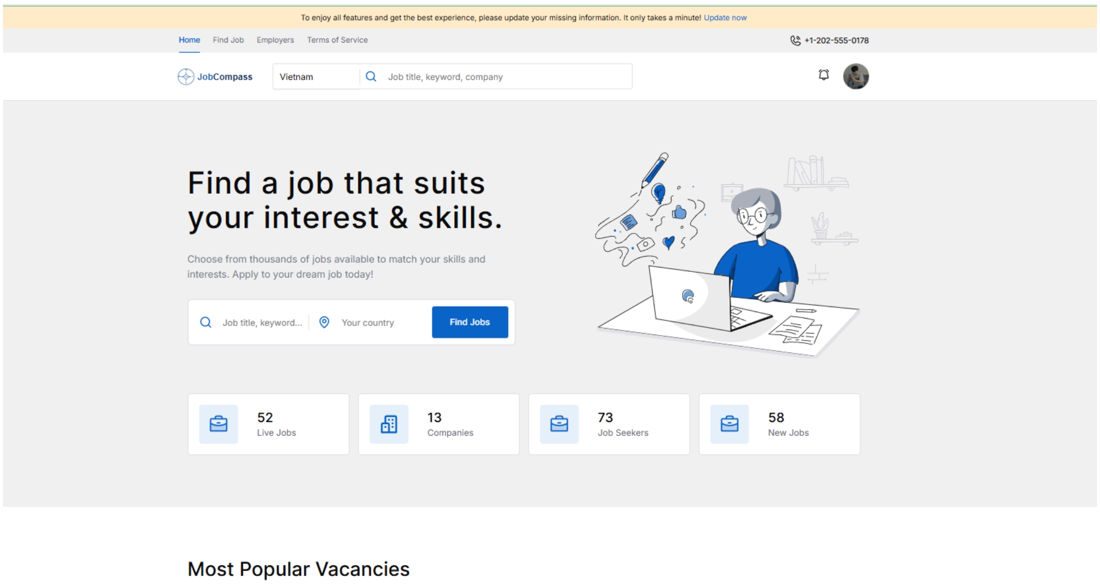

Figure 31: UI home page

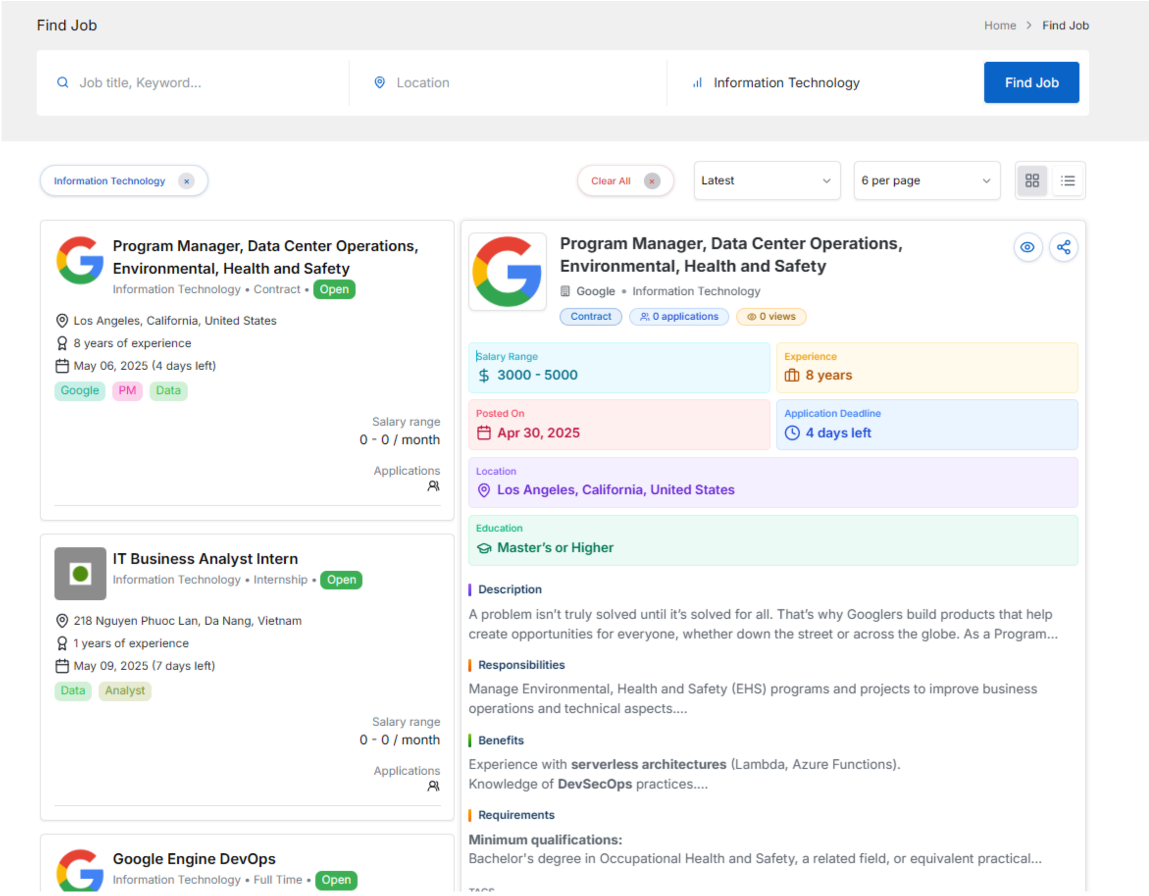

Figure 32: UI for find job feature

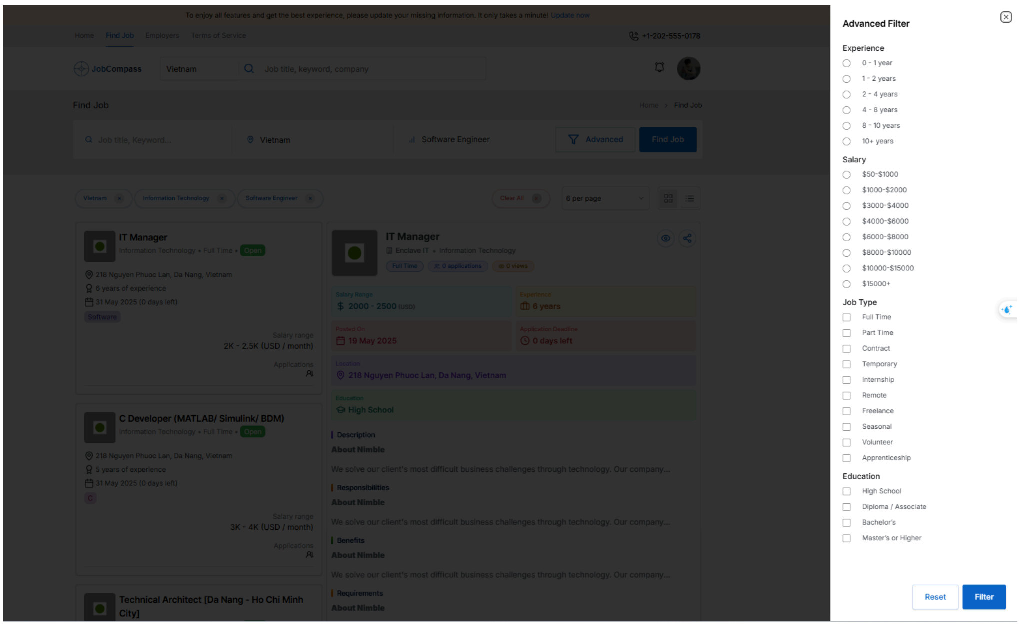

Figure 33: UI for filter job feature

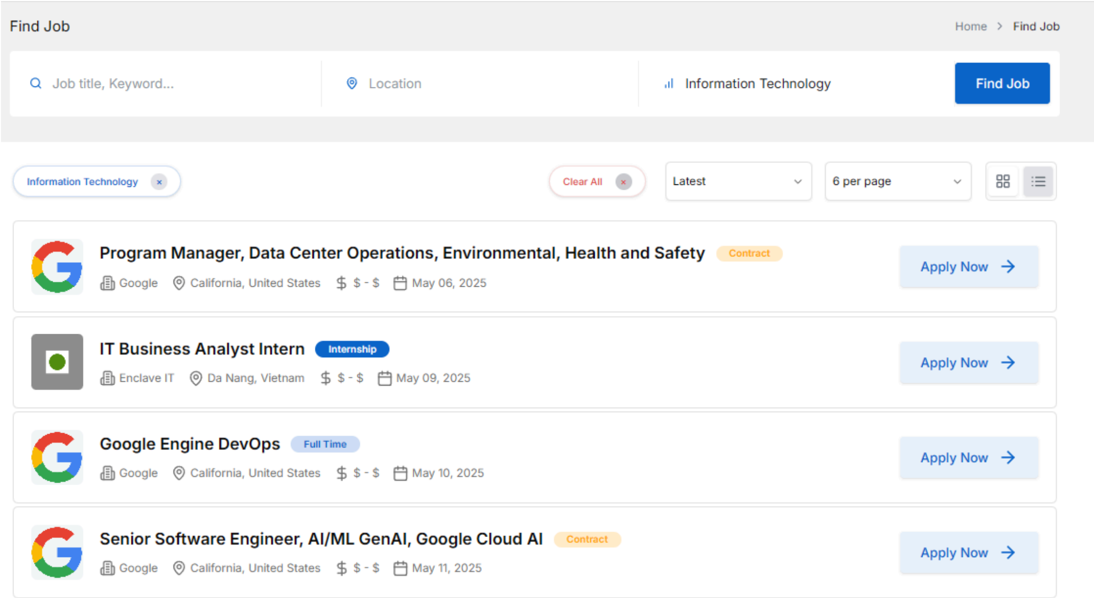

Figure 34: UI for list view job

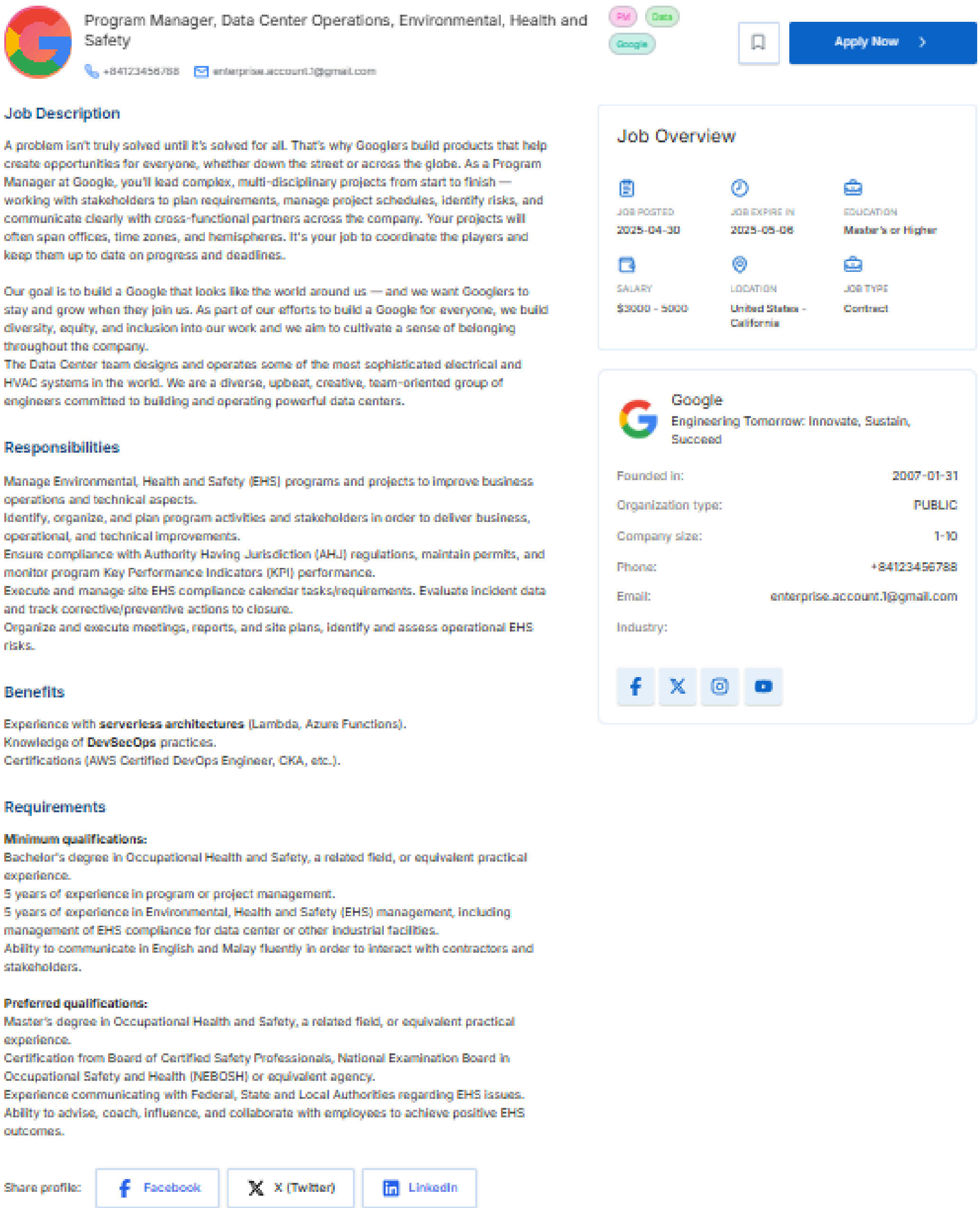

Figure 35: UI for job detail

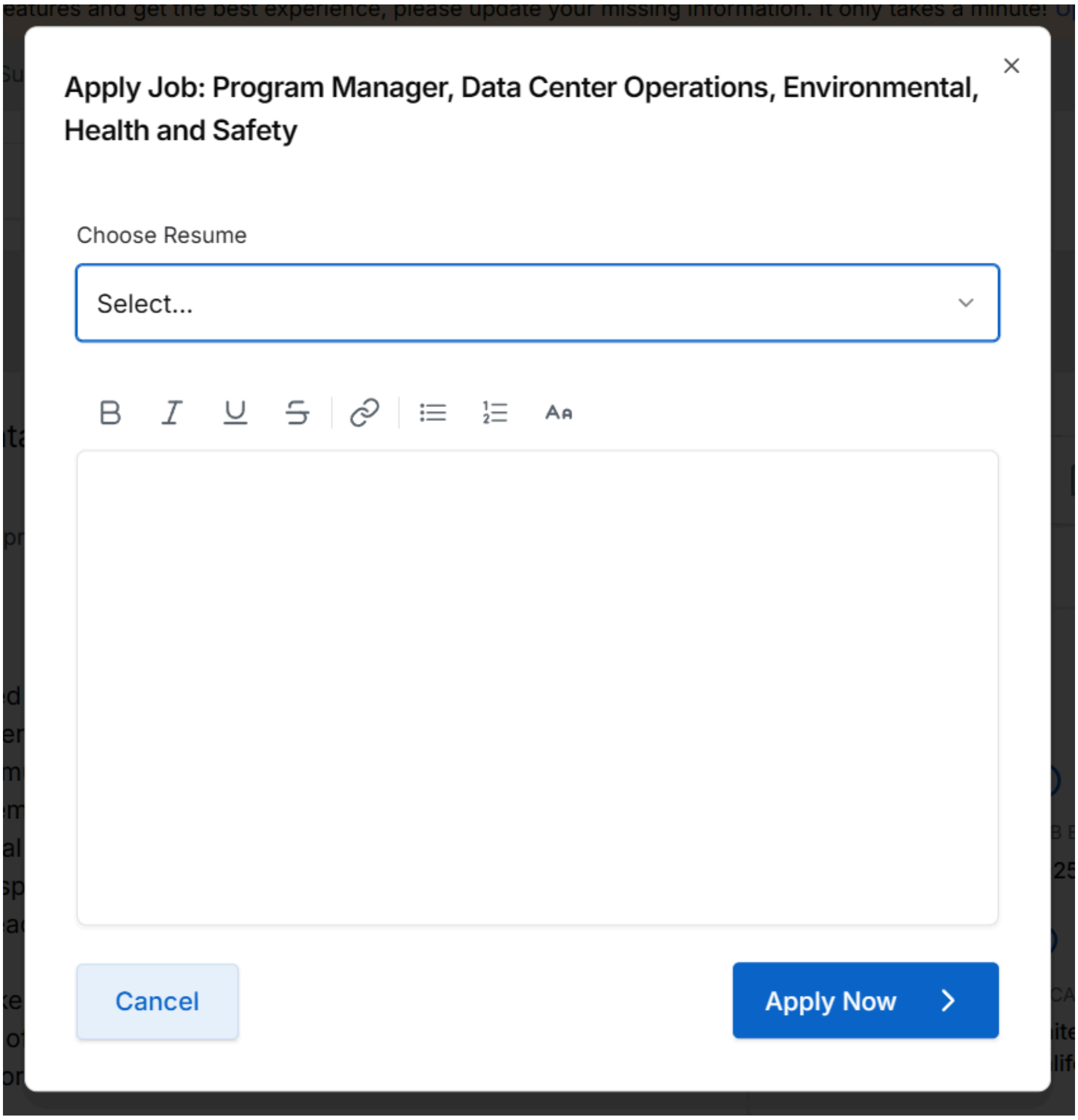

Figure 36: UI for apply job feature

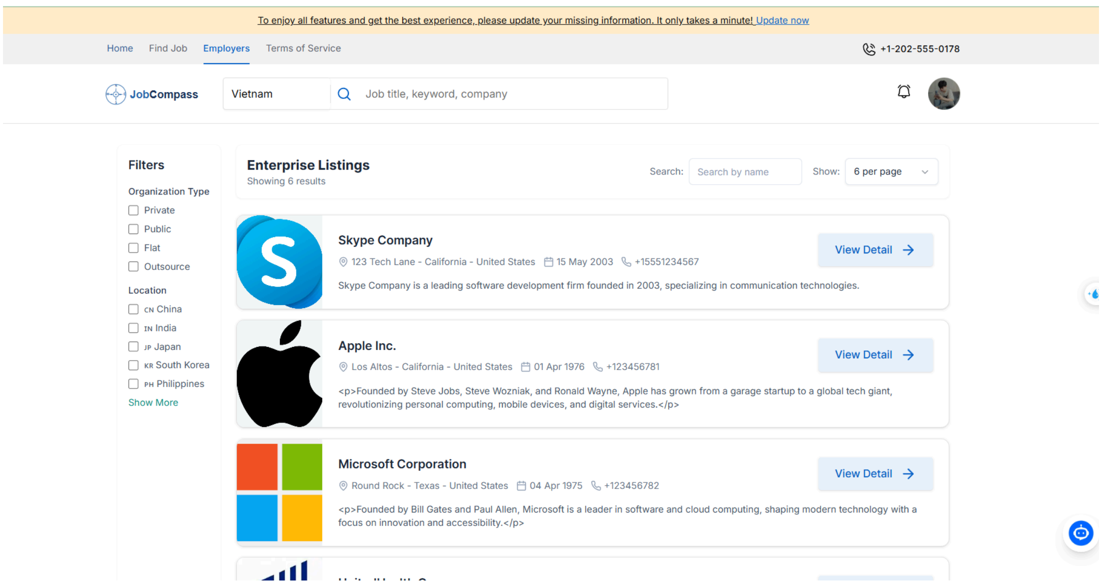

Figure 37: UI enterprise list

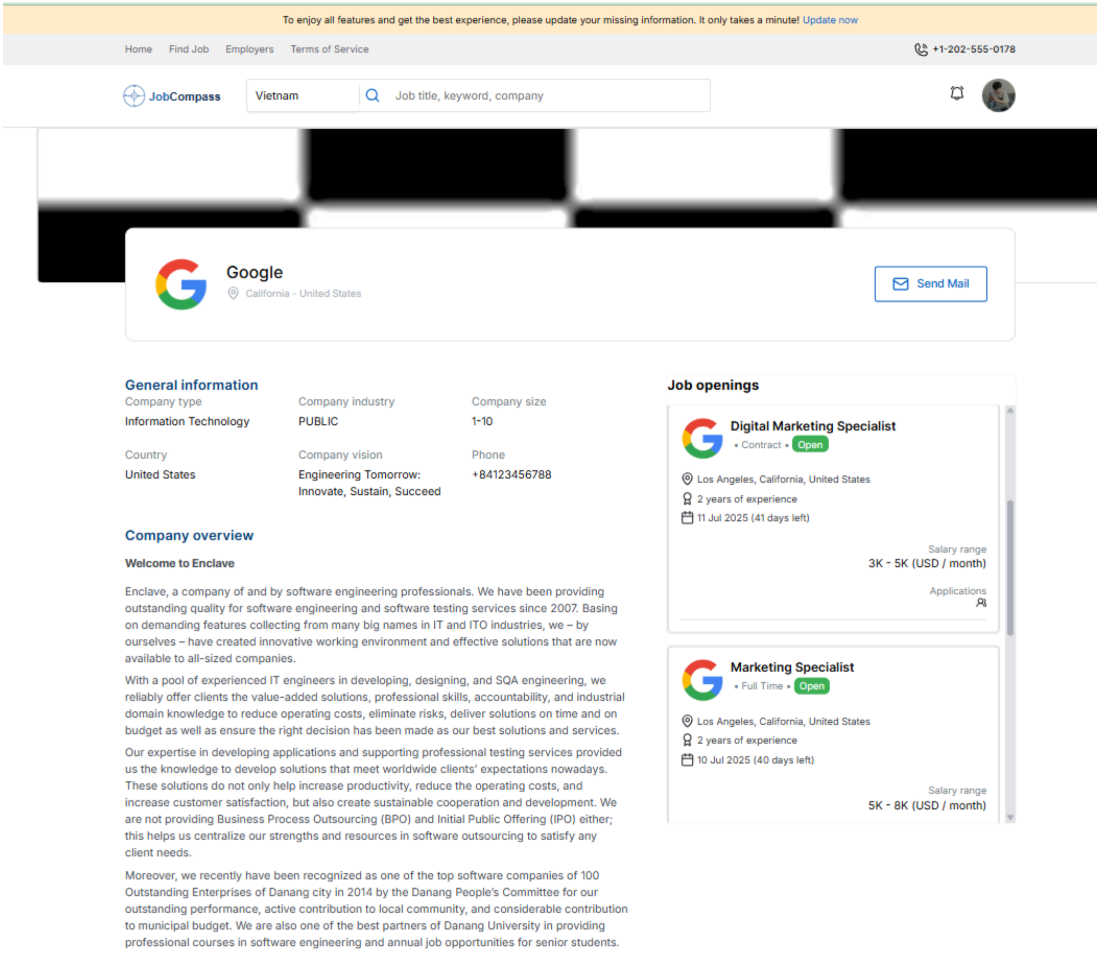

Figure 38: UI enterprise detail

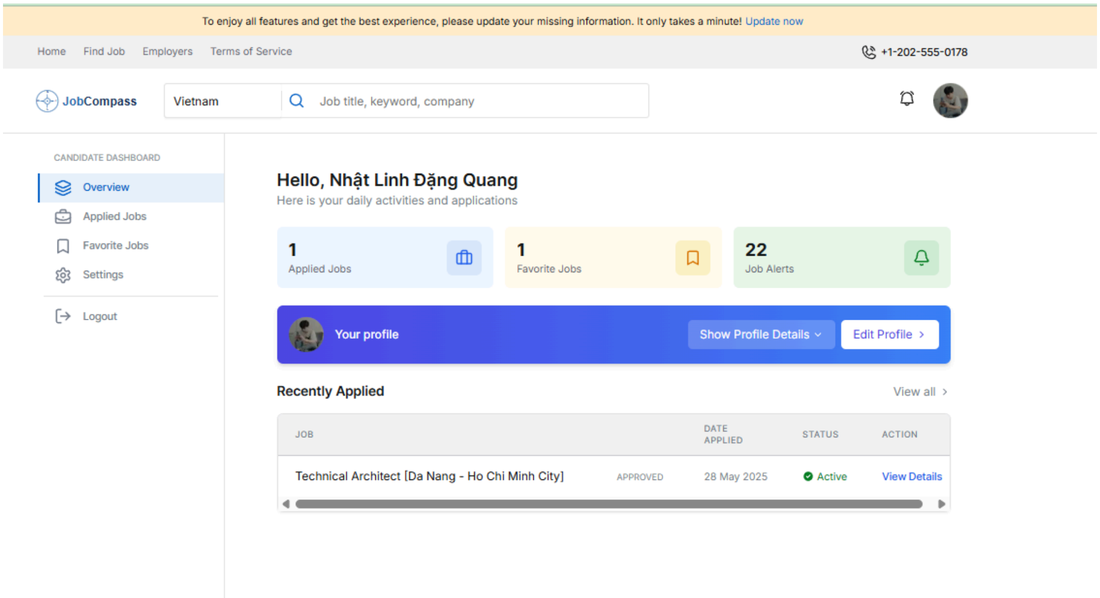

Figure 39: UI user profile

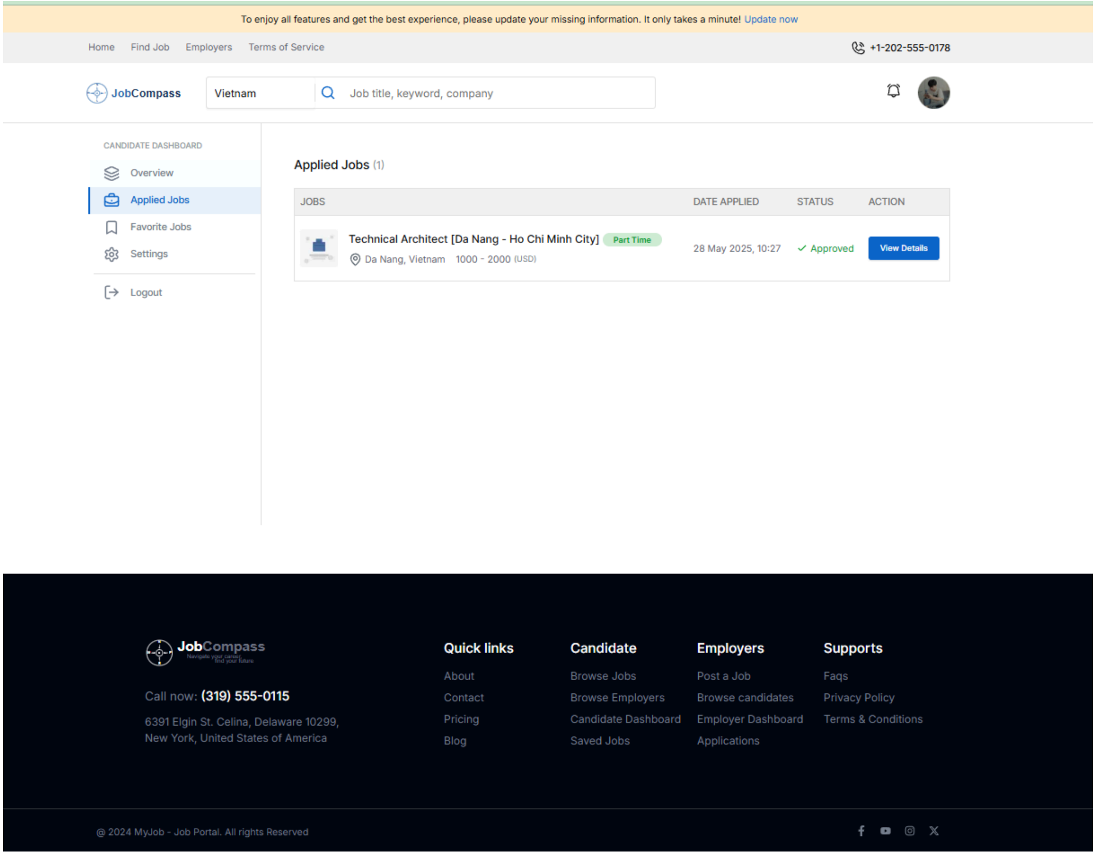

Figure 40: UI for view applied job

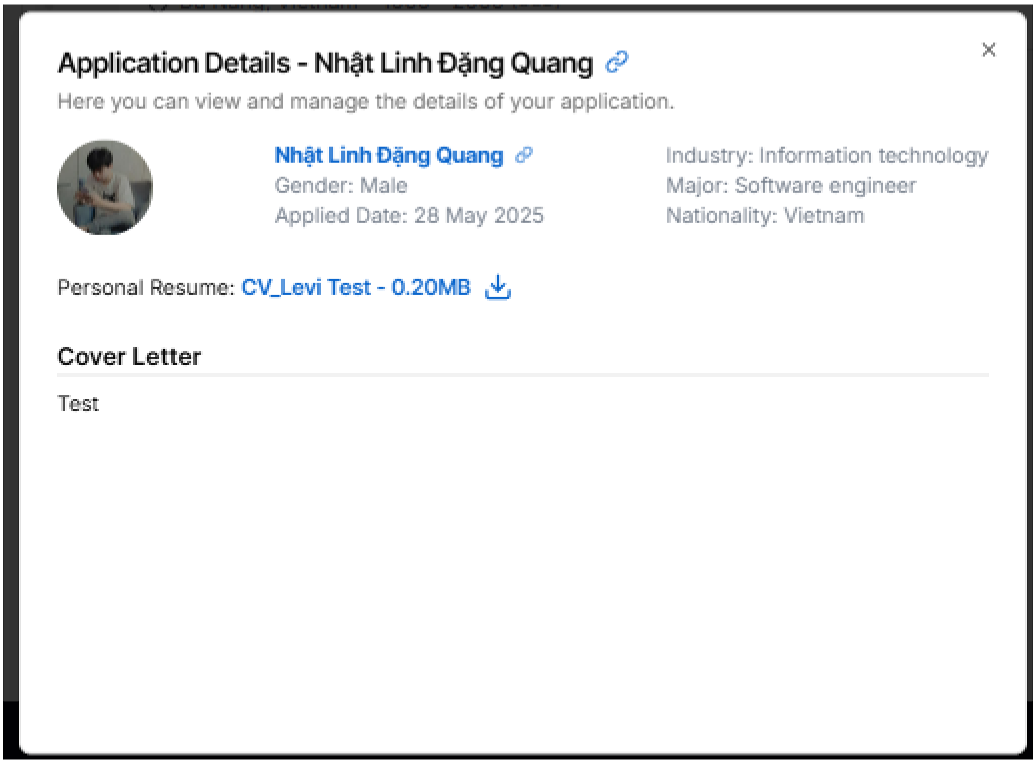

Figure 41: UI for view detail applied job

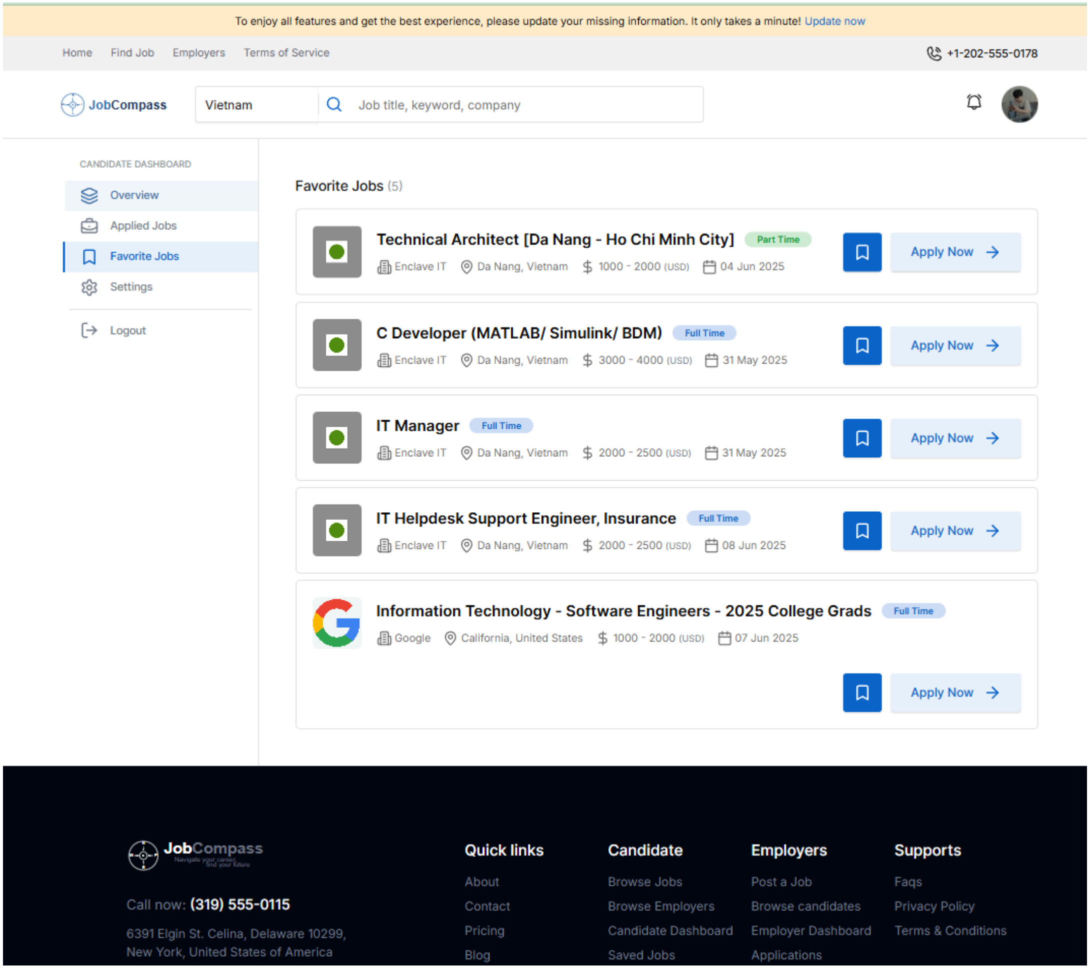

Figure 42: UI list favorite job view

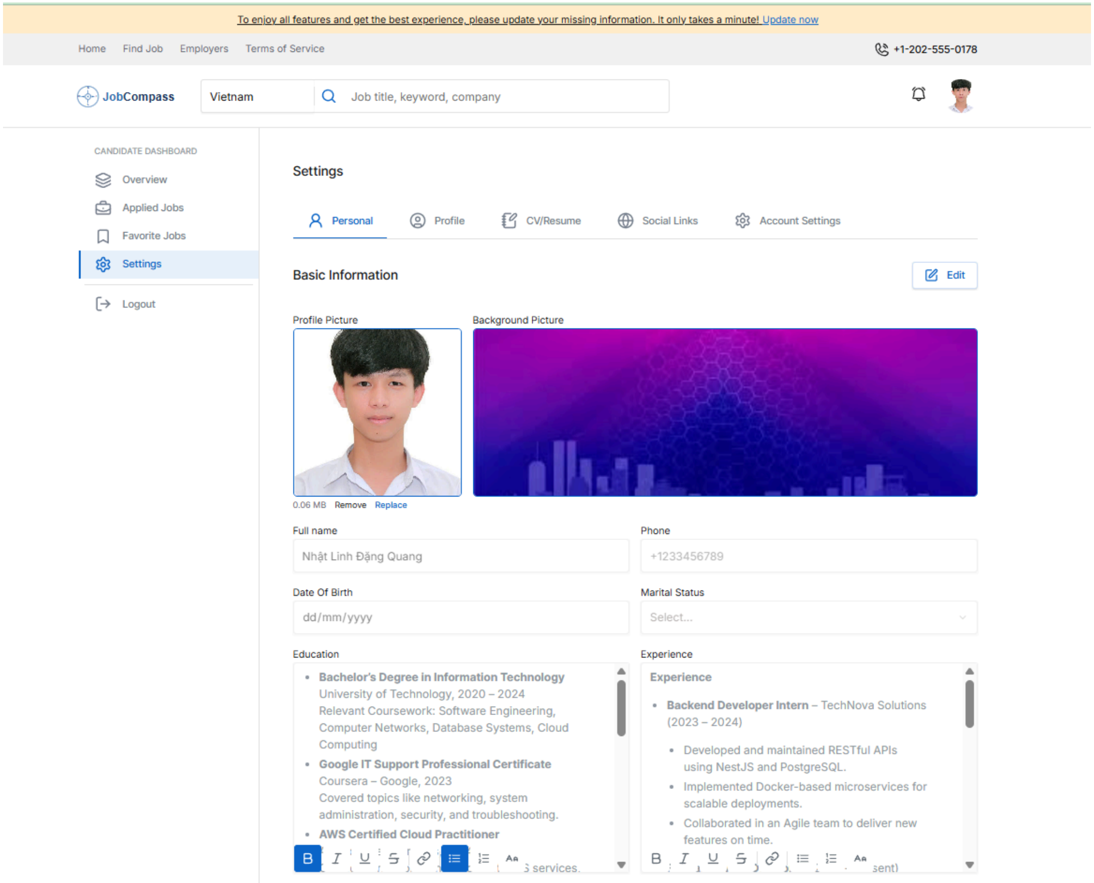

Figure 43: UI for set up personal user

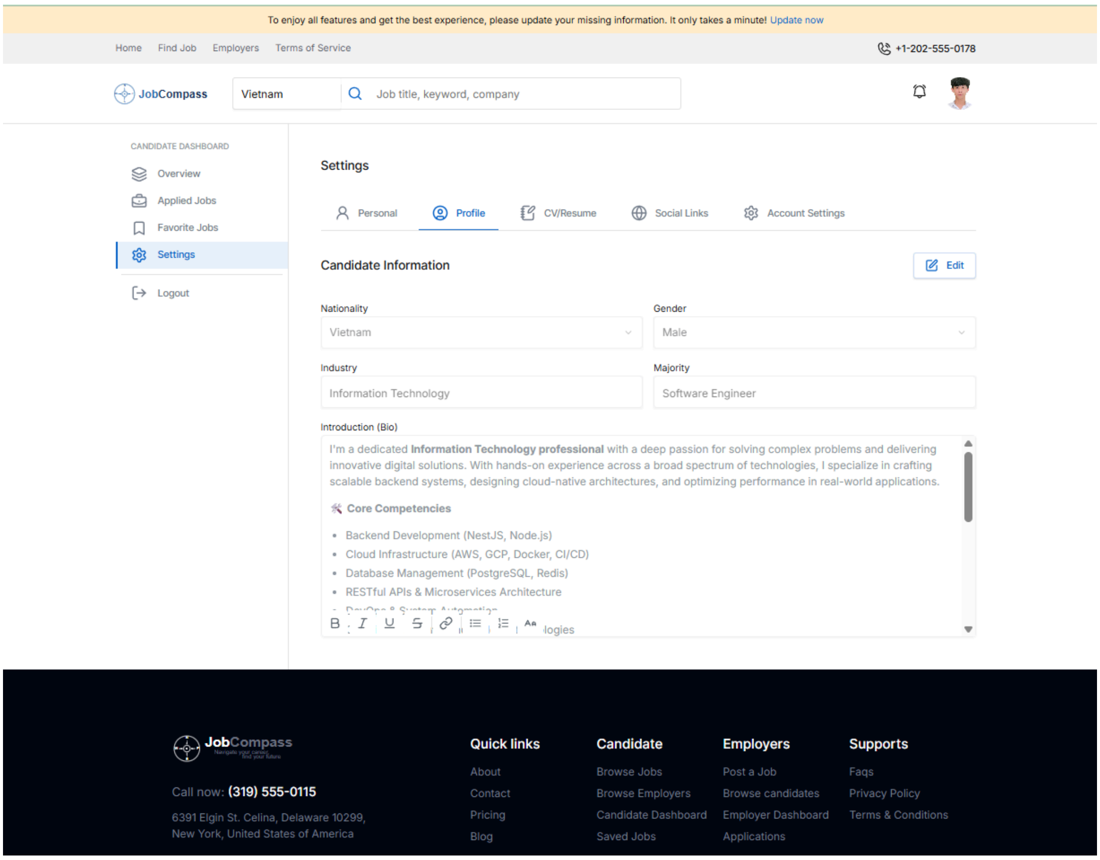

Figure 44: UI for setting profile

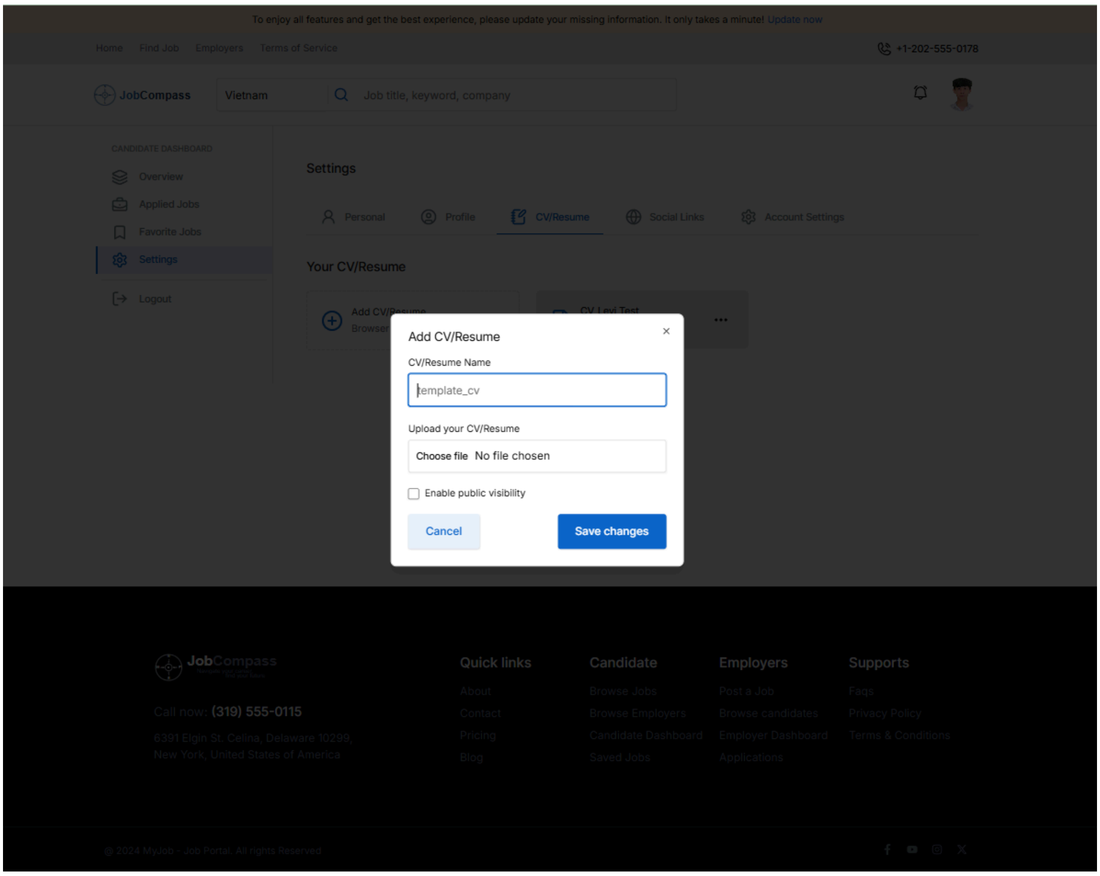

Figure 45: UI for posting CV

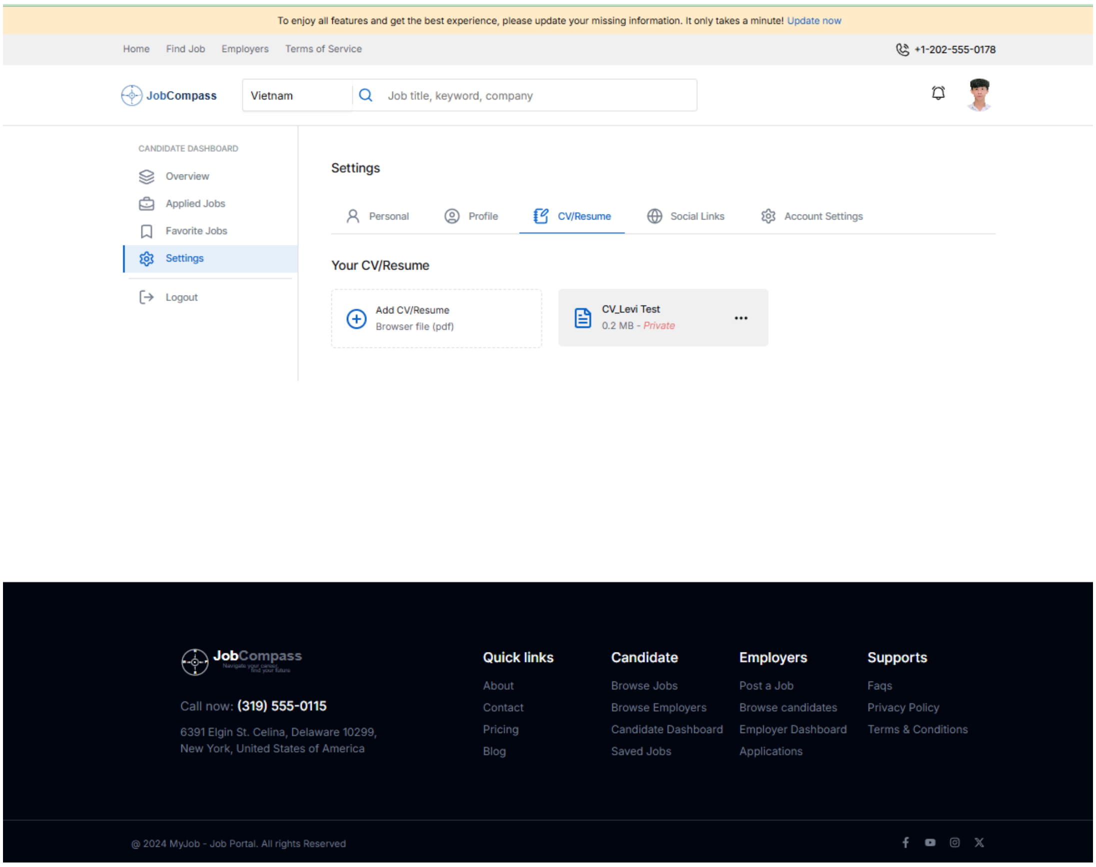

Figure 46: UI for managing CV

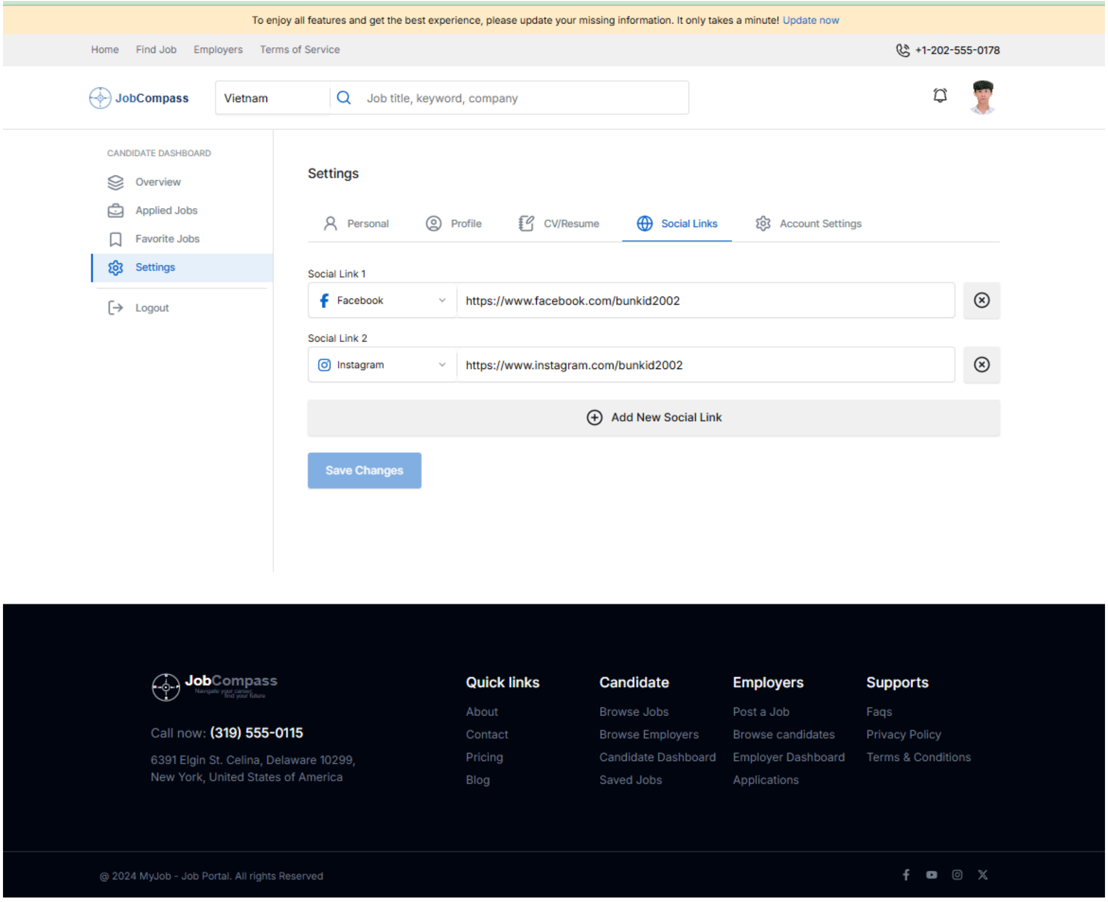

Figure 47: UI for managing social network

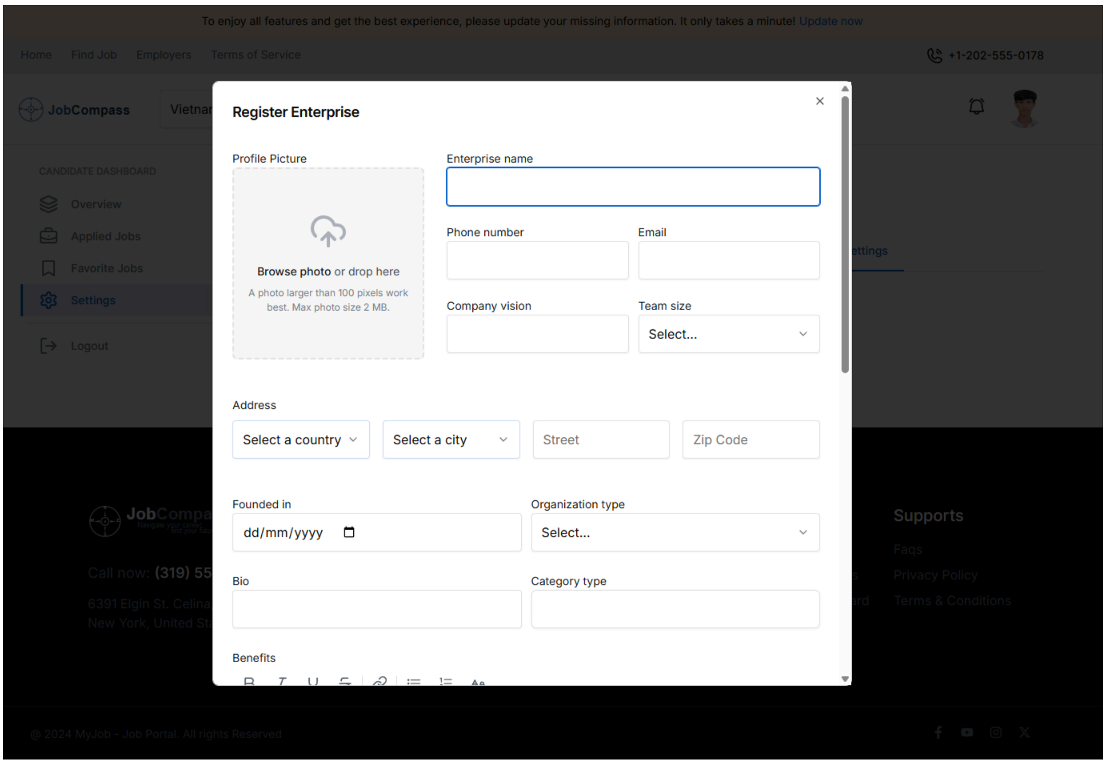

Figure 48: UI for enterprise registration

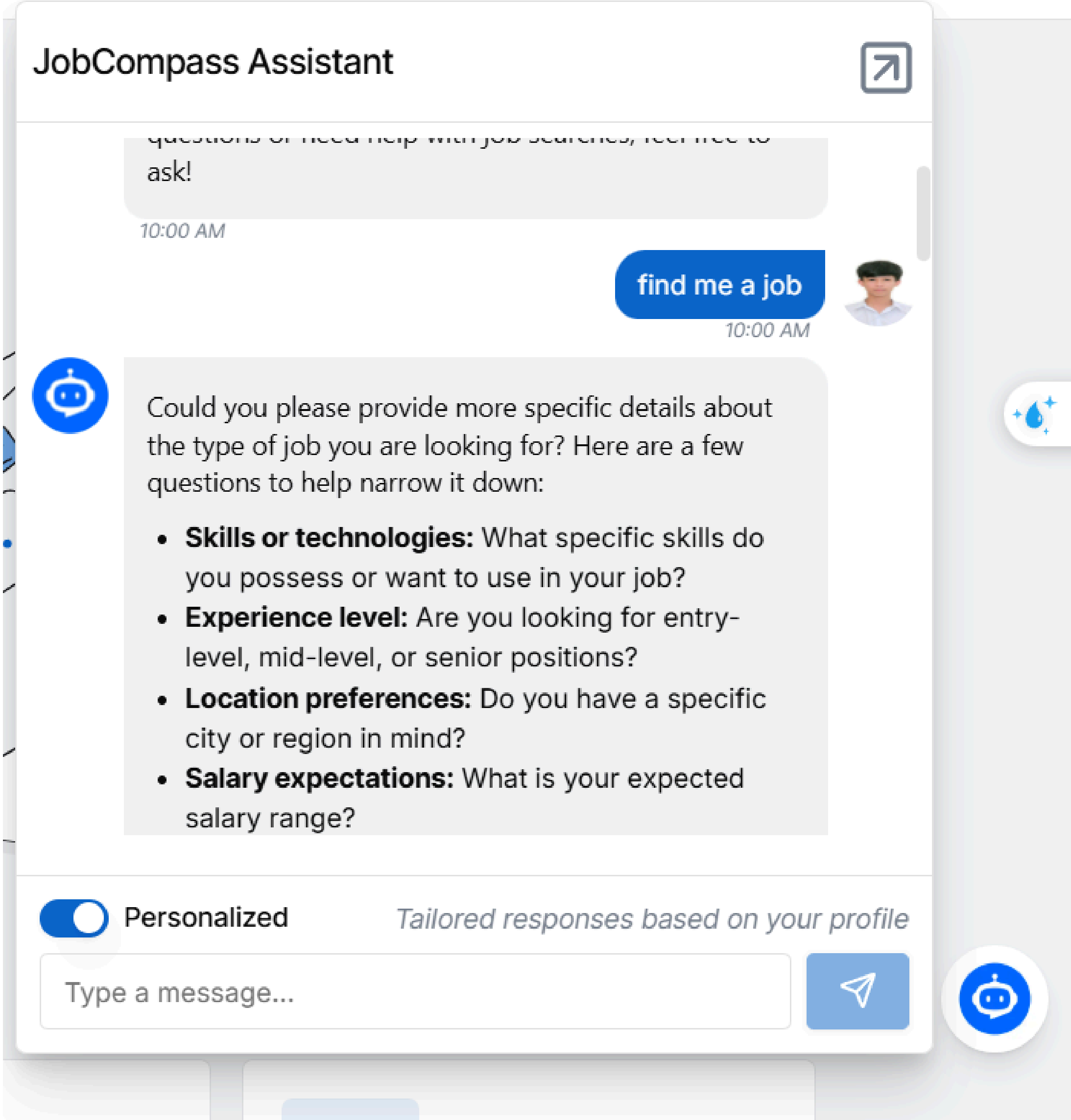

Figure 49: UI chat bot

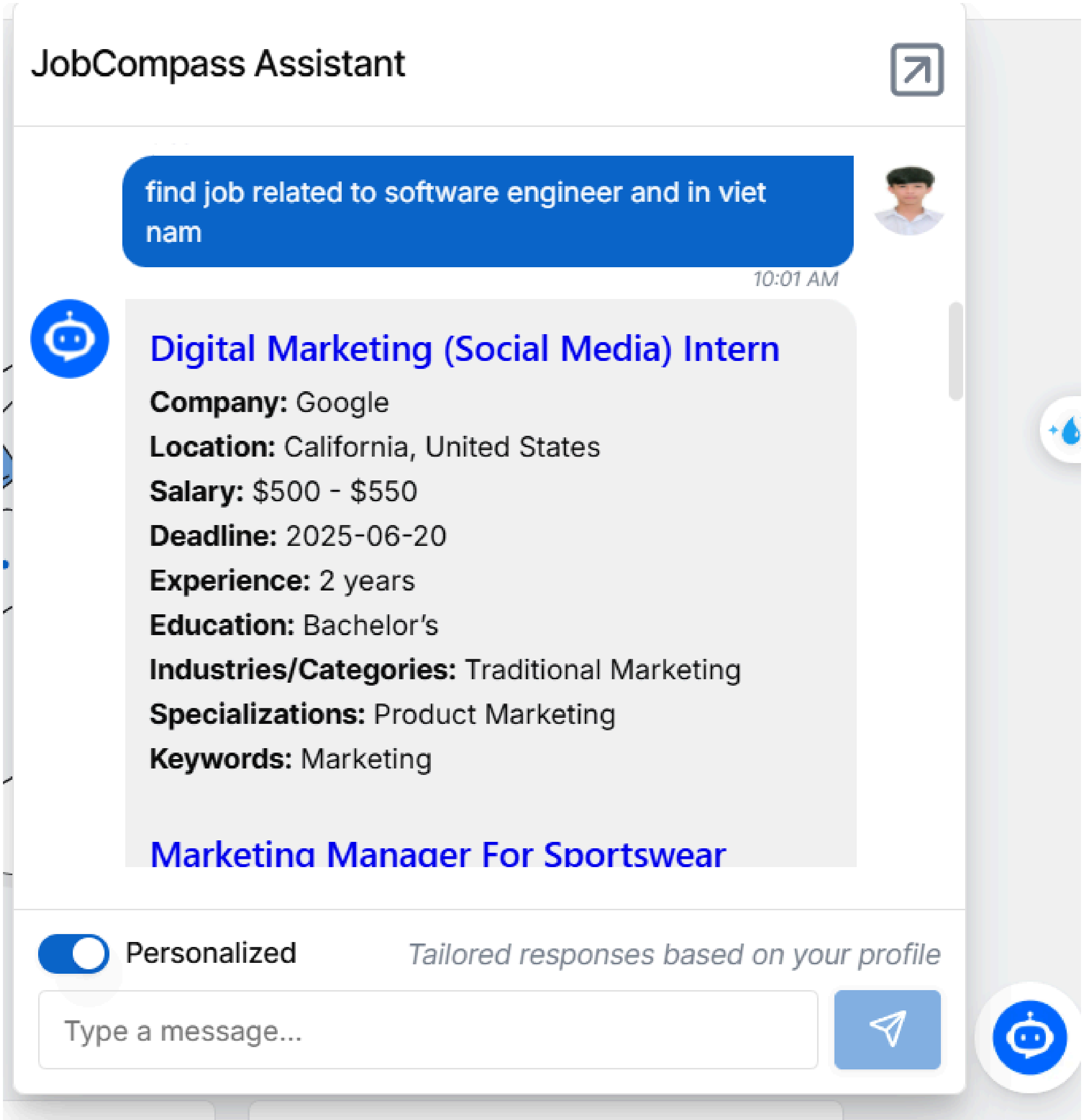

Figure 50: Find job follow user experience

0.7.3 UI for enterprise role

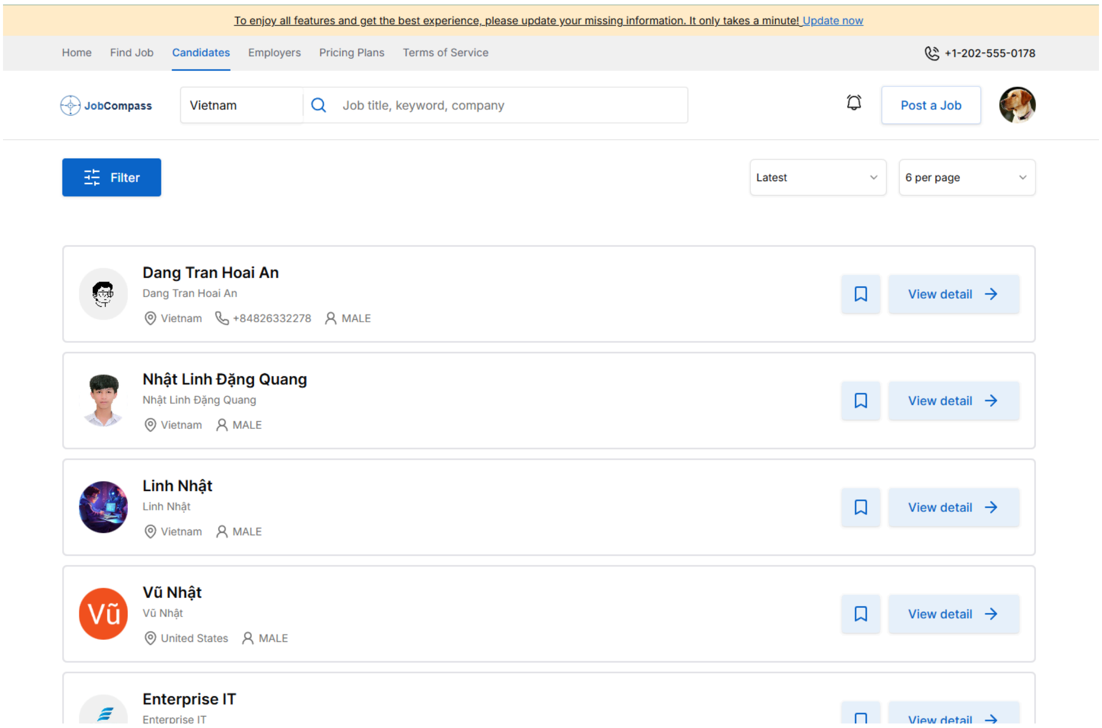

Figure 51: UI candidate list

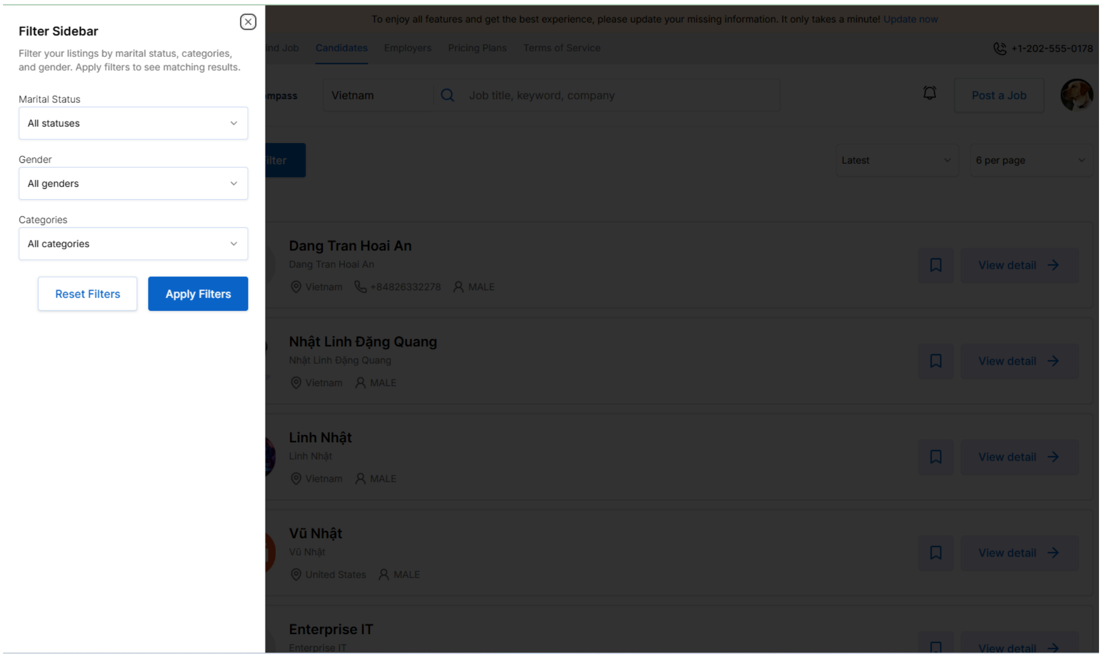

Figure 52: UI for filtering user

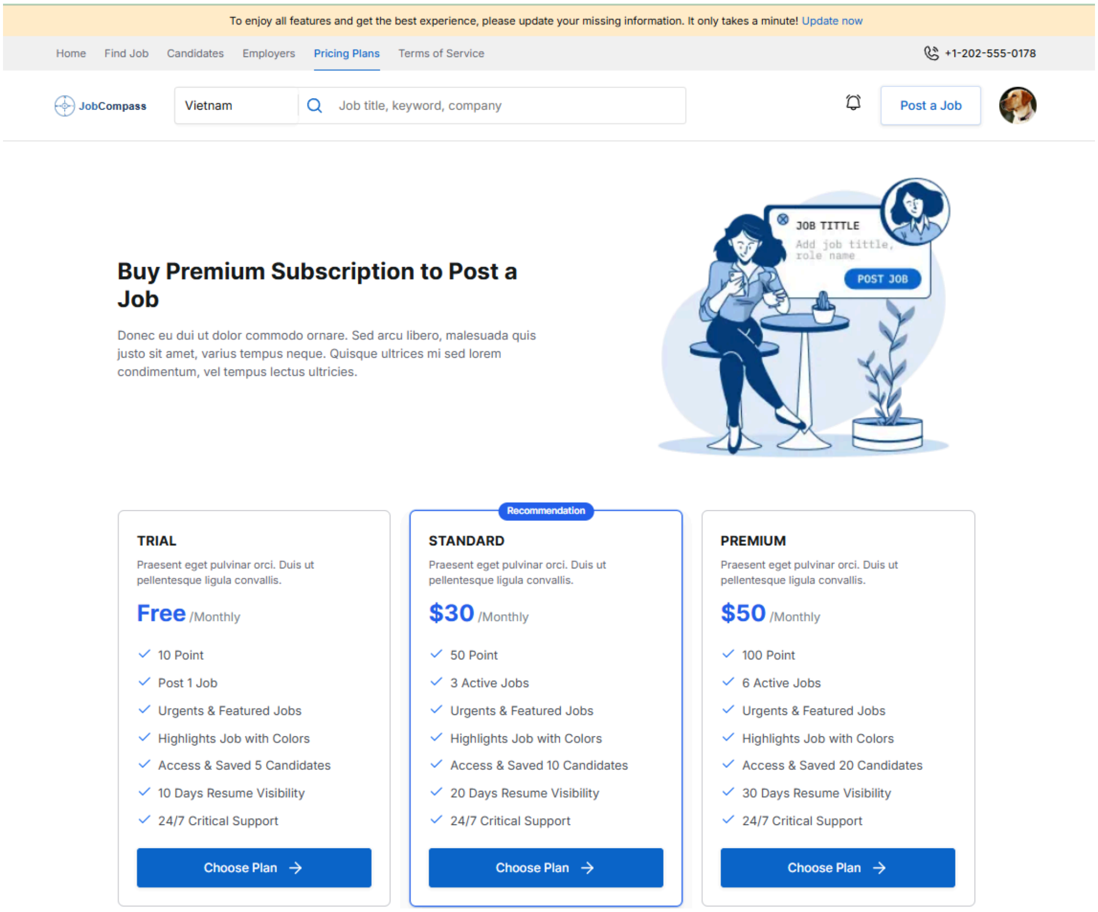

Figure 53: UI for pricing plan

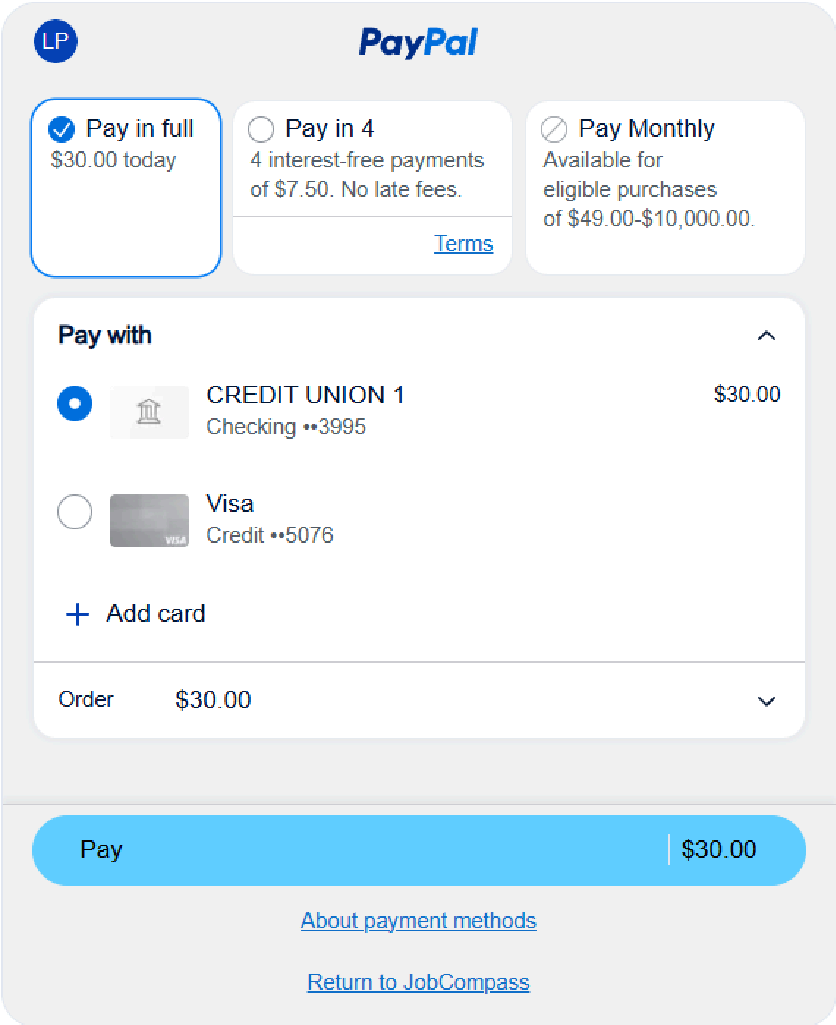

Figure 54: UI for complete payment

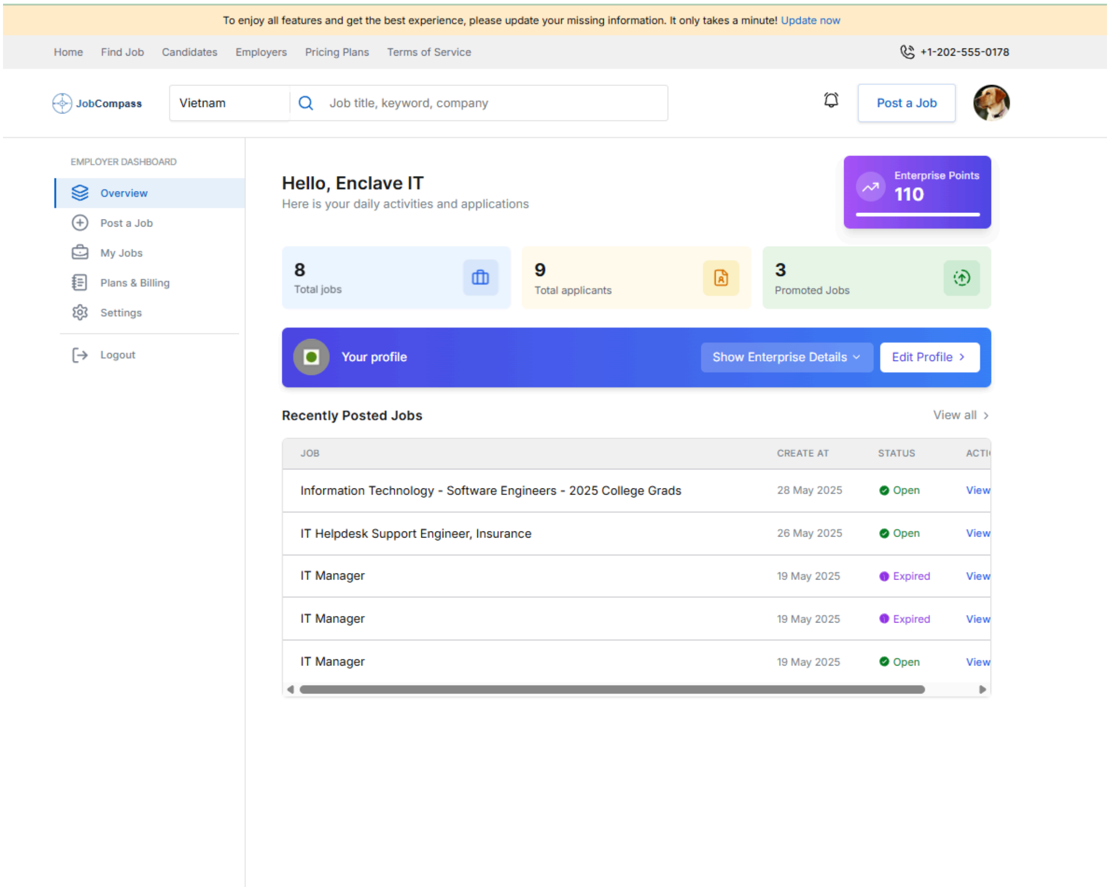

Figure 55: UI for enterprise overview

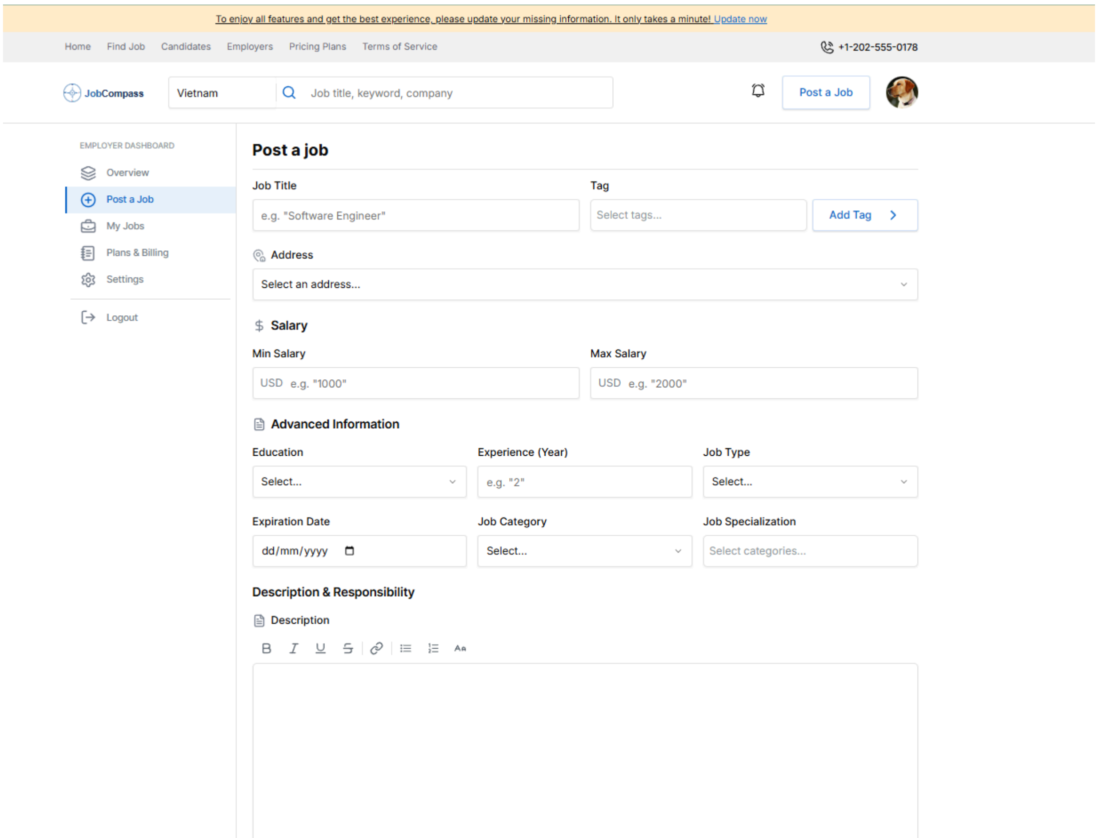

Figure 56: UI for posting job

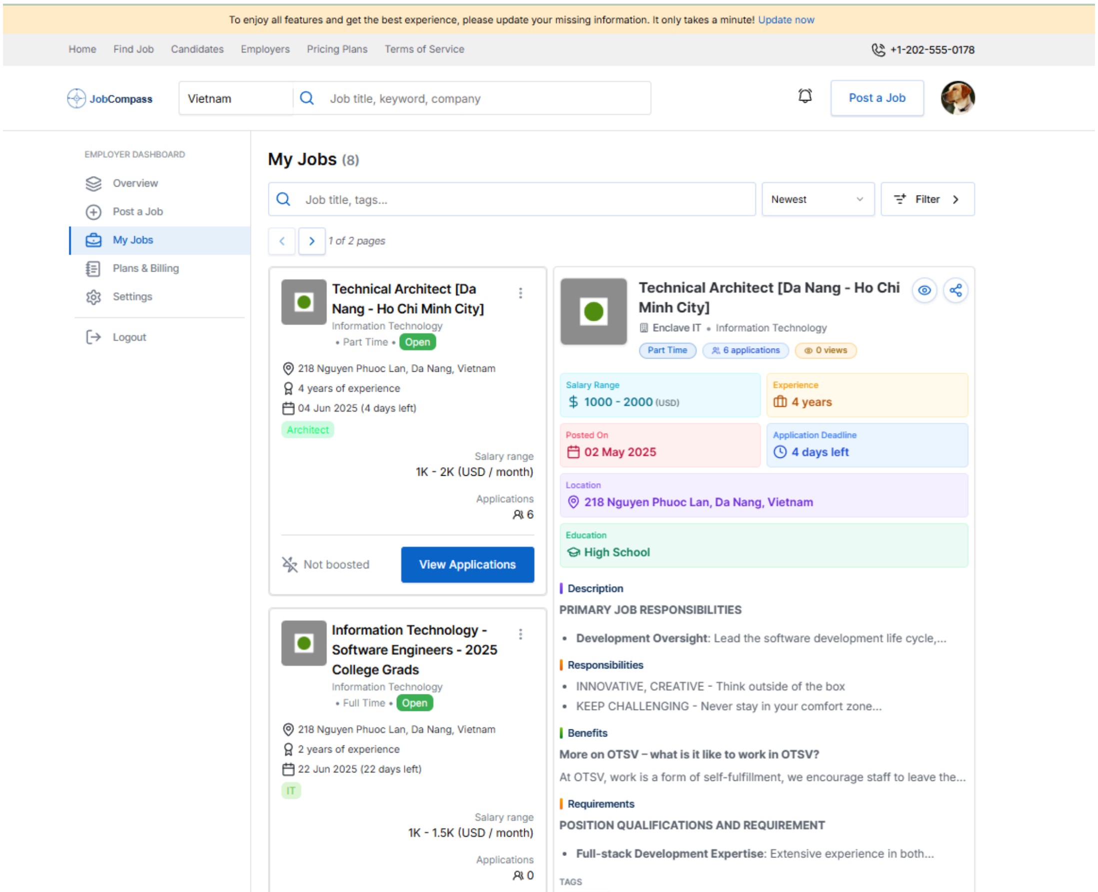

Figure 57: UI for managing job

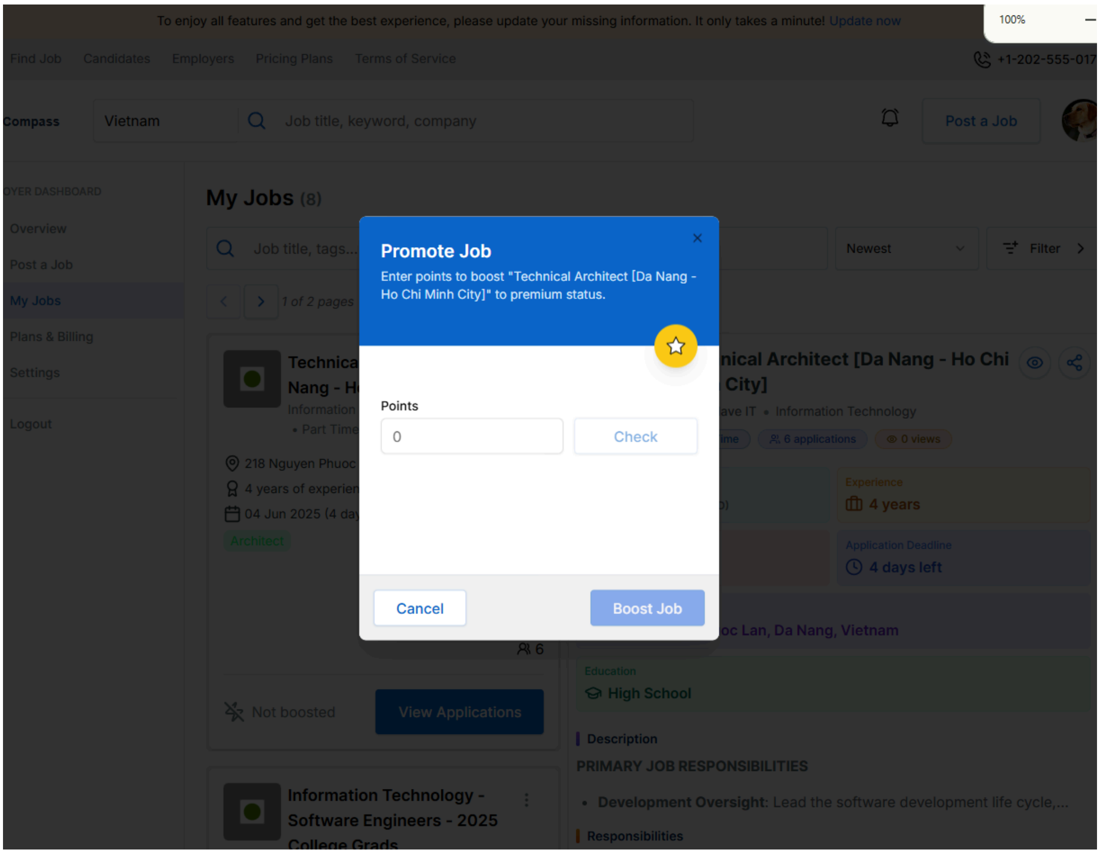

Figure 58: UI for boosting job

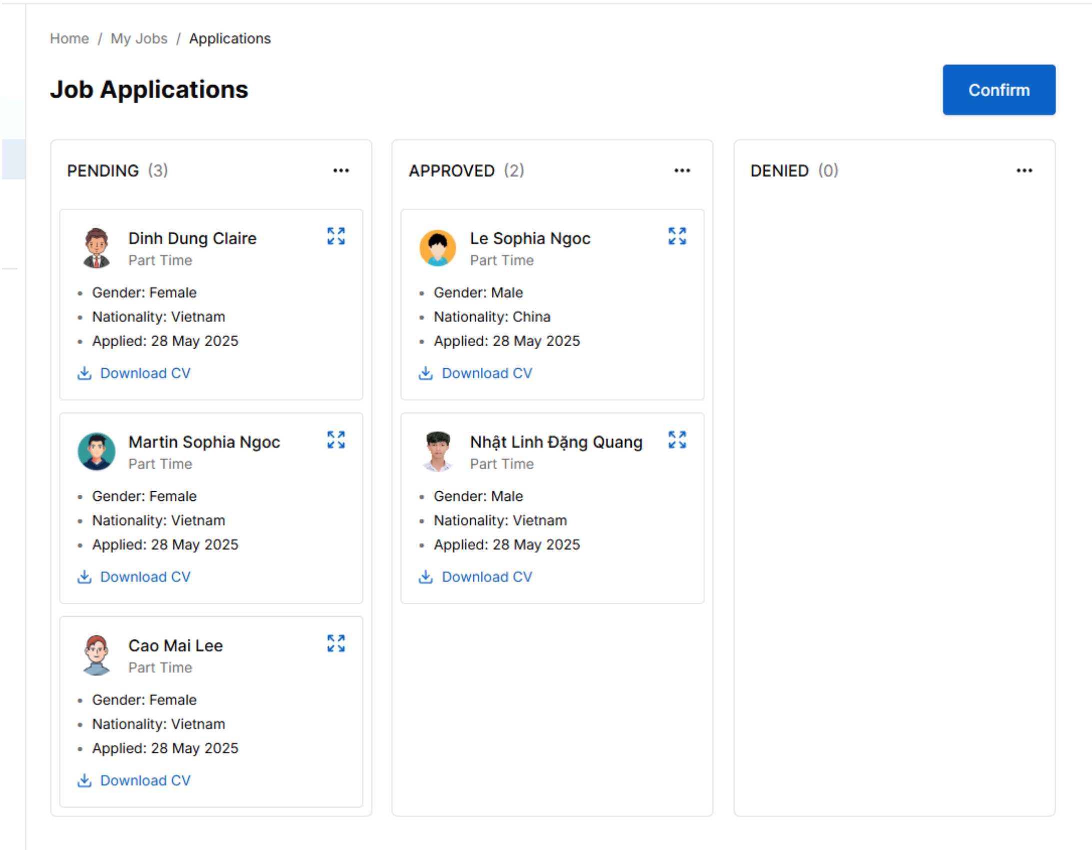

Figure 59: UI for managing applications

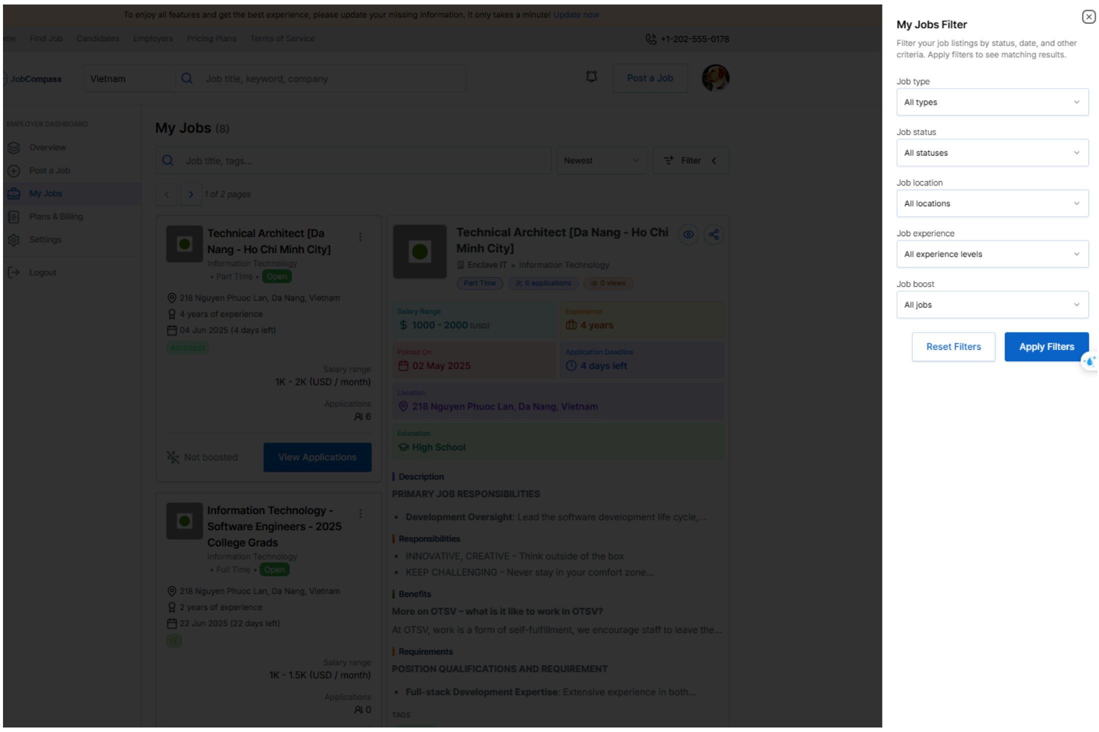

Figure 60: UI for filter job

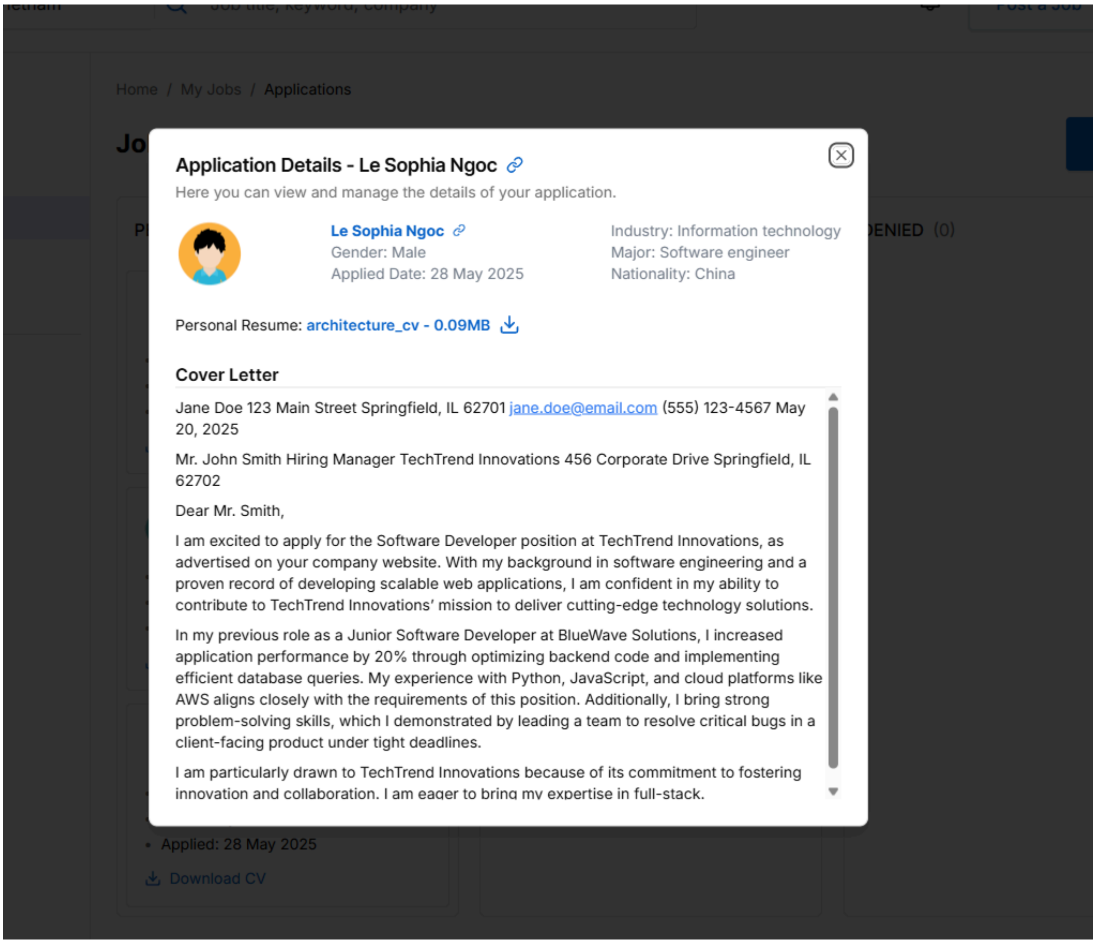

Figure 61: UI for application detail

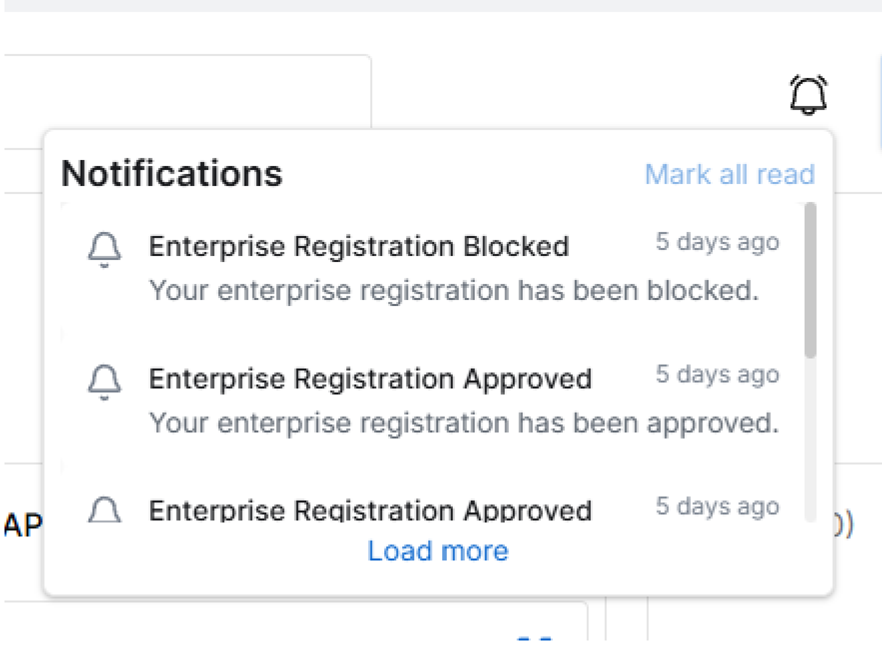

Figure 62: UI for notifications

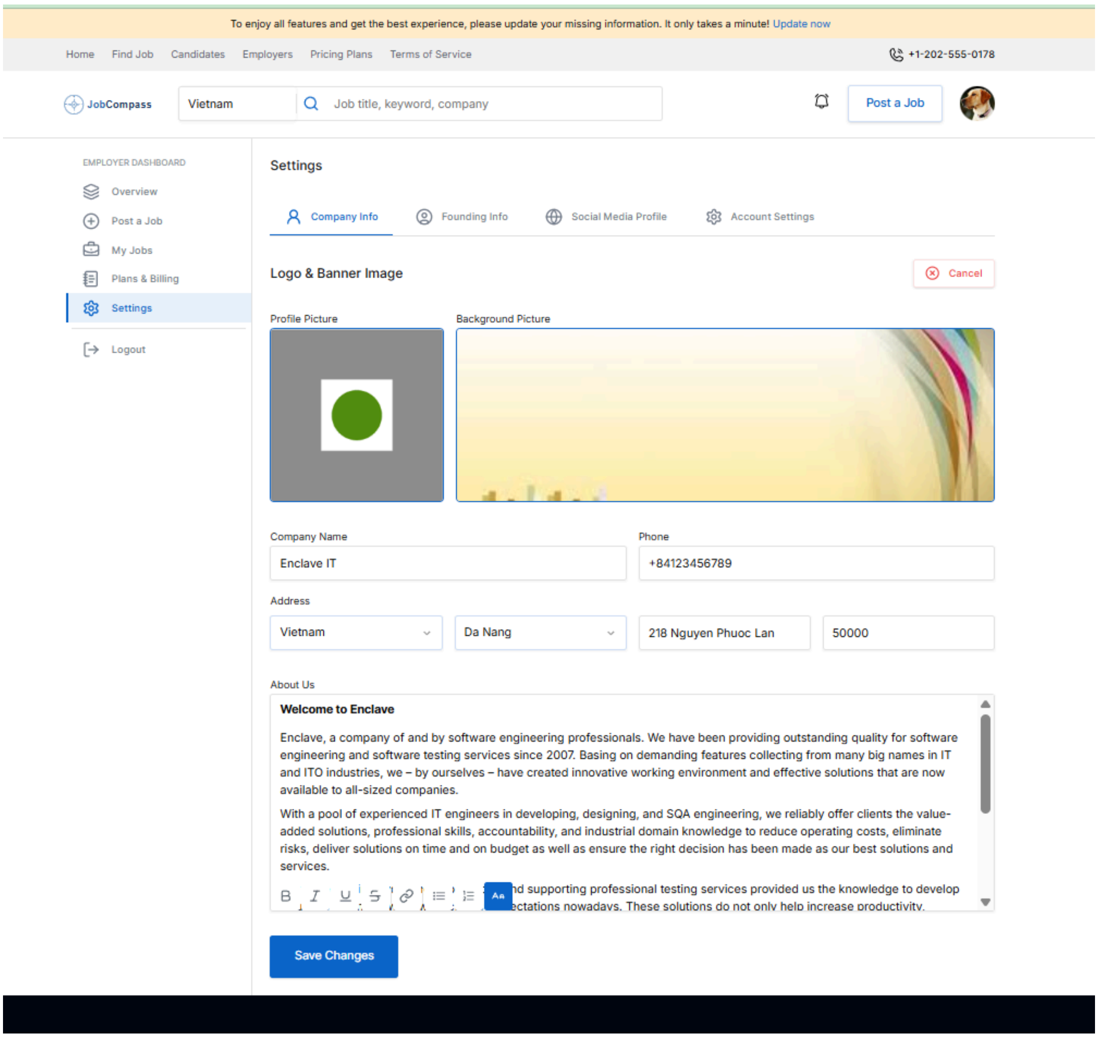

Figure 63: UI for setting enterprise

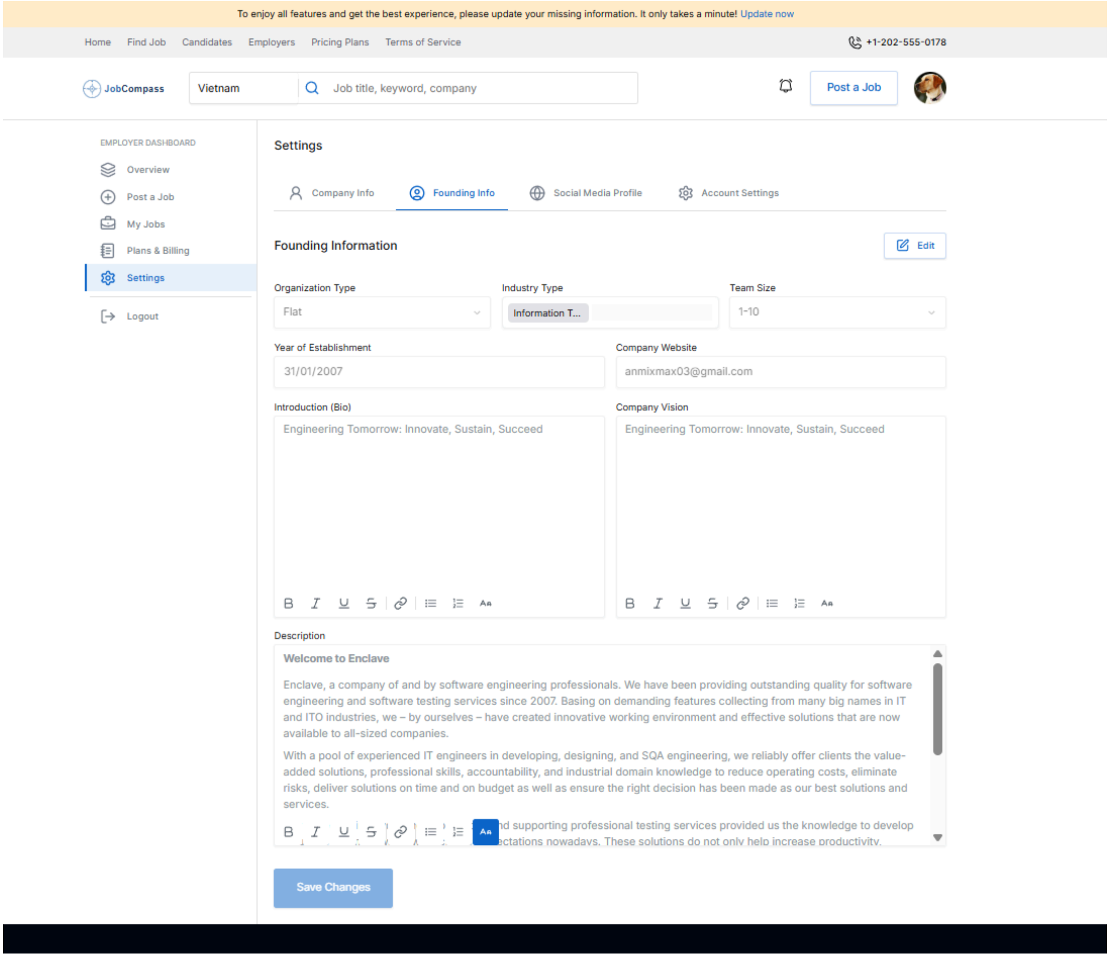

Figure 64: UI for setting up a founding enterprise

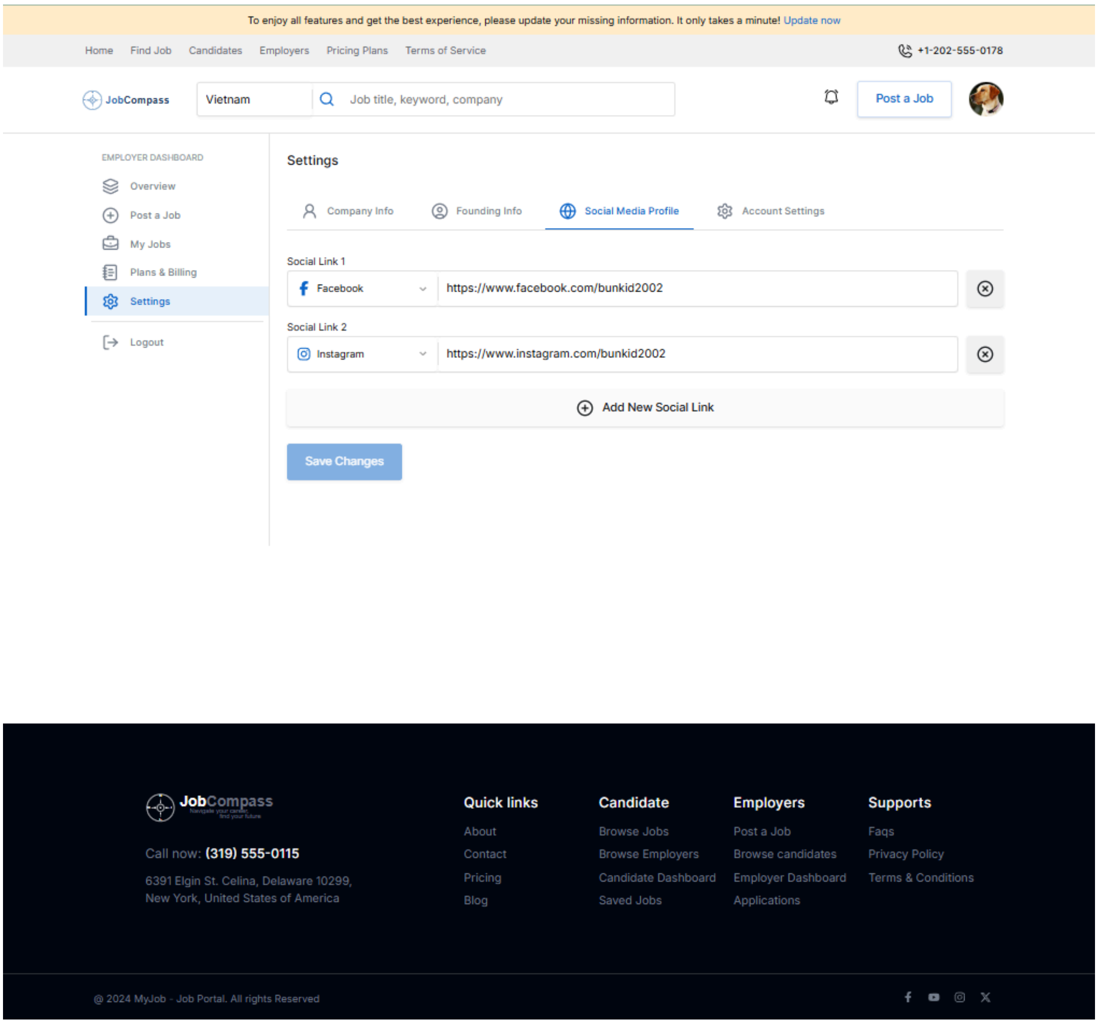

Figure 65: UI for social networking

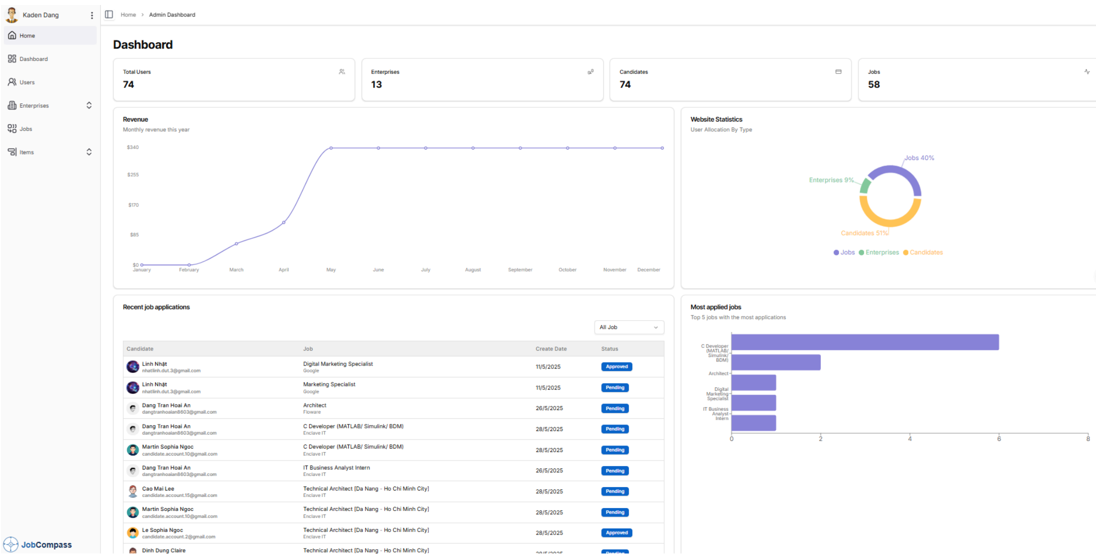

Figure 66: UI for admin dashboard

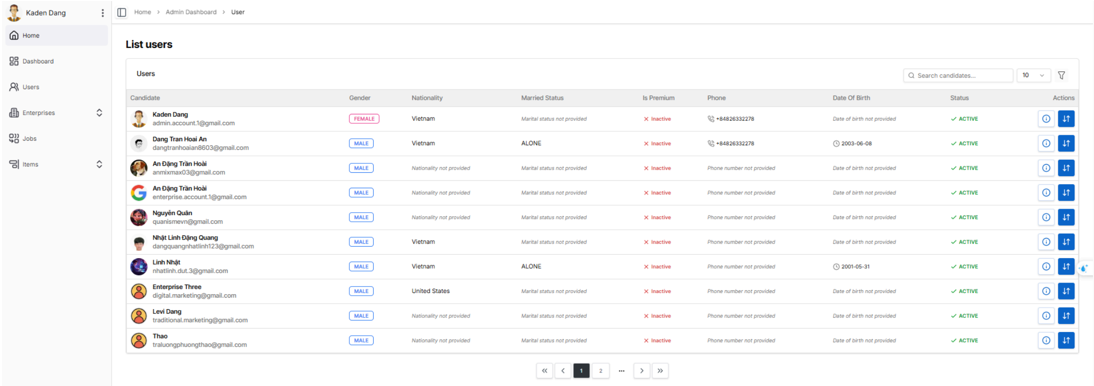

Figure 67: UI for listing users

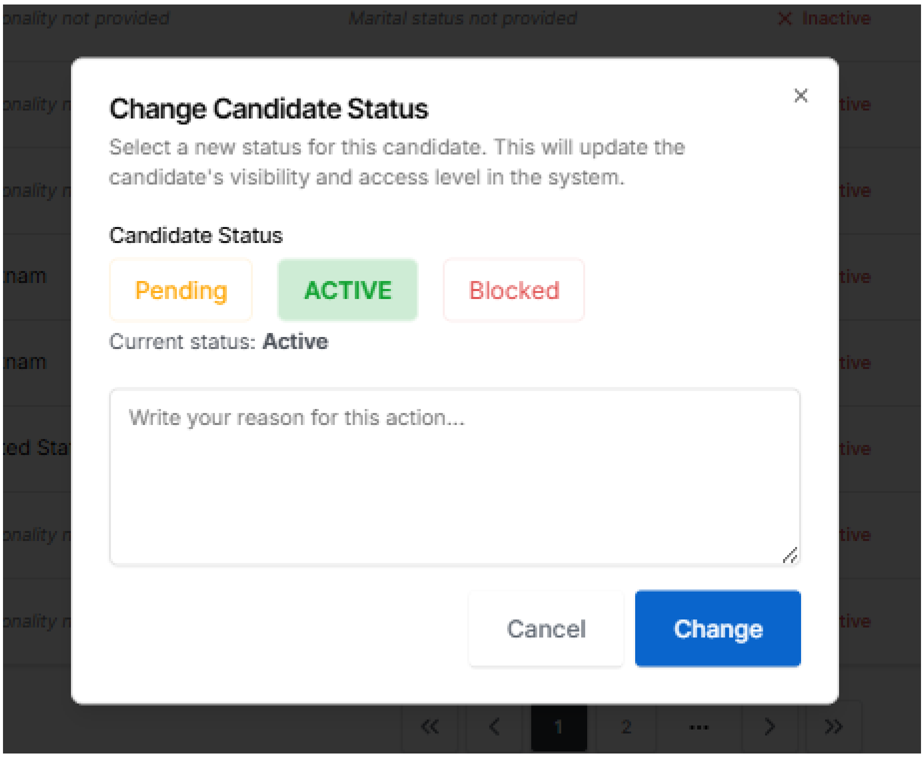

Figure 68: UI for change status

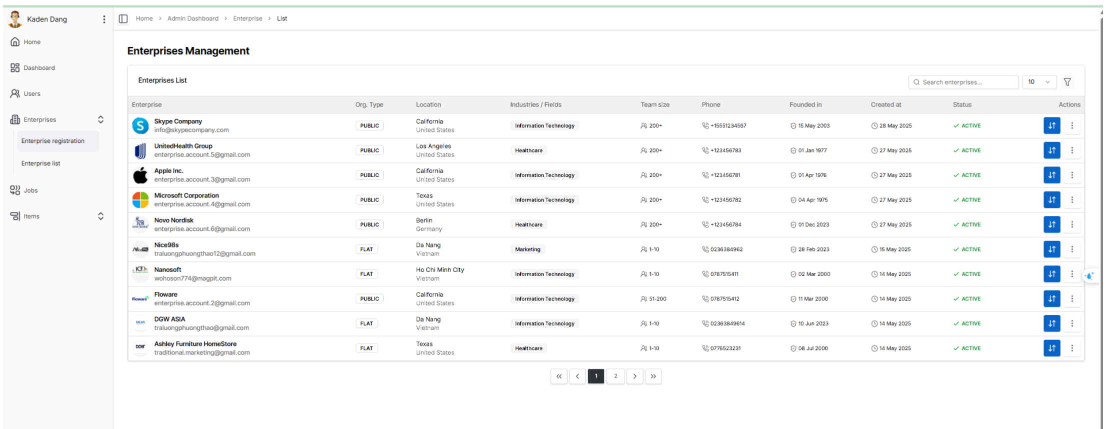

Figure 69: UI for managing enterprise

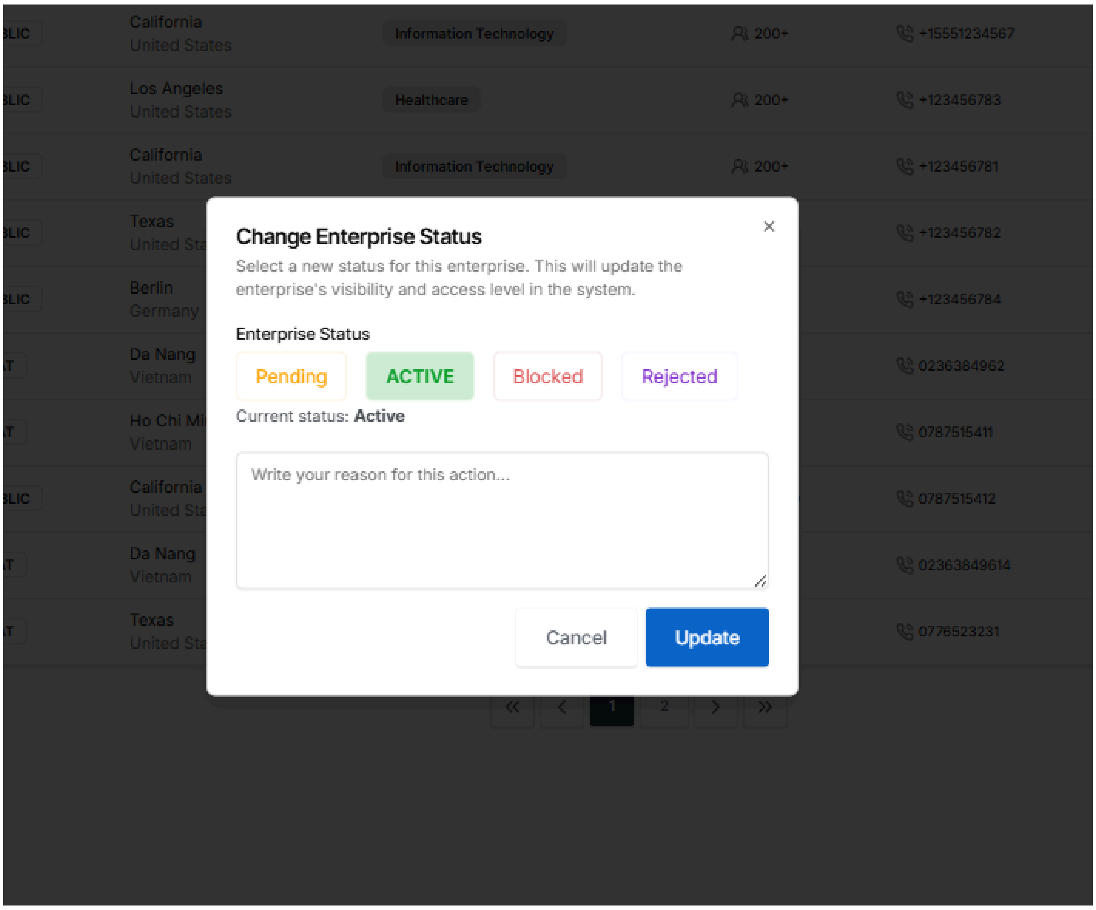

Figure 70: UI for change status enterprise

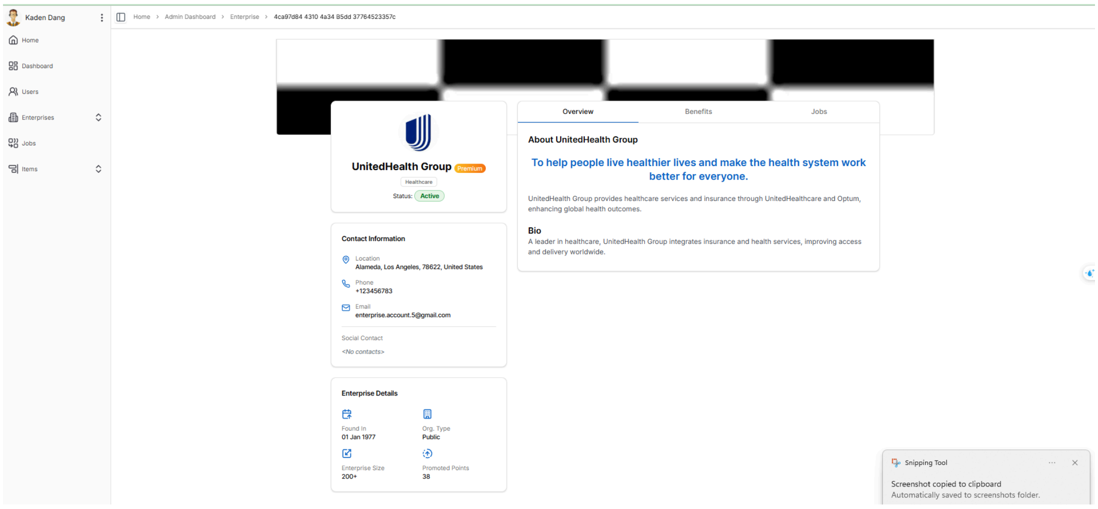

Figure 71: UI for enterprise detail

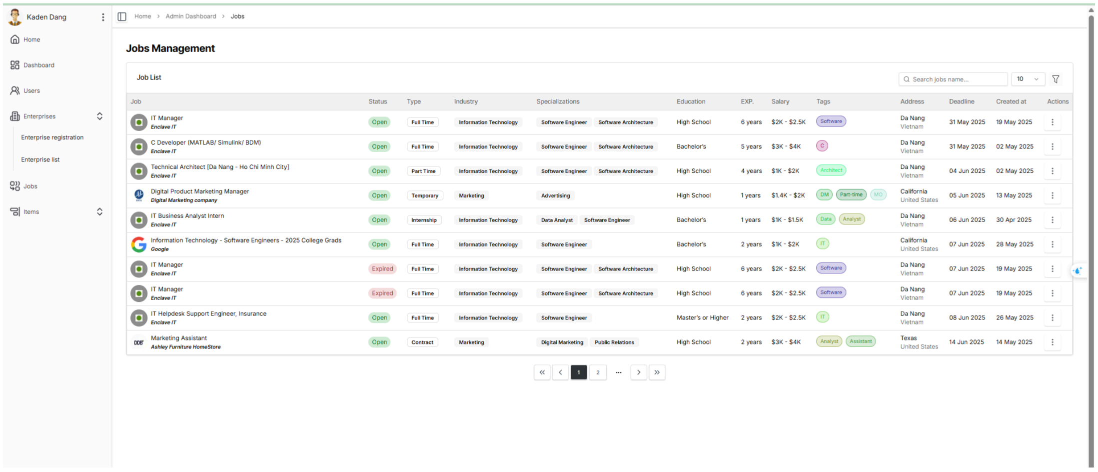

Figure 72: UI for list job

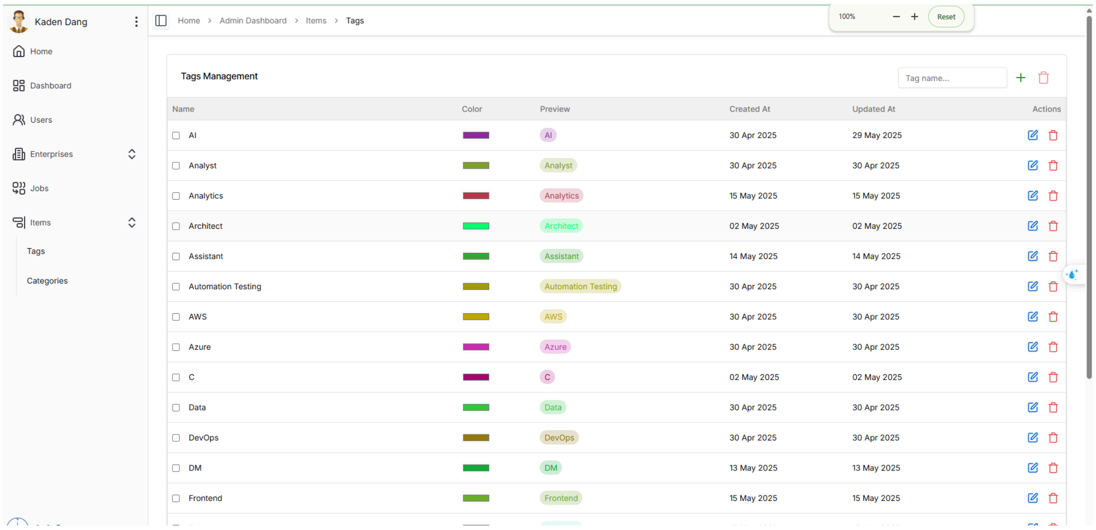

Figure 73: UI for managing tag

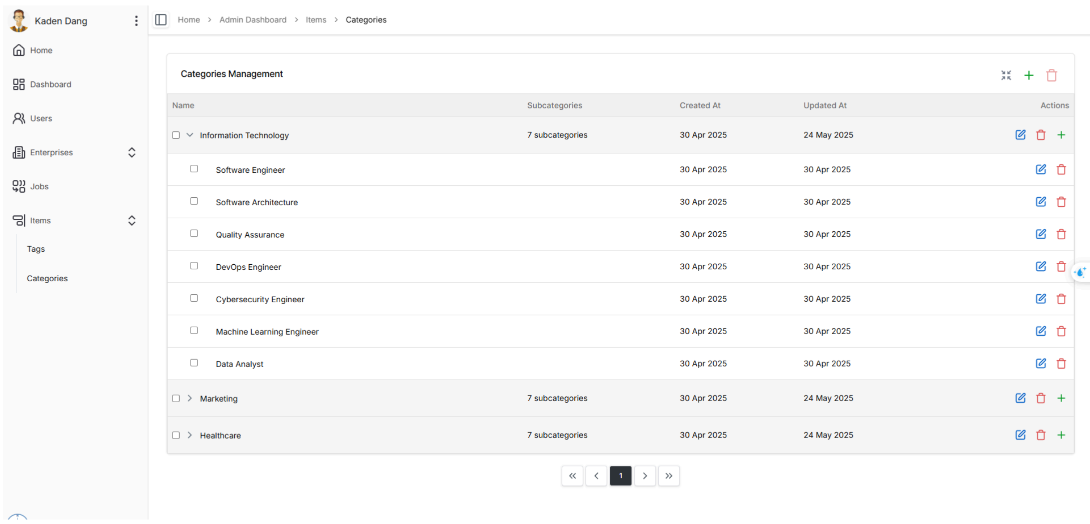

Figure 74: UI for managing category

0.7.4 Notifications outside

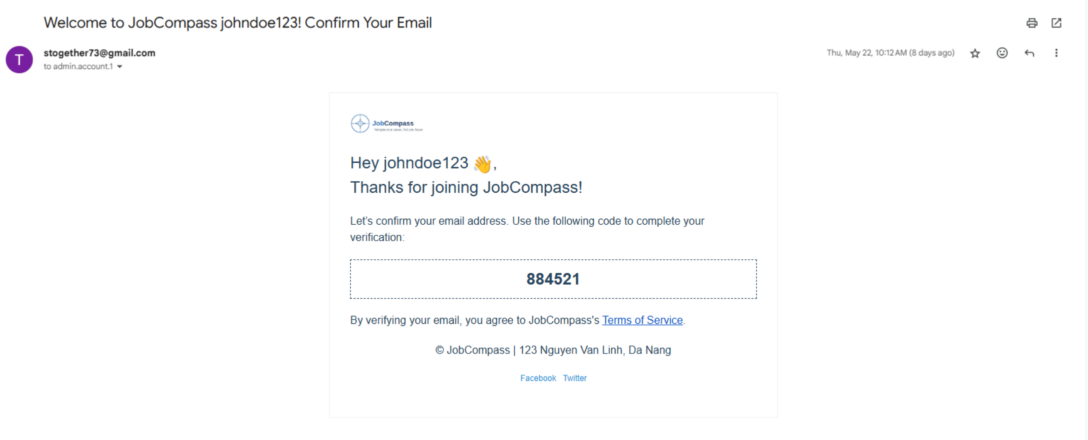

Figure 75: UI for code verification email

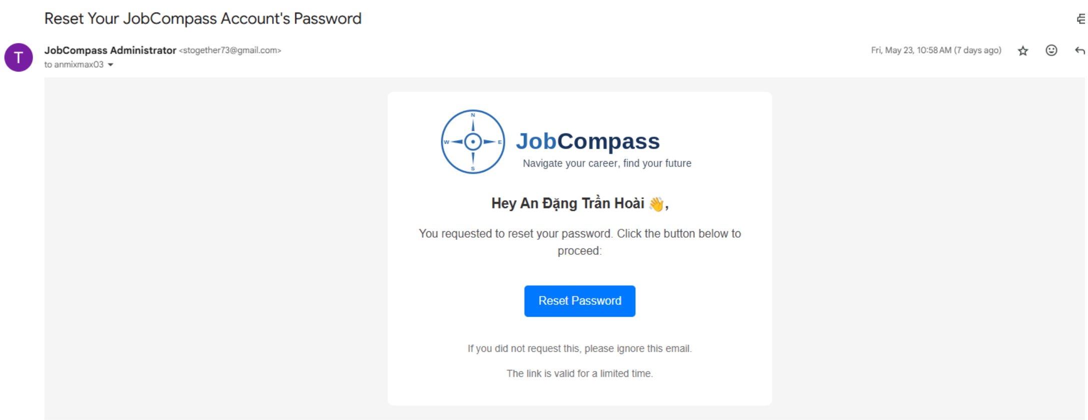

Figure 76: UI for reset password

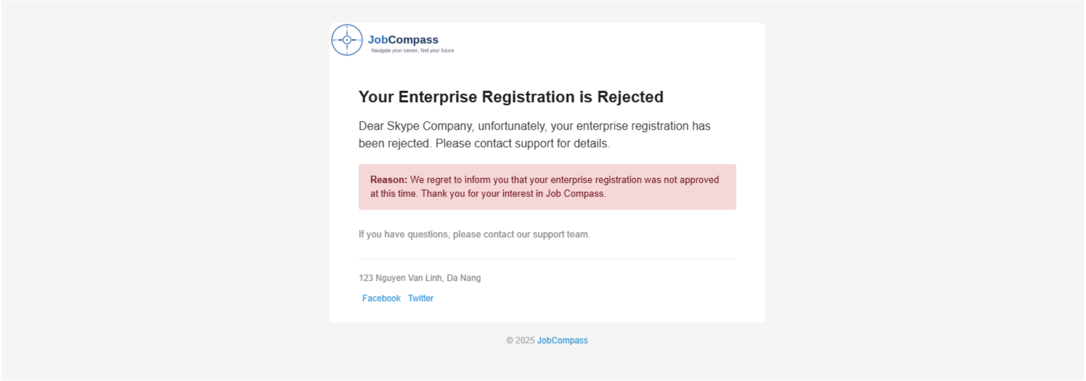

Figure 77: UI for admin reject form enterprise registration

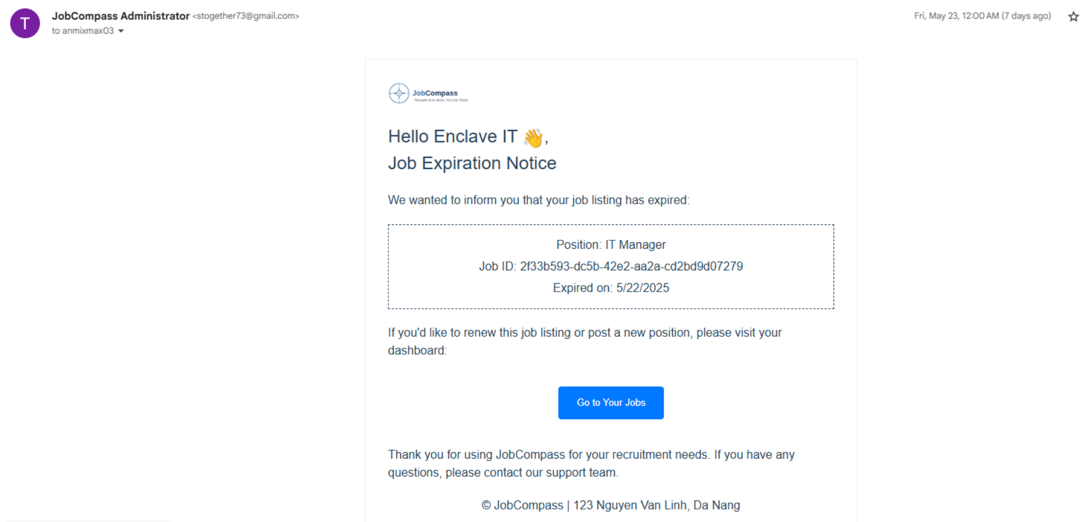

Figure 78: UI for job expired

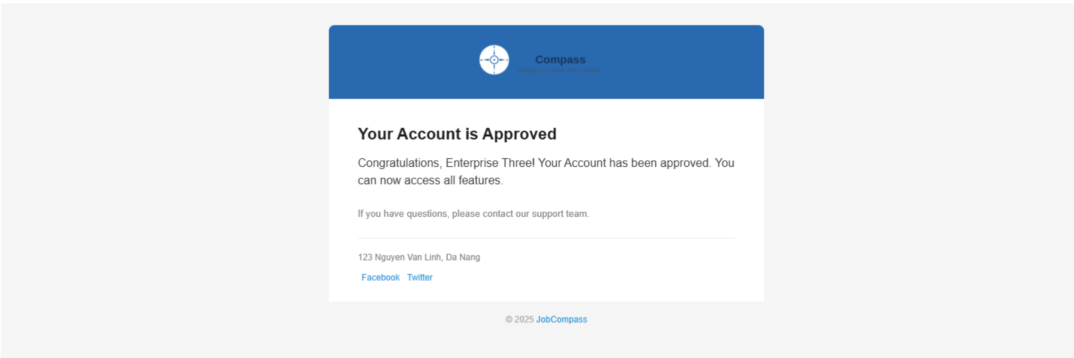

Figure 79: UI when admin accepted enterprise form

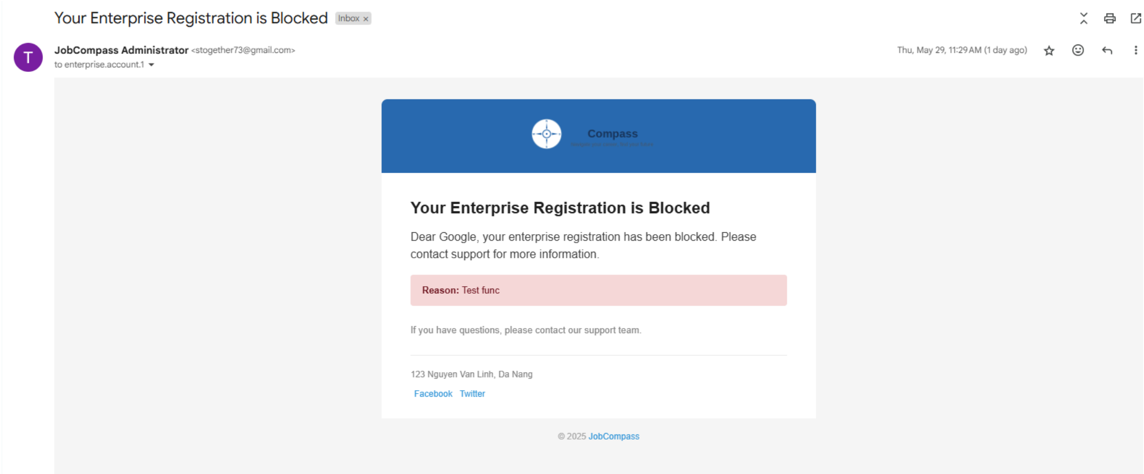

Figure 80: UI when admin block enterprise

0.8 Evaluation

0.8.1 The effectiveness of TF-IDF algorithms

The TF-IDF-based model was developed to extract meaningful keywords from job-related text data and assist in job classification and relevance scoring. The model was assessed based on the quality of keyword extraction, alignment with actual job categories, and its utility in downstream tasks such as industry prediction and semantic search.

Preprocessing Strategy

```
df['processed_text'] = df.apply(lambda row: ' '.join([
    preprocess_text(row['Job Title']) * 6,
    preprocess_text(row['Job Description']),
    preprocess_text(row['Job Requirements']),
    preprocess_text(row['Industry']) * 8,
    preprocess_text(row['Career Level']) * 2,
    preprocess_text(row['Job Type'])
]), axis=1)
```

Weighting rationale:

- **Industry**: Multiplied by 8 — to emphasize the job domain strongly.
- **Job Title**: Weighted 6 times — as a concise summary of the position.
- **Career Level**: Given moderate emphasis with a weight of 2 — indicating seniority or experience level.
- **Job Description, Requirements, and Job Type**: Included with default weight — to retain general context.

Figure 81: Word cloud

Displays the most prominent keywords extracted using TF-IDF. Dominant terms such as *"product"*, *"customer"*, *"experience"*, and *"marketing"* suggest that the algorithm effectively captured key themes present in the job postings.

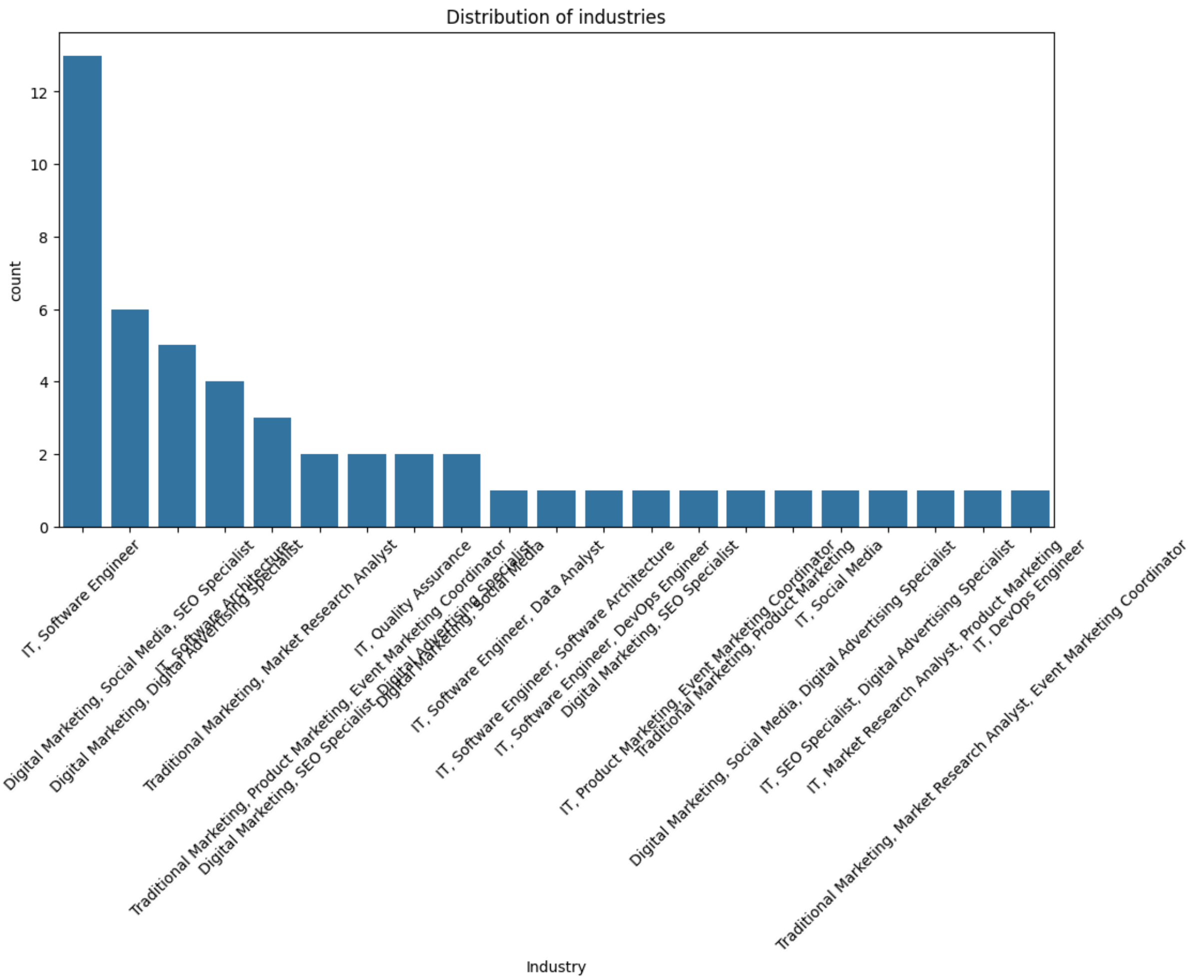

Figure 82: Industry type distribution

A bar chart showing the frequency of job postings across various industries. The consistency between frequently occurring industries (e.g., *"IT, Software Engineer"*, *"Digital Marketing"*, *"SEO Specialist"*) and the top TF-IDF keywords indicates strong alignment between extracted terms and actual domain representation.

0.8.2 The embedding models

In the model evaluation phase of our recommendation system, we selected and tested several text embedding models based on key criteria such as retrieval performance, inference cost, and deployability. Notably, the **Snowflake/snowflake-arctic-embed-m** model was chosen as the primary candidate due to its outstanding performance and balanced trade-off between accuracy and computational efficiency.

Model Description

Snowflake Arctic Embed is a family of embedding models developed by Snowflake, optimized for text retrieval tasks. The **arctic-embed-m** model, with 110 million parameters and a 768-dimensional embedding size, achieves a

NDCG@10 score of 54.90 on the MTEB (Massive Text Embedding Benchmark) leaderboard. This outperforms many well-known models such as bge-base-en-v1.5 (53.25) and gte-base (52.31).

The model is trained using a multi-stage pipeline:

- **Pretraining** on 400 million query-document pairs using in-batch negative sampling.
- **Fine-tuning** on around 1 million triplets (query, positive, negative) with hard-negative mining to improve accuracy.
- The training data includes both open-source corpora and proprietary web search data, enhancing the model's ability to handle natural user queries effectively.

Table 16: Performance Evaluation

Model	**MTEB NDCG@10**	**Parameters (M)**	**Embedding Dim**
Arctic-embed-m	**54.90**	110	768
bge-base-en-v1.5	53.25	~110	768
nomic-embed-text-v1.5	53.25	~100	768
GIST-Embedding-v0	52.31	~90	768
gte-base	52.31	~110	768

The results show that Arctic-embed-m clearly outperforms comparable models in its class, proving its effectiveness in encoding both queries and documents for retrieval tasks.

0.8.3 The LLMs

In the large language model (LLM) selection process, the key criteria were: reasoning ability, response latency, context window, deployment flexibility (cloud or local), and cost-effectiveness for production use. This section evaluates leading LLMs suitable for inference and reasoning tasks in our system, with a special focus on **GPT-4o mini**, OpenAI's newest lightweight model released in 2025.

Table 17: Other LLMs Evaluated

Model	**Provider**	**Context**	**Cost ($/1k tokens)**	**Notes**
GPT-4o	OpenAI	128k	~$0.005–0.015	State-of-the-art, best overall

Model	Provider	Context	Cost ($/1k tokens)	Notes
GPT-3.5-turbo	OpenAI	16k	~$0.001–0.002	Best value for moderate tasks
Mistral 7B/8x7B	Mistral AI	32k	Free (self-hosted)	Fast, cost-free for local use
LLaMA 3 (8B / 70B)	Meta	8k–32k	Free (self-hosted)	Competitive with GPT-3.5 when fine-tuned

Conclusion

GPT-4o mini was selected as the default LLM for the current phase of our system due to its:

- Strong balance of reasoning ability and speed.
- Compatibility with OpenAI's API and embedding models.
- Significantly lower cost compared to GPT-4, while still offering high-quality answers.

For open-source or offline deployments, **LLaMA 3 (8B or 70B)** or **Phi-3 Mini** may be considered in future iterations, especially for privacy-sensitive or resource-constrained environments.

0.8.4 Advantages

In terms of UI and UX, my application provides a simple UI with a good UX, so users find it easy full-featured experience.

Regarding functionalities, Time to implement and complete the process easily and quickly (in line with the criteria of shortening time and traditional recruitment process) the system provides all necessary features for users, which is declared in initial requirements:

- Job seekers must complete registration operations to have a user account in the system.
- Job seekers must login into the system with a valid account to use the features for job seekers.
- The system suggests jobs for candidates based on user behavior.
- Candidates can search for job information by name, location, salary, job.

- Every time, candidate apply for a job, this CV will be automatically submitted to that job posting. They may be able to change the CV uploaded to the system if needed.
- Job seekers can see the recent job view, favorite job, applied job.
- They can also see the status of that job and will be notified if it is approved or rejected by the Enterprise or when the job has ended the recruitment period.

0.8.5 Disadvantages

Besides the advantages mentioned above, our website still has some disadvantages:

First, The UI on the web application does not well Since, in some page I only focus on full screen responsive. Furthermore, the number of jobs data is quite small.

Secondly, the data to serve the recommender system is quite a bit small so that the model of recommender may perform well according to the required problem which was introduced before. Nevertheless, when the data get bigger in the future, the model will be facing some problems. In addition, the application has not been strongly popularized for cooperation yet and with the development of technology in 4.0 area the connection and unity between the systems is inevitable.

Finally, the system is only in charge of maximum 100 connection at the same time. When the number connection exceeds at that time, the system may occur overloaded server.

Chapter 4 CONCLUSION AND FUTURE WORK

0.9 Result

Through the completion of this project, I have gained valuable knowledge and practical experience from both research and experimentation.

Theoretical Knowledge

- Have basic knowledge about Machine learning.
- Have knowledge of system analysis and design programming, libraries, framework.
- Have experience in using Framework and language, version control system
- I have knowledge about recommendation algorithms.
- In practical term, I have created an application operating both mobile and web platform which helps to resolve the current employment issue.

Practical Achievements

- Successfully developed an application that operates on both mobile and web platforms, aiming to address current employment challenges.

0.10 Future work

As outlined in the evaluation section, there are still some limitations in the current system. Future improvements for this project may include:

- Strengthening collaboration with businesses to implement more professional and authoritative features, including a comprehensive customer support system.
- Integrating AI-powered CV assessment to assist enterprises in evaluating candidates more efficiently.
- Adding an online testing and interview system to enhance the connection between candidates and companies, making the recruitment process faster and more streamlined.

REFERENCES

1. https://www.geeksforgeeks.org/understanding-tf-idf-term-frequency-inverse-document-frequency/
2. https://viblo.asia/p/rag-ai-agents-va-agentic-rag-chung-la-gi-va-hoat-dong-nhu-the-nao-vlZL9KjW4QK
3. https://www.datacamp.com/tutorial/agentic-rag-tutorial
4. https://nextjs.org/
5. https://nestjs.com/
6. https://www.langchain.com/
7. https://flask.palletsprojects.com/en/stable/
8. https://viblo.asia/p/tim-hieu-ve-word-embedding-Ljy5VD7kZra